Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»

Кафедра информационных систем и программной инженерии

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту по дисциплине «Распределенные программные системы» на тему

Прототип программной системы, автоматизирующей сборку операционной системы Linux для микрокомпьютеров Luckfox

Выполнил:
студент гр. ПРИ-123
Жданов А.И.

Принял:
ст. пр. кафедры ИСПИ
Тимофеев А.А.

Владимир, 2026

АННОТАЦИЯ

Разработан прототип распределённой программной системы автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico. Спроектированы и реализованы пользовательский интерфейс на React, модель данных и база данных MySQL, аутентификация по JWT с ролевым разграничением доступа, каталоги образов и компонентов и административные функции. Проведено тестирование, по итогам которого оптимизация доступа к базе данных повысила пропускную способность в 2,6 раза.

Пояснительная записка: 124 с., 24 рис., 44 табл., 13 листингов, 23 источника, 8 прил.

ABSTRACT

A prototype of a distributed software system for automated building of Linux images for Luckfox Pico microcomputers was developed. The React user interface, the data model and MySQL database, JWT authentication with role-based access control, image and component catalogs and administrative functions were designed and implemented. Testing was performed, and the database access optimization increased throughput 2.6 times.

Explanatory note: 124 pages, 24 figures, 44 tables, 13 listings, 23 references, 8 appendices.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К СИСТЕМЕ 7
1.1 Назначение системы, цель и задачи проекта 7
1.2 Особенности создания Linux-образов для микрокомпьютеров Luckfox Pico 8
1.3 Анализ существующих решений и обоснование разработки 11
1.4 Словарь предметной области 14
1.5 Пользователи системы и их роли 15
1.6 Функциональные требования к системе 16
1.7 Нефункциональные требования к системе 18
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ 21
2.1 Архитектура клиентской и серверной частей приложения 21
2.2 Информационная модель и проектирование базы данных 24
2.3 Аутентификация, управление пользователями и разграничение доступа 27
2.4 Каталоги образов и компонентов и административные функции 31
2.5 Пользовательский интерфейс и структура навигации 33
2.6 Контракты взаимодействия клиентской и серверной частей 35
3 РЕАЛИЗАЦИЯ ПРОТОТИПА ПРОГРАММНОЙ СИСТЕМЫ 38
3.1 Реализация базы данных и слоя доступа к данным 38
3.2 Реализация аутентификации и управления пользователями 42
3.3 Реализация каталогов и административных функций 46
3.4 Реализация пользовательского интерфейса и навигации 48
3.5 Взаимодействие клиентского приложения с REST API 54
3.6 Конфигурация и развёртывание приложения 56
4 ТЕСТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ 61
4.1 Тестовый стенд, методика и критерии оценки 61
4.2 Модульное тестирование 62

4.3 Интеграционное тестирование серверной части 64
4.4 Сквозное тестирование пользовательских сценариев 65
4.5 Проверка конвейера сборки и отказоустойчивости 65
4.6 Нагрузочное тестирование и анализ производительности 67
4.7 Анализ выявленных дефектов и оптимизация системы 68
4.8 Оценка результатов и ограничения тестирования 72
5 ОРГАНИЗАЦИЯ РАБОТЫ НАД ПРОЕКТОМ 73
5.1 Распределение задач и зон ответственности 73
5.2 Организация совместной разработки и контроль версий 74
5.3 Календарный план выполнения работ 76
5.4 Использование инструментов искусственного интеллекта 77
ЗАКЛЮЧЕНИЕ 79
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 81
ПРИЛОЖЕНИЕ А РАСШИРЕННЫЕ ОПИСАНИЯ ПРЕЦЕДЕНТОВ 83
ПРИЛОЖЕНИЕ Б ДИАГРАММЫ ПРЕЦЕДЕНТОВ И БИЗНЕС-ПРОЦЕССОВ 90
ПРИЛОЖЕНИЕ В ДИАГРАММЫ КЛАССОВ И СОСТОЯНИЙ 92
ПРИЛОЖЕНИЕ Г СХЕМЫ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ И РАЗВЁРТЫВАНИЯ СИСТЕМЫ 94
ПРИЛОЖЕНИЕ Д СХЕМА БАЗЫ ДАННЫХ И ОПИСАНИЕ ТАБЛИЦ 96
ПРИЛОЖЕНИЕ Е ДИАГРАММА НАВИГАЦИИ И ЭКРАНЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА 100
ПРИЛОЖЕНИЕ Ж ПРОТОКОЛЫ И СВОДНЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ 105
ПРИЛОЖЕНИЕ И ОСНОВНЫЕ ЛИСТИНГИ ПРОГРАММНОГО КОДА 118

ВВЕДЕНИЕ

Микрокомпьютеры на базе энергоэффективных систем на кристалле всё шире применяются в системах видеонаблюдения, компьютерного зрения и промышленного интернета вещей. Одним из представителей этого класса устройств является линейка плат Luckfox Pico на процессорах Rockchip RV1103 и RV1106, сочетающих малые габариты и стоимость со встроенными средствами обработки видео и изображений [4]. Штатный образ операционной системы, поставляемый производителем, ориентирован на демонстрацию возможностей платы и, как правило, не соответствует требованиям конкретного изделия: в нём отсутствуют необходимые пакеты и библиотеки, а состав системы избыточен для решаемой задачи. Поэтому разработчикам приходится формировать собственные образы, что делает актуальным вопрос упрощения и удешевления этой процедуры.

Создание собственного образа для Luckfox Pico выполняется с помощью официального SDK, построенного на основе системы сборки Buildroot [5], [6]. Для этого необходимо развернуть на локальной машине под управлением Linux среду сборки, занимающую десятки гигабайт дискового пространства, разобраться в структуре конфигурационных файлов и зависимостях между пакетами, а затем дождаться завершения кросс-компиляции, первый запуск которой занимает часы. Указанные трудоёмкость, длительность и высокие требования к ресурсам рабочей станции, а также сложность воспроизведения результата на другой машине образуют основную проблему, на решение которой направлена настоящая работа.

Цель курсового проекта заключается в разработке прототипа распределённой программной системы, позволяющей быстро сконструировать конфигурируемый Linux-образ для микрокомпьютера Luckfox Pico и получить его без запуска ресурсоёмкого процесса сборки на локальном устройстве. Система реализуется как веб-приложение с клиент-серверной архитектурой:

пользователь выбирает компоненты в визуальном редакторе, серверная часть автоматически разрешает их зависимости, помещает заказ в очередь и выполняет сборку, после чего выдаёт готовый файл образа.

Объектом исследования является процесс создания и распространения пользовательских Linux-образов для микрокомпьютеров Luckfox Pico. Предметом исследования являются методы и средства автоматизации этого процесса в распределённой программной системе: конфигурирование состава образа, разрешение зависимостей компонентов, организация очереди сборок, управление пользователями и доступом, а также проверка качества и производительности системы.

Пояснительная записка состоит из введения, пяти глав, заключения, списка использованных источников и приложений. В первой главе выполнен анализ предметной области, существующих решений и сформулированы требования к системе. Во второй главе описано проектирование программной системы, включая архитектуру, модель данных и сценарии работы. В третьей главе рассмотрена реализация прототипа. Четвёртая глава посвящена тестированию системы и оценке её производительности. В пятой главе описана организация работы над проектом. В заключении подведены итоги выполненной работы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К СИСТЕМЕ

1.1 Назначение системы, цель и задачи проекта

Разрабатываемая система, получившая рабочее название Luckfox Image Builder, представляет собой распределённое веб-приложение, предназначенное для конструирования и удалённой сборки пользовательских Linux-образов для микрокомпьютеров Luckfox Pico. Система избавляет пользователя от необходимости локально разворачивать тяжёлую среду сборки, настраивать её и расходовать на сборку вычислительные ресурсы и дисковое пространство собственного компьютера.

Система предоставляет каталог готовых образов и каталог компонентов (пакетов и библиотек), визуальный редактор образов с автоматическим разрешением зависимостей, удалённую сборку с отслеживанием статуса и выдачу готового файла образа с расширением .img. Для сотрудников, отвечающих за наполнение каталога, предусмотрены административные функции: управление компонентами, изменение и удаление образов, управление учётными записями и ролями пользователей.

Практическая потребность в такой системе иллюстрируется следующим сценарием. Компания, выпускающая оборудование для видеонаблюдения и систем компьютерного зрения, переходит на использование микрокомпьютеров Luckfox для обработки видео и изображений. При переходе выясняется, что стандартный образ производителя не удовлетворяет требованиям системы, вследствие чего возникает необходимость в создании большого количества разных уникальных образов. Ручная подготовка каждого из них требует установки и освоения инструмента типа Buildroot, длительной настройки и ресурсоёмкой сборки, что и предлагается заменить работой в веб-интерфейсе.

Цель работы: разработать прототип распределённой программной системы, позволяющей удобно и быстро создать требуемый конфигурируемый

Linux-образ для микрокомпьютера Luckfox Pico без необходимости запускать долгий и требовательный процесс сборки на локальном устройстве.

Для достижения поставленной цели требуется решить следующие задачи:

– проанализировать предметную область, существующие решения и определить основные сущности, роли пользователей, функциональные и нефункциональные требования к системе;

– спроектировать архитектуру системы, модель данных и сценарии взаимодействия пользователей с системой;

– разработать средства хранения данных и организации каталогов образов и компонентов, реализовать регистрацию, аутентификацию пользователей и разграничение доступа на основе ролей;

– разработать редактор образов, позволяющий добавить в образ необходимые пакеты и библиотеки и автоматически разрешить их зависимости;

– реализовать удалённую сборку образов с последовательной обработкой заказов в очереди, отслеживанием статусов и хранением журналов сборки;

– реализовать выдачу готового образа пользователю и административные функции управления компонентами, образами и пользователями;

– разработать веб-интерфейс системы и обеспечить развёртывание всех её частей средствами контейнеризации;

– провести модульное, интеграционное, сквозное и нагрузочное тестирование системы, выявить дефекты и выполнить оптимизацию.

1.2 Особенности создания Linux-образов для микрокомпьютеров Luckfox Pico

Luckfox Pico – линейка компактных микрокомпьютеров на процессорах Rockchip RV1103 и RV1106. Обе микросхемы построены на архитектуре ARM

Cortex-A7 и рассчитаны на применение в устройствах видеонаблюдения и компьютерного зрения, поэтому включают аппаратные блоки обработки изображений и нейросетевого вывода [4]. Вычислительные ресурсы таких устройств ограничены: количество оперативной памяти измеряется десятками и сотнями мегабайт, а постоянным хранилищем служит флеш-память или карта microSD. Как следствие, стандартные готовые дистрибутивы общего назначения для них неприменимы, а состав системы приходится тщательно подбирать, включая в образ только действительно необходимые пакеты и библиотеки. В проекте используются платы, различающиеся моделью процессора и комплектацией: RV1103_Luckfox_Pico, RV1103_Luckfox_Pico_Plus и RV1106_Luckfox_Pico_Pro_Max.

Штатным способом получения образа для этих плат является официальный пакет разработчика (SDK) luckfox-pico, публикуемый производителем в открытом репозитории [5]. Управление сборкой осуществляется скриптом build.sh, который последовательно выполняет несколько этапов:

– выбор конфигурации платы с помощью файлов вида BoardConfig, определяющих модель процессора, тип носителя, вариант корневой файловой системы и параметры разделов;

– сборку загрузчика U-Boot;

– сборку ядра Linux с необходимыми драйверами;

– сборку корневой файловой системы (rootfs) средствами Buildroot;

– упаковку полученных частей в единый прошиваемый образ update.img.

Корневая файловая система формируется системой Buildroot – набором Makefile и патчей, который автоматизирует получение встраиваемого Linux-окружения, включая кросс-компилятор, ядро, загрузчик и пользовательские пакеты [6]. Набор пакетов задаётся конфигурацией, в которой каждому пакету соответствует параметр вида BR2_PACKAGE_имя, а зависимости между пакетами описываются в конфигурационных файлах Buildroot. По результатам

работы SDK формируются отдельные артефакты: uboot.img, boot.img, rootfs.img и итоговый update.img, предназначенный для прошивки платы.

Использование SDK предъявляет ряд требований к хост-машине. Сборка выполняется в среде Linux, поэтому при работе на других операционных системах необходима виртуальная машина или подсистема, например WSL. Требуется значительный объём свободного дискового пространства (десятки гигабайт для исходных текстов, кросс-компилятора и промежуточных файлов), выход в сеть для загрузки исходных архивов пакетов и наличие кросс-компилятора для архитектуры ARM. Первый прогон полной сборки занимает от одного до трёх часов, в дальнейшем время сокращается за счёт повторного использования результатов предыдущих этапов.

Анализ этого процесса позволяет выделить следующие проблемы, затрудняющие массовое создание собственных образов:

– сложность: для получения образа необходимо знать структуру SDK, назначение файлов конфигурации плат и правила описания зависимостей пакетов, а ошибки в конфигурации обнаруживаются лишь на поздних этапах сборки;

– длительность: полная сборка занимает часы, и каждое изменение состава образа требует повторного запуска процесса;

– ресурсоёмкость: сборка занимает десятки гигабайт диска и существенно нагружает процессор и оперативную память рабочей станции;

– воспроизводимость: результат зависит от состояния локальной среды, версий инструментов и доступности внешних источников, поэтому у разных разработчиков один и тот же состав может привести к разным образам или ошибкам;

– параллельные сборки: SDK использует единый рабочий каталог, и одновременный запуск нескольких сборок в нём приводит к взаимному повреждению файлов, что необходимо учитывать при организации централизованного сервиса.

Отдельные трудности возникают при работе в WSL. Во-первых, в переменную окружения PATH в этой среде автоматически добавляются пути Windows, содержащие пробелы (например, /mnt/c/Program Files/...), а Buildroot отказывается выполнять сборку, если в PATH присутствуют такие элементы. Во-вторых, при загрузке исходных архивов утилита wget без ограничений по времени может надолго «зависать» на недоступных зеркалах, не переходя к резервным источникам. Оба обстоятельства были выявлены при отладке реальной сборки и учтены при разработке сборочных скриптов системы (санитизация PATH и настройка тайм-аутов загрузки).

Таким образом, процесс создания образа для Luckfox Pico является сложным, длительным и ресурсоёмким, а его результат зависит от окружения. Это обосновывает вынесение сборки на выделенный сервер с готовой и проверенной средой и предоставление пользователю упрощённого интерфейса конфигурирования.

1.3 Анализ существующих решений и обоснование разработки

Для анализа были выбраны решения, с помощью которых на практике формируются образы для встраиваемых устройств, а также сервисы, упрощающие получение готовых образов. К ним относятся ручная сборка через SDK и Buildroot, Yocto Project, Armbian build framework, Raspberry Pi Imager и OpenWrt Firmware Selector.

Ручная сборка через SDK Luckfox и Buildroot [5], [6] является штатным способом и обеспечивает полный контроль над содержимым образа. Пользователь самостоятельно настраивает конфигурацию с помощью меню Buildroot, а зависимости пакетов разрешаются средствами самой системы сборки. Недостатками остаются описанные в подразделе 1.2 требования к локальной среде, длительность сборки и высокий порог входа.

Yocto Project [7] – универсальный набор инструментов для создания собственных Linux-дистрибутивов для встраиваемых систем. Он предоставляет

мощную систему слоёв и рецептов и подходит для промышленных проектов, однако отличается ещё более высокой сложностью освоения и большими требованиями к ресурсам. Для Luckfox Pico официальная поддержка производителя ориентирована на SDK на основе Buildroot, поэтому применение Yocto потребовало бы самостоятельной подготовки слоя поддержки платы.

Armbian build framework [19] – набор скриптов для сборки дистрибутивов Armbian для широкого круга одноплатных компьютеров на архитектуре ARM. Он позволяет выбирать плату, ядро и набор пакетов, но выполняется локально, требует подготовленной среды и ориентирован на платы общего назначения, а не на процессоры RV1103 и RV1106.

Raspberry Pi Imager [20] – настольное приложение для записи готовых образов на карты памяти. Оно имеет простой интерфейс и низкий порог входа, а также позволяет задать ряд начальных настроек (имя узла, сеть, учётную запись), однако не выполняет сборку и не позволяет выбирать состав пакетов, а список поддерживаемых устройств ограничен платами Raspberry Pi и совместимыми образами.

OpenWrt Firmware Selector [21] – веб-сервис, позволяющий выбрать устройство, добавить или исключить пакеты и получить собранную на сервере прошивку без развёртывания локальной среды. По принципу работы он является ближайшим аналогом разрабатываемой системы, но предназначен для маршрутизаторов и других устройств, работающих под управлением OpenWrt, и не поддерживает платы Luckfox Pico.

Сравнение рассмотренных решений выполнено по следующим критериям: поддержка Luckfox Pico, возможность выбора пакетов, автоматическое разрешение зависимостей, удалённая сборка без локальной среды, наличие веб-интерфейса, наличие библиотеки готовых образов и низкий порог входа. Результаты сравнения приведены в таблице 1, где «+» означает наличие возможности, «–» её отсутствие, «частично» – ограниченную или косвенную поддержку.

Таблица 1 – Сравнение существующих решений и разрабатываемой системы

Критерий	**SDK Luckfox и Buildroot**	**Yocto Project**	**Armbian build**	**Raspberry Pi Imager**	**OpenWrt Firmware Selector**	**Разрабатываемая система**
Поддержка Luckfox Pico	+	частично	–	–	–	+
Выбор пакетов	+	+	частично	–	+	+
Автоматическое разрешение зависимостей	+	+	частично	–	+	+
Удалённая сборка без локальной среды	–	–	–	–	+	+
Веб-интерфейс	–	–	–	–	+	+
Библиотека готовых образов	–	–	частично	+	+	+
Низкий порог входа	–	–	частично	+	+	+

Из таблицы 1 видно, что решения, поддерживающие Luckfox Pico и гибкий выбор состава, требуют локальной среды и высокой квалификации, а сервисы с низким порогом входа и удалённой сборкой либо не выполняют сборку (Raspberry Pi Imager), либо ориентированы на другие устройства (OpenWrt Firmware Selector). Готового решения, которое объединяет поддержку Luckfox Pico, визуальный выбор компонентов с автоматическим разрешением зависимостей, удалённую сборку и каталог готовых образов, не выявлено.

Разработка собственной системы обоснована следующим. Во-первых, потребность в большом количестве уникальных образов для Luckfox Pico не покрывается ни штатным образом производителя, ни сервисами для других платформ. Во-вторых, вынесение сборки на сервер позволяет один раз подготовить и проверить среду, устранив зависимость результата от рабочей станции каждого разработчика. В-третьих, автоматическое разрешение зависимостей и визуальный редактор снижают порог входа: пользователю достаточно выбрать нужные компоненты, не изучая устройство SDK. В-четвёртых, каталог готовых образов позволяет повторно использовать однажды созданные конфигурации. Идея онлайн-сборки с выбором пакетов, реализованная в OpenWrt Firmware Selector, является проверенной практикой и переносится в проект применительно к платам Luckfox Pico.

1.4 Словарь предметной области

На основе анализа предметной области выделен набор ключевых понятий, используемых при проектировании и реализации системы. Каждое понятие рассмотрено в словаре, приведённом в таблице 2: даны его определение и основные атрибуты, соответствующие модели данных системы. Взаимосвязи между сущностями предметной области показаны на диаграмме классов (см. приложение В).

Таблица 2 – Словарь предметной области

Термин	**Определение**	**Основные атрибуты**
Пользователь	Зарегистрированное в системе лицо, работающее с каталогами и редактором образов	Идентификатор, имя, фамилия, электронная почта (уникальна), пароль (хранится в виде хэша), роли, заказанные образы
Роль	Именованная категория пользователей, определяющая доступные им операции; в системе используются роли USER и ADMIN	Идентификатор, название роли, набор прав
Право (Permission)	Элементарное разрешение на выполнение действия, назначаемое роли	Идентификатор, описание действия
Образ (Image)	Собираемый или собранный образ операционной системы для платы Luckfox Pico с заданным составом компонентов	Идентификатор, название, описание, дистрибутив, плата, стоимость, статус, путь к артефакту, журнал сборки, версия записи, состав
Компонент (Component)	Программный пакет или библиотека, которые могут быть включены в образ	Идентификатор, название, тип, описание, стоимость, версия записи, зависимости
Тип компонента	Справочная категория компонента; в каталоге используются типы «Пакет» и «Библиотека»	Идентификатор, название типа
Зависимость компонента	Отношение, при котором для работы одного компонента необходим другой; зависимости образуют ориентированный граф, не допускающий циклов	Родительский компонент, зависимый компонент
Состав образа	Множество компонентов, включённых в образ, с учётом транзитивных зависимостей	Образ, набор компонентов
Заказ сборки	Запрос пользователя на сборку образа с выбранным названием, дистрибутивом, платой и набором компонентов	Название, описание, дистрибутив, плата, список идентификаторов компонентов, заказчик

Термин	Определение	Основные атрибуты
Очередь сборок	Последовательность заказов, ожидающих обработки; заказы выполняются по одному в порядке поступления	Ёмкость очереди (1 000 заказов), порядок обработки FIFO, число рабочих потоков (1)
Сборочный воркер	Фоновый поток серверной части, который берёт заказ из очереди и запускает сборочный скрипт	Имя потока, текущий образ, код завершения скрипта
Артефакт (.img)	Файл образа, получаемый в результате успешной сборки и доступный для скачивания	Путь к файлу, формат файла, привязка к образу
Журнал сборки	Текстовый вывод сборочного скрипта, сохраняемый вместе с образом и используемый для диагностики ошибок	Текст журнала, образ
Статус образа	Состояние образа в жизненном цикле сборки: QUEUED, BUILDING, READY, FAILED	Значение статуса, допустимые переходы
Дистрибутив	Базовая система, на основе которой формируется образ; в прототипе по умолчанию используется Buildroot	Название
Плата (board)	Модель микрокомпьютера Luckfox Pico, для которой собирается образ, например RV1103_Luckfox_Pico	Название конфигурации платы
SDK	Официальный пакет разработчика luckfox-pico, содержащий средства сборки загрузчика, ядра и корневой файловой системы	Путь к каталогу SDK, конфигурации плат
Режим симуляции	Режим работы сборочного скрипта по умолчанию, при котором вместо кросс-компиляции формируется ext4-файл с манифестом и списком пакетов; получаемый файл не является загрузочным образом	Признак режима (переменная LUCKFOX_REAL_BUILD), искусственная задержка

1.5 Пользователи системы и их роли

В ходе анализа предметной области выделены категории целевой аудитории, потребности которых необходимо учесть при проектировании и реализации системы.

К первичной категории относятся разработчики программного обеспечения для микрокомпьютеров Luckfox, в том числе сотрудники компании, выпускающей оборудование для видеонаблюдения и систем компьютерного зрения. Их основная проблема заключается в необходимости тратить время и место на диске рабочей станции для создания образа. Потребность состоит в

возможности собрать требуемый образ удалённо, не нагружая и не загромождая собственный компьютер, и получить в результате воспроизводимый артефакт.

К вторичной категории относятся энтузиасты и люди, занимающиеся разработкой на микрокомпьютерах в качестве хобби. Основной проблемой для них является сложность создания образа. Потребность состоит в возможности получить нужный образ, не тратя время на изучение зависимостей и установку сложных инструментов.

Исходя из целевой аудитории, определены следующие роли пользователей системы. Взаимодействие ролей с функциями системы показано на диаграмме прецедентов (см. приложение Б).

– Гость (неавторизованный пользователь) – человек, не прошедший аутентификацию. Ему доступен просмотр библиотек образов и компонентов (с поиском и фильтрацией), а также регистрация и вход в систему.

– Клиент (роль USER) – основной пользователь системы. Помимо возможностей гостя, он может создавать образ в редакторе, заказывать его сборку, просматривать свои образы и их статусы в разделе «Мои образы» и скачивать готовые образы. Роль USER назначается при регистрации по умолчанию.

– Администратор (роль ADMIN) – пользователь, ответственный за наполнение каталогов и безопасную работу системы. Он обладает всеми возможностями клиента, а также управляет компонентами (создание, изменение, удаление), изменяет и удаляет образы (при изменении состава образ пересобирается), управляет учётными записями пользователей и их ролями.

1.6 Функциональные требования к системе

На основе проведённого анализа сформирован перечень функциональных требований. В перечень включены функции, реализованные в прототипе. Оплата заказа и доставка готового образа по электронной почте рассматривались на этапе анализа как часть сценария заказа, однако в прототипе не реализованы и

заменены выдачей готового файла образа через скачивание авторизованному пользователю.

– система должна обеспечивать регистрацию пользователей по адресу электронной почты и паролю и вход в систему с выдачей токена доступа;

– система должна предоставлять любому посетителю просмотр библиотеки образов и библиотеки компонентов с постраничным выводом, поиском по названию и сортировкой;

– система должна обеспечивать фильтрацию образов по статусу, плате и составу компонентов, а компонентов – по типу и сортировку по цене;

– система должна предоставлять авторизованному пользователю визуальный редактор образа, в котором задаются название, описание, дистрибутив, плата и набор компонентов;

– система должна автоматически дополнять выбранный набор компонентов их транзитивными зависимостями и отклонять заказ при обнаружении цикла в графе зависимостей;

– система должна рассчитывать стоимость образа как сумму стоимостей всех вошедших в него компонентов;

– система должна принимать заказ на сборку образа и выполнять сборку асинхронно, обрабатывая заказы последовательно по одному в порядке поступления;

– система должна поддерживать статусы образа QUEUED (в очереди), BUILDING (сборка), READY (готов) и FAILED (ошибка) и отображать актуальный статус пользователю с автоматическим обновлением;

– система должна сохранять журнал сборки образа и путь к полученному артефакту;

– система должна предоставлять авторизованному пользователю скачивание готового файла образа в формате .img;

– система должна показывать авторизованному пользователю список заказанных им образов в разделе «Мои образы»;

– система должна обеспечивать администратору создание, изменение и удаление компонентов с указанием типа и зависимостей;

– система должна обеспечивать администратору изменение образов с их повторной постановкой в очередь сборки и удаление образов вместе с артефактами;

– система должна запрещать изменение образа, находящегося в очереди или в процессе сборки;

– система должна обеспечивать администратору просмотр списка пользователей, создание, изменение и удаление учётных записей и изменение ролей пользователей;

– система должна поддерживать два режима сборки: режим симуляции, формирующий проверочный ext4-артефакт без кросс-компиляции, и режим реальной сборки с помощью SDK Luckfox.

1.7 Нефункциональные требования к системе

Нефункциональные требования описывают не прямую функциональность системы, а критерии, определяющие качество её работы, удобство использования и эксплуатационные характеристики. Требования сформулированы на этапе анализа и уточнены при подготовке методики тестирования.

Требования по производительности:

– время отклика системы для стандартных операций, кроме сборки образов, не должно превышать 5 секунд при одновременной работе до 500 пользователей;

– максимальное время сборки одного образа не должно превышать 24 часа;

– при нагрузочном тестировании доля ошибочных ответов не должна превышать 1 %;

– 95-й процентиль времени чтения (просмотр каталогов) не должен превышать 500 мс;

– 95-й процентиль времени записи не должен превышать 1 000 мс;

– 95-й процентиль времени входа в систему не должен превышать 1 000 мс;

– 95-й процентиль времени скачивания образа размером 18 МБ не должен превышать 3 000 мс.

Требования по надёжности:

– сборки должны выполняться последовательно, одна за другой, поскольку SDK использует единый рабочий каталог и параллельный запуск повреждает результаты;

– операции, затрагивающие несколько таблиц, должны выполняться атомарно в рамках транзакций;

– при одновременном редактировании образов и компонентов должны предотвращаться потерянные обновления с помощью оптимистичной блокировки;

– ошибка сборки одного образа не должна прекращать обработку очереди, а причина ошибки должна сохраняться в журнале сборки.

Требования по безопасности:

– доступ к заказу сборки и скачиванию образов должен быть разрешён только после аутентификации, а операции управления компонентами, образами и пользователями – только администратору;

– аутентификация должна выполняться с использованием токенов JWT без хранения сессии на сервере;

– пароли пользователей должны храниться в базе данных только в виде хэшей BCrypt;

– пароль должен содержать не менее 8 символов, включая цифру, строчную и заглавную буквы и специальный символ;

– система должна разграничивать права доступа в соответствии с ролями «Гость», «Клиент» (USER) и «Администратор» (ADMIN);

– число неудачных попыток входа должно ограничиваться, чтобы затруднить подбор паролей.

Требования к интерфейсу:

– интерфейс должен быть интуитивно понятным и не требовать от пользователя знаний об устройстве SDK и Buildroot;

– интерфейс должен быть адаптивным и корректно отображаться на основных разрешениях экрана 1920×1080 и 375×667;

– состояние сборки образа должно быть видно пользователю в библиотеке и в личном разделе без ручного обновления страницы.

Требования к развёртыванию и эксплуатации:

– система должна разворачиваться средствами Docker Compose в виде трёх сервисов: базы данных, серверной части и клиентской части [18];

– параметры подключения к базе данных, путь к сборочному скрипту, каталог артефактов и режим сборки должны задаваться переменными окружения;

– режим сборки по умолчанию должен быть безопасным для демонстрации (симуляция), а переход к реальной сборке должен выполняться установкой переменной окружения без изменения кода.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ

2.1 Архитектура клиентской и серверной частей приложения

Программная система Luckfox Image Builder построена по клиент-серверной схеме из трёх независимо развёртываемых частей: одностраничного веб-приложения (SPA), серверной части с REST-интерфейсом и реляционной СУБД. Клиентская часть реализована на React 18 со сборкой средствами Vite, маршрутизацией React Router, хранилищем состояния Zustand и HTTP-клиентом Axios [12]. Серверная часть выполнена на Spring Boot 3.2 (Java 17) с модулями Web, Data JPA, Security и Validation [8]; данные хранятся в MySQL 8 [11]. Части взаимодействуют по HTTP в формате JSON, а подлинность пользователя подтверждается токеном JWT в заголовке Authorization. Общая схема взаимодействия компонентов приведена на рисунке Г.1, схема развёртывания – на рисунке Г.3 (см. приложение Г).

Серверная часть разделена на слои с односторонними зависимостями: контроллеры (controller) принимают запрос, проверяют DTO и формируют ответ; сервисы (service) содержат бизнес-правила и определяют границы транзакций; репозитории Spring Data JPA (repository) обеспечивают доступ к данным; сущности (entity) отображают таблицы [22]. Сквозные механизмы вынесены в пакеты config (безопасность, CORS, наполнение базы, пул задач), filter (JWT-фильтр) и exception (исключения и глобальный обработчик ошибок). Подсистема конфигурирования и сборки образов размещена в тех же слоях и описана в записке второго участника проекта; она использует общие сущности Image и Component, каталог компонентов и механизм аутентификации.

При проектировании принят ряд решений.

– одностраничное приложение вместо серверного рендеринга. Интерфейс интерактивен: редактор образа хранит выбранные компоненты и стоимость на клиенте, списки фильтруются и листаются без перезагрузки, статусы сборок обновляются в фоне. Сервер отдаёт только данные, а

отображение отделено от бизнес-логики. Платой стала необходимость учитывать роли на клиенте для управления видимостью элементов; окончательную проверку прав всегда выполняет сервер;

– бессессионная (stateless) аутентификация на основе JWT. Сервер не хранит сеансы, поэтому экземпляры приложения взаимозаменяемы, а перезапуск не разлогинивает пользователей. Токен содержит лишь адрес электронной почты и срок действия, а роли при каждом запросе загружаются из базы данных, поэтому изменение ролей вступает в силу немедленно;

– передача наружу только объектов передачи данных (DTO). Сущности JPA связаны рекурсивно (компонент – зависимости – зависимости), и их прямая сериализация привела бы к бесконечной рекурсии и обращению к ленивым коллекциям вне транзакции. DTO также скрывают внутренние поля: хеш пароля, путь к артефакту и журнал сборки в ответы не попадают;

– пагинация и фильтрация в базе данных: условия поиска преобразуются в SQL, поэтому объём ответа и нагрузка не зависят от размера каталога;

– отказ от открытой сессии Hibernate на время обработки запроса (spring.jpa.open-in-view=false): все DTO строятся внутри транзакций сервисов, и соединение возвращается в пул до сериализации ответа. Решение принято по итогам нагрузочного тестирования (дефекты DEF-12, DEF-22).

Клиентская часть организована по функциональным группам каталога frontend/src (таблица 3).

Таблица 3 – Модули клиентской части приложения

Каталог, файл	**Назначение**	**Ключевые элементы**
main.jsx, App.jsx	Точка входа, провай-деры, шапка, таблица маршрутов	BrowserRouter, AuthProvider, Routes
pages/	Страницы, соответ-ствующие маршрутам	ImageLibrary, ComponentLibrary, ImageEditor, ComponentEditor, Account, UserManagement, Login, Register

Каталог, файл	Назначение	Ключевые элементы
components/	Повторно используемые элементы интерфейса	Header, ProtectedRoute, ImageCard, ComponentCard, ComponentFilter, Pagination, StatusBadge
api/	Доступ к REST-интерфейсу	client.js (экземпляр Axios, интерсепторы), api.js (функции запросов, извлечение текста ошибки)
context/	Состояние аутентификации	AuthContext: токен, профиль, признаки isAuthenticated и isAdmin, login, register, logout
store/	Состояние редактора образа до отправки на сервер	editorStore (Zustand): название, плата, выбранные компоненты, предварительная стоимость
hooks/	Общая логика списков	usePagedList: запрос страницы, задержка ввода 250 мс, сброс страницы при смене фильтров, отбрасывание устаревших ответов, необязательный опрос
constants.js	Справочники клиента	Дистрибутивы, платы, роли, варианты сортировки по цене

Состояние разделено по времени жизни данных: токен и профиль, общие для всего приложения, находятся в контексте React и localStorage; данные редактора образа, не нужные после отправки заказа, – в лёгком хранилище Zustand; данные списков локальны для страницы и инкапсулированы в хуке usePagedList, что устраняет повторение кода на страницах со списками.

Развёртывание описано в docker-compose.yml и включает три контейнера: db (MySQL 8.0, именованный том mysql_data, проверка готовности), backend (стартует после успешной проверки базы данных) и frontend (собранное SPA, которое раздаёт встроенный статический сервер Node.js на порту 5173 с возвратом index.html для неизвестных путей, необходимым для клиентской маршрутизации). Параметры подключения к базе данных передаются серверу переменными окружения, адрес API подставляется в клиент при сборке переменной VITE_API_URL [18]. Пул соединений рассчитан на 20 соединений с ожиданием не более 3 с: при перегрузке клиент быстро получает 503 вместо долгого ожидания.

2.2 Информационная модель и проектирование базы данных

Концептуальная модель этапа проектирования содержала пользователя (User), роль (Role), право доступа (Premission), образ (Image), компонент (Component) и тип компонента (component_type) со связующими таблицами для отношений «многие ко многим»; скрипт java_working.sql (каталог DOCs) описывает 11 таблиц с первичными и внешними ключами. При реализации модель перенесена в сущности JPA и расширена под требования подсистемы сборки и производительности. ER-диаграмма модели этапа проектирования приведена на рисунке Д.1 (приложение Д). Фактическая схема, создаваемая Hibernate, содержит те же 11 таблиц, но таблица Image дополнена полями name, distro, board, artifact_path, build_log и version, а Component – полем version (см. таблицы приложения Д). Диаграмма классов приведена на рисунке В.1 (приложение В) [2], [3]. Сущности и их назначение приведены в таблице 4.

Таблица 4 – Сущности информационной модели

Таблица (сущность)	**Назначение**	**Ключевые атрибуты и связи**
User (User)	Пользователь	id, name, surename, email (уникальный), password (хеш BCrypt); роли – M:N через user_role; заказанные образы – M:N через user_image_order
Role (Role)	Роль	id, Role_name (USER, ADMIN); права – M:N через role_premission
Premission (Permission)	Право доступа	id, action; в логике доступа не используется
Image (Image)	Образ	id, name, description, distro, board, cost, state, artifact_path, build_log, version; состав – M:N с Component через image_component_dependency
Component (Component)	Компонент каталога	id, name, description, cost, version; тип – N:1 к component_type; зависимости – M:N к самому себе через dependency
component_type (ComponentType)	Тип компонента	id, Type («Пакет», «Библиотека» в начальных данных)
user_role, role_premission, user_image_order, image_component_dependency, dependency	Связующие таблицы	Пары внешних ключей; для dependency – parent_id и dependent_id (ребро графа зависимостей)

Отношения выбраны следующим образом. Пользователь может иметь несколько ролей (администратор в начальных данных имеет ADMIN и USER), поэтому связь «пользователь – роль» имеет вид «многие ко многим». Владельцем связи «пользователь – заказанные образы» является User: образ не хранит ссылку на заказчика, и связь снимается при удалении любой из сторон без затрагивания другой. Зависимости компонентов образуют самоссылающееся отношение «многие ко многим», то есть ориентированный граф; его разрешение при заказе описано в записке второго участника. Тип компонента вынесен в отдельную таблицу, чтобы администратор мог расширять справочник без изменения схемы. Таблицы Premission и role_premission сохранены из концептуальной модели, но полномочия определяются именами ролей, чего достаточно для двух ролей прототипа.

Отличия фактической модели от концептуальной сводятся к неразрушающим расширениям и служебным решениям.

– в таблицу Image добавлены столбцы name, distro, board, artifact_path и build_log для подсистемы сборки; они создаются при запуске (ddl-auto=update) без потери данных;

– в таблицы Image и Component добавлен столбец version с аннотацией @Version (оптимистичная блокировка): при одновременном редактировании записи вторая запись с устаревшей версией отклоняется, и обработчик ошибок возвращает 409. При десяти параллельных изменениях одного компонента успешным оказалось одно, остальные получили 409 (тест INT-28);

– для email пользователя задан уникальный индекс: проверка в сервисе дополняется защитой от гонки, и при 20 параллельных регистрациях одного адреса в базе осталась одна запись (тест INT-18);

– слово User зарезервировано в MySQL, поэтому имя таблицы квотируется (@Table(name = "`User`")); остальные имена заданы стратегией PhysicalNamingStrategyStandardImpl, чтобы сохранить имена модели, включая исторические написания Premission и surename;

– добавлены индексы под операции каталогов: для образов – (state, cost) и (cost), для компонентов – (name, id) и (type, cost) (дефект DEF-23). Поиск по подстроке LIKE '%…%' индексом не ускоряется.

Схема создаётся средствами Hibernate (ddl-auto=update) [10]: для прототипа отдельные миграции не нужны, схема всегда соответствует сущностям. Файл java_working.sql сохранён как документная модель и не монтируется в контейнер MySQL, так как типы BIGINT UNSIGNED скрипта и BIGINT сгенерированной схемы конфликтуют при создании внешних ключей. Режим update не удаляет и не переименовывает столбцы, поэтому для промышленной эксплуатации следует перейти на инструмент миграций (например, Flyway или Liquibase).

Правила удаления заданы в сервисах, а не каскадами базы данных. Компонент нельзя удалить, если он входит в образы или является зависимостью другого компонента (методы existsByComponentsContains и existsByDependenciesContains). Удаление образа сначала снимает связи с заказчиками. Удаление пользователя снимает только его связи, а заказанные образы остаются в библиотеке (тест INT-39). Если СУБД всё же отклоняет операцию по внешнему ключу, обработчик ошибок возвращает 409. Каталог остаётся согласованным без каскадного удаления данных, которое трудно откатить.

При первом запуске, когда таблица компонентов пуста, компонент DataSeeder создаёт роли, типы, семь содержательных компонентов с зависимостями (например, opencv-python, зависящий от numpy и python3), 60 тестовых компонентов для проверки пагинации, администратора admin@luckfox.io и 15 образов, заказанных через обычную процедуру заказа сборки. Учётные данные администратора демонстрационные и подлежат замене при промышленном развёртывании.

2.3 Аутентификация, управление пользователями и разграничение доступа

Для аутентификации выбраны токены JWT с подписью HS256 (библиотека jjwt 0.11.5), для защиты паролей – хеш-функция BCrypt из Spring Security [8]. В базе хранится только хеш пароля, а ответ на вход содержит лишь токен.

Регистрация (POST /api/auth/register) проверяется правилами Bean Validation: имя и адрес электронной почты обязательны, адрес корректен, пароль содержит не менее 8 символов, цифру, строчную и заглавную буквы и спецсимвол из набора @#$%^&+=!. Новому пользователю присваивается только роль USER, поэтому получить права администратора самостоятельно нельзя; их назначает администратор в разделе управления пользователями. После регистрации сразу возвращается токен, и пользователь оказывается вошедшим. Порядок обработки входа (POST /api/auth/login) и доступа к защищённому ресурсу показан на рисунке 1.

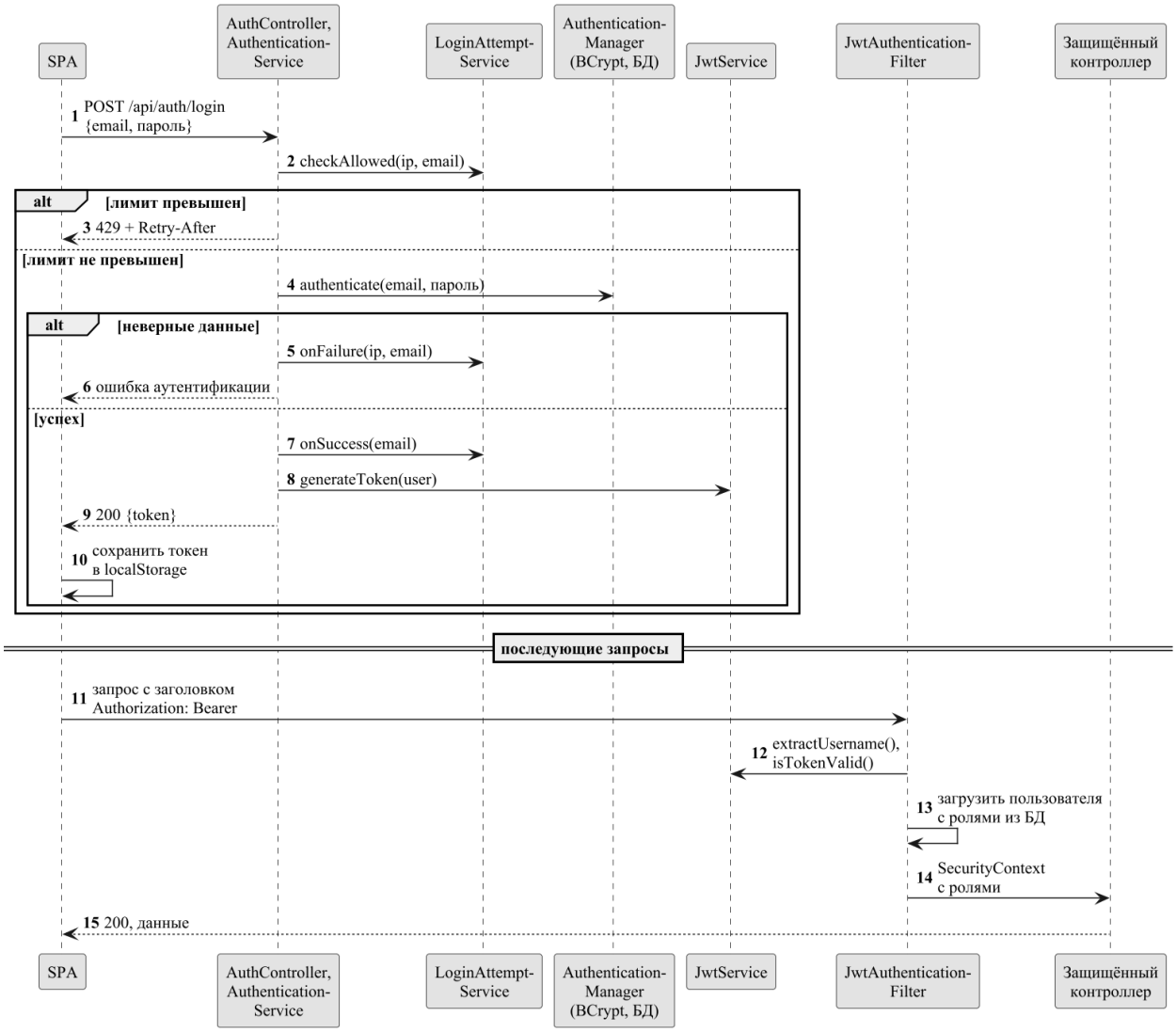


Рисунок 1 – Диаграмма последовательности аутентификации и доступа к защищённому ресурсу

AuthenticationService сначала обращается к LoginAttemptService, проверяющему лимит неудачных попыток, и лишь затем передаёт данные в AuthenticationManager, который загружает пользователя по адресу (запрос сразу выбирает роли) и сравнивает хеш BCrypt. При успехе JwtService формирует токен с полями sub (адрес), iat и exp и возвращает его в ответе 200 вида {"token": "…"}. Клиент сохраняет токен в localStorage и запрашивает профиль GET /api/users/me; интерсептор Axios добавляет заголовок Authorization: Bearer к каждому последующему запросу. Хранение в localStorage позволяет пережить перезагрузку страницы, но делает токен доступным сценариям страницы, поэтому защита от внедрения стороннего кода (XSS) лежит на приложении.

На сервере каждый запрос с заголовком Bearer проходит через JwtAuthenticationFilter, встроенный в цепочку перед UsernamePasswordAuthenticationFilter. Фильтр проверяет подпись и срок действия, загружает пользователя с ролями и помещает аутентификацию в контекст безопасности; роли преобразуются в полномочия ROLE_USER и ROLE_ADMIN. Запрос без заголовка продолжается как анонимный, и решение принимают правила SecurityConfig. Сессии не создаются (STATELESS); защита CSRF отключена, так как токен передаётся в заголовке, а не в cookie и не отправляется браузером автоматически.

Правила доступа заданы в два уровня. Конфигурация SecurityConfig открывает для всех маршруты /api/auth/** и чтение каталогов (GET /api/components/**, /api/component-types/**, /api/images и /api/images/{id}): гость может оценить систему до регистрации. Остальные запросы требуют аутентификации, поэтому новый маршрут по умолчанию закрыт; скачивание образа (GET /api/images/{id}/download) открытому правилу не соответствует из-за дополнительного сегмента пути. Второй уровень – аннотация @PreAuthorize("hasRole('ADMIN')") на методах, изменяющих каталоги и управляющих пользователями (@EnableMethodSecurity). Матрица доступа приведена в таблице 5.

Таблица 5 – Матрица доступа к операциям системы

Операция	**Маршрут**	**Гость**	**USER**	**ADMIN**
Регистрация, вход	POST /api/auth/register, /api/auth/login	да	да	да
Просмотр образов, компонентов, типов	GET /api/images, /api/images/{id}, /api/components, /api/components/{id}, /api/component-types	да	да	да
Заказ сборки образа	POST /api/images/build	нет	да	да
Скачивание образа	GET /api/images/{id}/download	нет	да	да
Изменение и удаление образа	PUT, DELETE /api/images/{id}	нет	нет	да
Создание, изменение и удаление компонента	POST, PUT, DELETE /api/components	нет	нет	да
Профиль и заказанные образы	GET /api/users/me, /api/users/me/images	нет	да	да
Управление пользователями	GET, POST, PUT, DELETE /api/users, /api/users/{id}	нет	нет	да

Управление пользователями (UserService) включает создание, изменение, удаление и постраничный поиск по имени, фамилии и адресу с фильтром по роли. Действуют правила самозащиты: администратор не может удалить себя и не может снять с себя роль ADMIN, иначе система могла бы остаться без администратора. Оба правила проверяются по адресу из токена и дают ответ 400 (тест INT-42). Адрес электронной почты должен быть уникален; при изменении пустой пароль означает «оставить прежний»; неизвестная роль даёт 400.

Отдельная защита введена для входа. Нагрузочное тестирование показало, что сервер принимает неограниченное число попыток подбора пароля, а BCrypt при 100 попытках в секунду занимал около восьми ядер и замедлял вход остальных в 11 раз (дефект DEF-21). Поэтому LoginAttemptService ведёт счётчики неудач для каждого адреса и каждого IP-адреса в фиксированном окне 300 с. При 5 неудачах для адреса или 20 для IP-адреса запрос отклоняется с кодом 429 и заголовком Retry-After ещё до проверки пароля, поэтому перебор не расходует процессор на BCrypt. Успешный вход сбрасывает счётчик адреса; пороги задаются свойствами security.login.*. Повторный прогон подтвердил 429 после пятой неудачи и отсутствие влияния на вход другого пользователя (тест INT-48). Ограничение решения – хранение счётчиков в памяти процесса: для нескольких экземпляров сервера нужно общее хранилище.

Выявленные при тестировании недостатки безопасности внесены в реестр дефектов (см. приложение Ж). Ключ подписи JWT задан константой в исходном коде (DEF-07); срок жизни токена вследствие ошибки в арифметике составляет 4 часа вместо заявленных 24 (DEF-06); поддельный или просроченный токен и токен удалённого пользователя приводят к ответу 403 вместо 401 (DEF-05, DEF-08); запрос без токена и неверный пароль также дают 403 (DEF-03, DEF-04), поэтому клиентский перехватчик, реагирующий на 401, не сбрасывает сессию; после смены собственного адреса прежний токен перестаёт действовать (DEF-18); скачивание образа не проверяет владельца (DEF-16). Первоочередные меры

– вынесение ключа в секреты окружения и добавление AuthenticationEntryPoint, возвращающего 401; архитектура аутентификации при этом не меняется.

2.4 Каталоги образов и компонентов и административные функции

Каталоги – основной пользовательский сценарий, поэтому запросы к ним спроектированы единообразно. Списки образов, компонентов и пользователей отдаются страницами PageResponse и принимают общие параметры page и size; поиск и фильтры комбинируются логическим И и преобразуются в условия Spring Data JPA Specification, которые выполняются в базе данных вместе с пагинацией (LIMIT/OFFSET и подсчёт записей). Параметры приведены в таблице 6.

Таблица 6 – Параметры запросов списков

Параметр	**Список**	**Значение и умолчание**
page	все	Номер страницы с нуля; отрицательное значение заменяется нулём
size	все	Размер страницы; по умолчанию 12 для образов и 20 для компонентов и пользователей; ограничен диапазоном от 1 до 100
search	все	Подстрока без учёта регистра: в названии (образы, компоненты) либо в имени, фамилии или адресе (пользователи)
state	образы	Статус сборки (QUEUED, BUILDING, READY, FAILED)
board	образы	Точное название платы
components	образы	Идентификаторы через запятую; образ должен содержать все перечисленные компоненты
type	компоненты	Идентификатор типа компонента
role	пользователи	Название роли (USER, ADMIN)
sort	образы, компоненты	price_asc или price_desc – по стоимости, при равной – порядок по умолчанию (образы – от новых к старым, компоненты – по названию)

Ограничение размера страницы защищает сервер от запросов, извлекающих весь каталог за одно обращение: size=1000 обрезается до 100, size=0 заменяется единицей, страница за пределами набора пуста (тест INT-19). Порядок сортировки дополняется идентификатором записи, поэтому при

переходе между страницами записи не дублируются и не пропадают (тест INT-22).

Наиболее сложен фильтр образов по компонентам: пользователь выбирает несколько компонентов и получает образы, содержащие все выбранные. Соединение (join) со связующей таблицей вернуло бы дубли образов и потребовало бы группировки, поэтому для каждого выбранного компонента добавляется отдельное условие EXISTS с подзапросом по таблице состава; повторяющиеся идентификаторы удаляются, а основная выборка остаётся без соединений, что сохраняет корректность подсчёта и пагинации. Стоимость решения – один подзапрос на компонент. Фильтр работает по составу, зафиксированному при заказе, и учитывает автоматически добавленные зависимости. Справочник типов отдаётся отдельным запросом GET /api/component-types без пагинации. Символы % и _ в строке поиска не экранируются и работают как подстановочные (дефект DEF-11).

Административные функции доступны роли ADMIN и включают три группы операций.

– управление компонентами (ComponentService). Создание и изменение принимают ComponentRequest: название и неотрицательная стоимость обязательны, тип должен существовать, зависимости необязательны. При сохранении проверяются существование зависимостей и защита графа: компонент не может зависеть от себя, а зависимость, транзитивно ведущая обратно к изменяемому компоненту, отклоняется как циклическая (обход в ширину). Для нового компонента проверка не нужна, так как на него ещё никто не ссылается. Удаление используемого компонента запрещено: возвращается 400 с названием компонента и указанием, что изменить сначала (тест INT-26);

– изменение и удаление образов. Администратор открывает образ в редакторе (маршрут /editor/image?id=…) или удаляет его. Правила состояний (изменение запрещено в QUEUED и BUILDING, удаление – в BUILDING), пересчёт состава и повторная постановка в очередь выполняются подсистемой

сборки второго участника; клиент лишь показывает эти операции только администратору и блокирует удаление образа в статусе BUILDING;

– управление пользователями (раздел 2.3) на странице с формой создания и изменения, таблицей, поиском и фильтром по роли.

Удаление подтверждается диалогом интерфейса, а сообщения сервера, например об использовании компонента в образах, выводятся в строке ошибки страницы.

2.5 Пользовательский интерфейс и структура навигации

Основной принцип интерфейса – предсказуемость: единая шапка с меню, одинаковое поведение всех списков (поиск, фильтры, пагинатор), карточки с заметным статусом и действиями, немедленное сообщение о результате операции. Состав меню зависит от роли: гостю доступны «Образы», «Компоненты», вход и регистрация; пользователю добавляются «Создать образ» и «Аккаунт»; администратору – «+ Компонент» и «Пользователи». Структура переходов показана на диаграмме навигации, разработанной на этапе проектирования интерфейса (рисунок Е.1, приложение Е). При реализации она упрощена: страница оплаты не реализована и заменена скачиванием готового образа, отдельные страницы просмотра образа и компонента заменены карточками в библиотеках, а настройка пользователя выполняется формой на странице управления пользователями. Страницы, маршруты и доступ в реализованном приложении приведены в таблице 7.

Таблица 7 – Страницы приложения и маршруты

Маршрут	**Страница**	**Назначение**	**Доступ**
/ и неизвестные пути	–	Перенаправление на /library/images	все
/library/images	ImageLibrary	Библиотека образов: поиск, фильтры по статусу, плате и компонентам, сортировка, скачивание; для администратора – изменение и удаление	все
/library/components	ComponentLibrary	Библиотека компонентов: поиск, фильтр	все

Маршрут	Страница	Назначение	Доступ
		по типу, сортировка по цене; для администратора – изменение и удаление	
/login, /register	Login, Register	Вход и регистрация	все
/editor/image	ImageEditor	Выбор компонентов, предварительная стоимость, заказ сборки; с параметром id – изменение образа	USER, ADMIN (изменение – ADMIN)
/editor/component	ComponentEditor	Создание и изменение компонентов, зависимости	ADMIN
/admin/users	UserManagement	Список, поиск, создание, изменение и удаление пользователей	ADMIN
/account	Account	Профиль и заказанные образы со статусами и скачиванием	USER, ADMIN

Доступ к страницам ограничивает компонент ProtectedRoute: без токена он перенаправляет на /login, сохраняя исходный адрес, чтобы вернуть пользователя после входа; при параметре adminOnly пользователь без роли ADMIN попадает в библиотеку образов. Эта проверка – удобство интерфейса, а не защита: настоящие ограничения обеспечивает сервер (таблица 5), и подмена состояния в браузере не даёт доступа к данным. До получения профиля показывается индикатор загрузки.

В редакторе образа слева расположен каталог компонентов с поиском и фильтром по типу, справа – выбранные компоненты и форма параметров. Предварительная стоимость считается на клиенте по выбранным компонентам; итоговую с учётом транзитивных зависимостей определяет сервер. Статусы сборок обновляются автоматически: страницы «Библиотека образов» и «Аккаунт» опрашивают сервер каждые 5 с, поэтому пользователь видит смену состояний «В очереди», «Сборка», «Готов» без перезагрузки. Кнопка «Скачать .img» показывается только при признаке downloadable; файл загружается через fetch с заголовком авторизации, так как обычная ссылка не передала бы токен.

Интерфейс прошёл путь от макетов Figma этапа проектирования (библиотеки, редакторы образа и компонента, вход и регистрация) к реализации: композиция экранов и навигация сохранены, добавлены страница аккаунта,

управление пользователями, фильтры по статусу, плате и компонентам, сортировка, пагинатор и автообновление статусов. Страницы оплаты в реализации нет: оплата не входит в прототип и заменена скачиванием готового файла. Оформление выполнено единой таблицей стилей со шрифтом Inter и цветовой схемой на переменных CSS.

Адаптивность обеспечена сеточной вёрсткой карточек и переключением редактора в одну колонку при ширине окна меньше 860 пикселей. Ограничение текущей версии: при такой ширине меню шапки скрывается без замены (например, бургер-меню), поэтому переходы между разделами возможны только по кнопкам страниц или по адресу.

2.6 Контракты взаимодействия клиентской и серверной частей

Обмен ведётся по HTTP в формате JSON (скачивание образа – поток application/octet-stream), все пути начинаются с /api, авторизация – заголовок Authorization: Bearer. Базовый адрес клиент берёт из переменной VITE_API_URL, а при её отсутствии использует относительный путь /api; в режиме разработки запросы перенаправляет прокси Vite на порт 8080, и политика CORS не применяется. В Docker Compose адрес задаётся при сборке (http://localhost:8080/api), запросы становятся межисточниковыми, и сервер разрешает CORS для /api/** с методами GET, POST, PUT, DELETE и OPTIONS. Разрешён единственный источник http://localhost:5173, зашитый в WebConfig (дефект DEF-02): фронтенд на другом адресе не сможет обратиться к API без пересборки сервера, поэтому источник следует вынести в конфигурацию.

Страница списка PageResponse одинакова для образов, компонентов и пользователей, что позволяет использовать один пагинатор и один хук (таблица 8).

Таблица 8 – Поля страницы списка (PageResponse)

Поле	**Тип**	**Назначение**
content	массив	Элементы страницы (ImageDto, ComponentDto

Поле	Тип	Назначение
		или UserResponse)
page	число	Номер страницы с нуля
size	число	Фактический размер страницы после ограничения
totalElements	число	Общее число записей, удовлетворяющих условиям
totalPages	число	Общее число страниц

Основные DTO. UserResponse содержит id, name, surname, email и множество названий ролей; хеш пароля не передаётся. ComponentDto содержит id, name, description, cost, тип (id и название), список идентификаторов зависимостей dependencyIds и список пар «идентификатор, название» dependencies, избавляющий редактор от дополнительных запросов. ImageDto содержит параметры образа, статус, признак downloadable и плоский список компонентов (поля описаны в записке второго участника). Запросы ComponentRequest и UserRequest проверяются Bean Validation. Со смежной подсистемой сборки контракт ограничен запросом BuildRequest, представлением ImageDto и кодами 202, 400 и 404 при заказе и изменении образа, поэтому обе подсистемы развиваются независимо.

Успешные ответы имеют коды 200 (чтение, создание и изменение компонентов и пользователей, вход, регистрация), 202 (заказ и изменение образа: запрос принят, сборка асинхронна) и 204 (удаление). Формат ошибок определяет GlobalExceptionHandler: тело – JSON-объект вида {"error": "текст"} для бизнес-правил и «поле – сообщение» для проверки полей; функция extractError клиента приводит оба вида к строке для интерфейса. Используемые коды приведены в таблице 9.

Таблица 9 – Коды ошибок и их источники

Код	Источник	Тело ответа
400	Проверка полей (MethodArgumentNotValidException); бизнес-правила (BadRequestException): самоблокировка администратора, повторный адрес, цикл зависимостей, удаление используемого компонента, скачивание неготового образа	{поле: сообщение} или {"error": "…"}

Код	Источник	Тело ответа
403	Ответ Spring Security по умолчанию: нет или недействителен токен, недостаточно прав, неверный пароль	Пустое тело
404	Объект не найден (ResourceNotFoundException)	{"error": "…"}
409	Нарушение целостности данных или конфликт версий (@Version)	{"error": "…"}
429	Превышен лимит неудачных попыток входа	{"error": "…"}, заголовок Retry-After (секунды)
503	Исчерпан пул соединений с базой данных	{"error": "Сервис перегружен, повторите запрос позже"}, Retry-After: 1

Таблица показывает слабое место контракта: код 401 сервером не используется, для неаутентифицированных запросов возвращается 403 с пустым телом (DEF-03, DEF-04, DEF-05). Перехватчик Axios при 401 удаляет токен и переводит на страницу входа, но при недействительном токене этого не происходит, и пользователь видит сообщение «Request failed with status code 403» (тест E2E-06). Ошибки разбора параметров у анонимного пользователя также дают 403 вместо 400 (DEF-10), а слишком длинное название компонента – 409 с вводящим в заблуждение текстом (DEF-13). Контракт, в котором 401 означает «нужна аутентификация», а 403 – «недостаточно прав», требует добавить AuthenticationEntryPoint и обработку исключений JWT в фильтре; клиент к такому изменению готов, так как уже обрабатывает 401.

3 РЕАЛИЗАЦИЯ ПРОТОТИПА ПРОГРАММНОЙ СИСТЕМЫ

3.1 Реализация базы данных и слоя доступа к данным

Серверная часть системы реализована на Java 17 с использованием Spring Boot 3.2 (модули Web, Data JPA, Security, Validation), Hibernate 6 и MySQL 8 [8], [10], [11]. Приложение построено по слоистой схеме «контроллер – сервис – репозиторий», рекомендованной для разделения ответственности между уровнями [22]: контроллеры пакета controller принимают HTTP-запросы и проверяют права доступа, сервисы пакета service содержат бизнес-логику и транзакции, репозитории пакета repository инкапсулируют доступ к базе данных, а наружу возвращаются объекты передачи данных из пакета dto, но не сущности. Подсистема сборки образов, очередь и разрешение зависимостей реализованы вторым участником команды и описаны в его части работы; в настоящем разделе они упоминаются только как смежная подсистема, использующая описанные здесь сущности.

Модель предметной области реализована шестью JPA-сущностями: User, Role, Permission, ComponentType, Component и Image. Физическая схема создаётся самим Hibernate (параметр ddl-auto=update), а стратегия именования PhysicalNamingStrategyStandardImpl сохраняет имена таблиц и столбцов в точности такими, как указано в аннотациях @Table и @Column, без преобразования регистра. Единственное исключение из простого именования – таблица пользователей: слово User зарезервировано в MySQL, поэтому в сущности имя таблицы явно заключено в обратные апострофы (@Table(name = "`User`")). Связи «многие ко многим» оформлены аннотациями @ManyToMany и @JoinTable и порождают пять связующих таблиц (user_role, role_premission, user_image_order, image_component_dependency и dependency) (таблица 10).

Таблица 10 – Сущности, таблицы и связи модели данных

Сущность	**Таблица**	**Связи**	**Особенности**
User	`User`	user_role (роли, EAGER); user_image_order (заказанные образы, LAZY)	реализует UserDetails; уникальный email
Role	Role	role_premission (права)	значения USER и ADMIN
Permission	Premission	–	зарезервирована схемой; в проверках доступа не используется
ComponentType	component_type	–	типы «Пакет» и «Библиотека»
Component	Component	ссылка на тип (@ManyToOne); dependency (самосвязь, LAZY)	@Version; два индекса
Image	Image	image_component_dependency (состав, LAZY)	@Version; два индекса; статус, путь к артефакту, журнал сборки

Ассоциации, содержащие потенциально большие коллекции (зависимости компонента и состав образа), объявлены с ленивой загрузкой (FetchType.LAZY). Роли пользователя, напротив, загружаются немедленно (FetchType.EAGER): они нужны при каждой авторизации, а их число не превышает двух. Сущности Component и Image снабжены полем версии с аннотацией @Version, что включает оптимистичную блокировку: при одновременном изменении одной записи двумя администраторами второй запрос завершится исключением ObjectOptimisticLockingFailureException, которое глобальный обработчик преобразует в ответ 409 с сообщением «Запись была изменена другим пользователем, обновите данные». Фрагмент сущности Component приведён в листинге 1.

Листинг 1 – Сущность Component (фрагмент)

```
@Entity
// Индексы под сортировку каталога по названию и фильтр по типу
// с сортировкой по цене (DEF-23).
@Table(name = "Component", indexes = {
        @Index(name = "idx_component_name", columnList = "name, id"),
        @Index(name = "idx_component_type_cost", columnList = "type, cost")
})
public class Component {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false)
    private String name;

    @ManyToOne
    @JoinColumn(name = "type", nullable = false)
    private ComponentType type;

    @Column(nullable = false)
    private Long cost;

    @Version
    @Column(name = "version")
    private Long version;

    // Самоссылающаяся связь «многие ко многим»
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "dependency",
        joinColumns = @JoinColumn(name = "parent_id"),
        inverseJoinColumns = @JoinColumn(name = "dependent_id"))
    private Set<Component> dependencies;
}
```

Индексы, объявленные в аннотациях @Table(indexes = ...), добавлены по итогам нагрузочного тестирования (дефект DEF-23, см. раздел 4.7). Для таблицы Component создан составной индекс по названию и идентификатору (сортировка каталога по умолчанию) и индекс по типу и стоимости (фильтр по типу с сортировкой по цене); для таблицы Image – индексы по статусу и стоимости и по одной стоимости. Поиск по подстроке (LIKE '%…%') индексом не ускоряется и по-прежнему требует полного прохода по таблице, что зафиксировано в отчёте о тестировании как частичное исправление.

Доступ к данным организован репозиториями Spring Data JPA: каждый интерфейс расширяет JpaRepository, а для сущностей, по которым выполняется динамический поиск (Component, Image, User), дополнительно JpaSpecificationExecutor. Это позволяет передавать в метод findAll объект Specification, собирающий условия запроса из необязательных параметров, и получать страницу результата с отдельным запросом COUNT. Остальные запросы выражены производными методами, имена которых разбираются Spring Data: findByRoleName и findByTypeIgnoreCase (справочники), findByState, existsByComponentsContains и existsByDependenciesContains (проверки использования компонента), findByOrderedImagesContains (образы

пользователя). Метод UserRepository.findByEmail снабжён аннотацией @EntityGraph(attributePaths = "roles"): он вызывается фильтром аутентификации при каждом запросе с токеном, и графом сущности роли загружаются тем же запросом с JOIN, а не отдельным.

Параметры работы Hibernate и пула соединений заданы в файле application.properties (таблица 11). Часть значений получена по итогам оптимизации по результатам нагрузочных испытаний: пул увеличен с 10 до 20 соединений, время ожидания соединения сокращено до 3 с (при перегрузке клиент быстро получает ответ 503 вместо ожидания 30 с), а порог пакетной догрузки ленивых коллекций задан равным 100 (дефекты DEF-12, DEF-22, DEF-23).

Таблица 11 – Параметры доступа к данным (application.properties)

Параметр	**Значение**	**Назначение**
spring.jpa.hibernate.ddl-auto	update	создание и неразрушающее расширение схемы средствами Hibernate
spring.jpa.hibernate.naming.physical-strategy	PhysicalNamingStrategyStandardImpl	имена таблиц и столбцов как в аннотациях
hibernate.default_batch_fetch_size	100	догрузка ленивых коллекций пачками с условием IN (устранение N+1)
spring.jpa.open-in-view	false	соединение не удерживается на время сериализации ответа
hikari.maximum-pool-size	20 (DB_POOL_SIZE)	размер пула соединений
hikari.connection-timeout	3 000 мс (DB_POOL_TIMEOUT_MS)	быстрый отказ при исчерпании пула
hikari.auto-commit	false	Hibernate не переключает autocommit вокруг каждой транзакции

Начальное наполнение базы выполняет компонент DataSeeder, реализующий интерфейс CommandLineRunner. Он срабатывает при запуске приложения и пропускает работу, если таблица компонентов не пуста, поэтому

повторные запуски не создают дублей. Компонент создаёт роли USER и ADMIN, типы «Пакет» и «Библиотека», каталог из 67 компонентов (семь именованных – python3, glibc, numpy, opencv-python, wiringpy, ffmpeg и openssh – с зависимостями, и 60 тестовых компонентов Dummy0–Dummy59, необходимых, чтобы каталог занимал несколько страниц), администратора admin@luckfox.io и 15 стартовых образов (три содержательных – CV Starter, IoT Edge и Media Box, и 12 тестовых). Образы заказываются от имени администратора через тот же метод ImageService.orderBuild, что и в редакторе, поэтому стартовые данные проходят полный конвейер: разрешение зависимостей, постановку в очередь и сборку.

3.2 Реализация аутентификации и управления пользователями

Аутентификация построена на токенах JWT (JSON Web Token) и не использует серверных сеансов. Точки входа расположены в контроллере AuthController: POST /api/auth/register создаёт учётную запись и сразу возвращает токен, POST /api/auth/login проверяет учётные данные и возвращает токен в теле AuthenticationResponse. Обе операции реализует AuthenticationService. При регистрации проверяется уникальность адреса электронной почты (иначе ответ 400 с сообщением «Пользователь с таким email уже существует»), пароль хешируется алгоритмом BCrypt через бин PasswordEncoder и новому пользователю назначается роль USER. При входе проверка выполняется стандартным AuthenticationManager: его провайдер DaoAuthenticationProvider получает пользователя через UserDetailsService, реализованный поиском по email, и сравнивает пароль с хешем. Пароли в открытом виде нигде не хранятся и не записываются в журнал.

Формат запроса на регистрацию контролируется аннотациями Jakarta Bean Validation в классе RegisterRequest: имя и email обязательны, email проверяется аннотацией @Email, пароль имеет длину не менее восьми символов и должен соответствовать регулярному выражению ^(?=.*[0-9])(?=.*[a-z])(?=.*[A-

Z])(?=.*[@#$%^&+=!]).*$, то есть содержать цифру, строчную и заглавную буквы и один из специальных символов @#$%^&+=!. Те же правила применены к классу UserRequest, используемому администратором. Нарушение ограничений приводит к ответу 400 с картой «поле – сообщение», которую формирует GlobalExceptionHandler.

Токен формируется классом JwtService на основе библиотеки jjwt 0.11.5: в него записываются адрес электронной почты (subject), время выпуска и время истечения, подпись выполняется алгоритмом HS256. Проверка токена (isTokenValid) сравнивает имя пользователя и срок действия. При тестировании выявлены два дефекта этого класса. Во-первых, срок жизни вычисляется выражением 1000 * 60 * 24 * 10 миллисекунд, что составляет 4 часа, тогда как комментарий в коде и проектная документация заявляют 24 часа (DEF-06, низкая критичность). Во-вторых, ключ подписи записан константой непосредственно в исходном коде и находится в репозитории; любой, кто имеет доступ к исходным текстам, способен выпустить токен администратора (DEF-07, критическая критичность, в сборке прототипа не исправлено).

Проверка токена на каждом запросе выполняется фильтром JwtAuthenticationFilter, унаследованным от OncePerRequestFilter. Фильтр читает заголовок Authorization, при наличии префикса Bearer извлекает из токена адрес электронной почты, загружает пользователя (одним запросом с ролями благодаря @EntityGraph), проверяет токен и помещает объект аутентификации в SecurityContextHolder. Запрос без заголовка проходит дальше без аутентификации и отклоняется правилами доступа. Правила и порядок фильтров заданы в SecurityConfig (листинг 2).

Листинг 2 – Цепочка фильтров безопасности (SecurityConfig)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthFilter;
    private final AuthenticationProvider authenticationProvider;
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/components/**").per-
mitAll()
                .requestMatchers(HttpMethod.GET, "/api/component-
types/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/images").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/images/*").permitAll()
                .anyRequest().authenticated())
            .sessionManagement(s -> s.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .authenticationProvider(authenticationProvider)
            .addFilterBefore(jwtAuthFilter,
                    UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Защита от подделки межсайтовых запросов (CSRF) отключена, так как токен передаётся в заголовке Authorization, а не в cookie, и сеансы не создаются (SessionCreationPolicy.STATELESS). Публичными являются регистрация, вход и чтение каталогов и библиотеки образов: шаблон /api/images/* охватывает только запрос образа по идентификатору, поэтому путь /api/images/{id}/download, содержащий два сегмента, требует авторизации. Все остальные запросы допускаются только для аутентифицированных пользователей. Более тонкие ограничения выражены аннотациями @PreAuthorize("hasRole('ADMIN')") на методах контроллеров; они работают благодаря @EnableMethodSecurity. Сводка правил приведена в таблице 12.

Таблица 12 – Правила доступа к ресурсам API

Категория ресурсов	**Пути и методы**	**Доступ**
Аутентификация	POST /api/auth/register, POST /api/auth/login	все
Чтение каталогов и библиотеки	GET /api/components, /api/components/{id}, /api/component-types, /api/images, /api/images/{id}	все
Заказ и скачивание образа	POST /api/images/build, GET /api/images/{id}/download	авторизованный пользователь
Профиль	GET /api/users/me, GET /api/users/me/images	авторизованный пользователь

Категория ресурсов	Пути и методы	Доступ
Изменение каталогов и образов	POST, PUT, DELETE /api/components; PUT, DELETE /api/images/{id}	ADMIN
Управление пользователями	GET, POST /api/users; GET, PUT, DELETE /api/users/{id}	ADMIN

Управление пользователями (класс UserService и контроллер UserController) доступно только администратору. Список пользователей выдаётся постранично методом search: в Specification объединяются поиск без учёта регистра по имени, фамилии и адресу электронной почты (оператор LIKE) и фильтр по роли через соединение с таблицей ролей (с признаком DISTINCT, чтобы пользователь с двумя ролями не дублировался); результат упорядочивается по идентификатору. Создание пользователя требует пароль, при изменении пустой пароль означает «оставить прежний». Уникальность адреса проверяется с исключением самого редактируемого пользователя, неизвестное имя роли приводит к ошибке 400. Введены две защиты от самоблокировки: администратор не может снять с себя роль ADMIN и не может удалить собственную учётную запись (в обоих случаях ответ 400 с пояснением). Заказанные пользователем образы при его удалении остаются в библиотеке, удаляются лишь связи в таблице user_image_order. Обычному пользователю предназначены методы GET /api/users/me (профиль) и GET /api/users/me/images (заказанные образы), используемые на странице аккаунта.

Для защиты входа от подбора паролей реализован сервис LoginAttemptService (дефект DEF-21, выявленный нагрузочным тестированием: за четыре минуты с одного адреса было принято 9 618 попыток входа, а вычисления BCrypt занимали около восьми ядер процессора). Сервис хранит в потокобезопасной коллекции ConcurrentHashMap счётчики неудачных попыток в фиксированном окне 300 с отдельно по адресу клиента (максимум 20) и по адресу электронной почты, приведённому к нижнему регистру (максимум 5). Метод checkAllowed вызывается до проверки пароля: при превышении любого лимита выбрасывается TooManyRequestsException, которую обработчик

преобразует в ответ 429 с заголовком Retry-After, содержащим оставшееся время блокировки в секундах, причём вычисления BCrypt не выполняются. Неудачный вход увеличивает оба счётчика (onFailure), успешный сбрасывает счётчик адреса почты (onSuccess). Лимиты заданы параметрами security.login.* файла application.properties. Устаревшие записи удаляются при превышении 10 000 ключей. Реализация хранит состояние в памяти процесса и рассчитана на один экземпляр серверной части; для нескольких экземпляров необходим общий репозиторий счётчиков, например Redis. Кроме того, адрес клиента определяется по getRemoteAddr, поэтому при размещении за обратным прокси все клиенты будут считаться имеющими адрес прокси.

3.3 Реализация каталогов и административных функций

Каталог компонентов обслуживают ComponentController и ComponentService. Метод search принимает необязательные параметры: подстроку названия, идентификатор типа, вид сортировки, номер и размер страницы. Условия собираются в Specification и объединяются логическим И: сравнение названия выполняется без учёта регистра, фильтр по типу – по идентификатору связанной сущности. Пагинация выполняется на стороне СУБД (LIMIT/OFFSET и отдельный COUNT), а не в памяти приложения. Справочник типов отдаёт ComponentTypeController (GET /api/component-types) для заполнения выпадающих списков интерфейса.

Единый способ формирования страницы, сортировки и ограничения параметров вынесен в класс PageResponse, общий для библиотек компонентов, образов и списка пользователей. Метод from преобразует страницу Spring Data в объект с полями content, page, size, totalElements и totalPages, применяя функцию преобразования сущности в DTO. Метод request защищает сервер от неверных значений: номер страницы не может быть отрицательным, размер страницы ограничен диапазоном от 1 до 100 (MAX_SIZE), так что запрос size=100000 не приводит к выгрузке всей таблицы. Метод sort принимает значения price_asc и

price_desc, задающие сортировку по стоимости; при равной цене используется порядок по умолчанию (для компонентов – по названию и идентификатору, для образов – от новых к старым), поэтому порядок строк детерминирован и страницы не пересекаются.

Изменение каталога доступно администратору. Метод apply сервиса ComponentService, общий для создания и редактирования, проверяет существование типа и каждой зависимости (иначе 404), запрещает зависимость компонента от самого себя и выявляет циклы, обходом в ширину проверяя, не зависит ли уже добавляемая зависимость от редактируемого компонента (метод dependsOn). Удаление компонента отклоняется с ответом 400, если он входит в состав какого-либо образа или служит зависимостью другого компонента; проверки выполняют методы existsByComponentsContains и existsByDependenciesContains. Это гарантирует, что удаление не нарушает целостность уже собранных образов и графа зависимостей. Дополнительно нарушения внешних ключей, не пойманные явными проверками, глобальный обработчик преобразует в ответ 409.

Поиск в библиотеке образов реализован методом ImageService.search (листинг 3); заказ, изменение и сборка образов относятся к подсистеме второго участника и здесь не рассматриваются. Метод принимает подстроку названия, статус, плату, список идентификаторов компонентов, вид сортировки и параметры страницы. Фильтр по компонентам возвращает образы, содержащие все перечисленные компоненты: для каждого идентификатора в Specification добавляется коррелированный подзапрос EXISTS, поэтому в основной выборке не возникает дублей строк, характерных для соединения по связи «многие ко многим», и подсчёт общего числа записей остаётся корректным. Повторяющиеся идентификаторы предварительно удаляются через LinkedHashSet.

Листинг 3 – Динамический поиск образов (ImageService.search, фрагмент)

```
Specification<Image> spec = (root, query, cb) -> {
    List<Predicate> where = new ArrayList<>();
    if (name != null && !name.isBlank()) {
```

```
            where.add(cb.like(cb.lower(root.get("name")),
                    "%" + name.trim().toLowerCase() + "%"));
        }
        if (state != null && !state.isBlank()) {
            where.add(cb.equal(root.get("state"), state.toUpperCase()));
        }
        if (board != null && !board.isBlank()) {
            where.add(cb.equal(root.get("board"), board));
        }
        if (componentIds != null) {
            for (Long componentId : new LinkedHashSet<>(componentIds)) {
                Subquery<Long> sq = query.subquery(Long.class);
                Root<Image> img = sq.from(Image.class);
                Join<Image, Component> comp = img.join("components");
                sq.select(img.get("id")).where(
                        cb.equal(img.get("id"), root.get("id")),
                        cb.equal(comp.get("id"), componentId));
                where.add(cb.exists(sq));
            }
        }
        return cb.and(where.toArray(new Predicate[0]));
    };
```

Перечень эндпоинтов каталогов и административных функций сведён в таблицу 13. Значения размера страницы по умолчанию: 12 для образов, 20 для компонентов и пользователей.

Таблица 13 – Эндпоинты каталогов и администрирования

Метод и путь	**Доступ**	**Назначение**
GET /api/components	все	страница каталога: search, type, sort, page, size
GET /api/components/{id}	все	компонент с типом и списком зависимостей
POST /api/components	ADMIN	создание компонента; 400 при циклической зависимости
PUT /api/components/{id}	ADMIN	изменение компонента и его зависимостей
DELETE /api/components/{id}	ADMIN	удаление; 400, если компонент используется
GET /api/component-types	все	справочник типов компонентов
GET /api/images	все	библиотека образов: search, state, board, components, sort, page, size
GET /api/users	ADMIN	страница пользователей: search, role, page, size
POST /api/users	ADMIN	создание пользователя (пароль обязателен)
PUT /api/users/{id}	ADMIN	изменение; нельзя снять с себя роль ADMIN
DELETE /api/users/{id}	ADMIN	удаление; нельзя удалить себя

3.4 Реализация пользовательского интерфейса и навигации

Клиентское приложение представляет собой одностраничное приложение (SPA) на React 18 [12], собираемое Vite 5. В качестве маршрутизатора

используется react-router-dom 6, для запросов – Axios 1.6, для хранения состояния редактора образа – Zustand 4.5. Точка входа main.jsx оборачивает корневой компонент App в BrowserRouter и провайдер AuthProvider, обеспечивающий доступ к состоянию аутентификации из любого компонента. Пока профиль пользователя не загружен, App выводит индикатор «Загрузка…», чтобы страницы не мигали как для анонимного пользователя.

Маршруты объявлены в компоненте App.jsx. Библиотеки образов и компонентов доступны без входа, редактор образа и страница аккаунта требуют аутентификации, редактор компонентов и управление пользователями – роли ADMIN. Ограничение реализует компонент ProtectedRoute: неаутентифицированного пользователя он перенаправляет на страницу /login, запоминая исходный адрес в состоянии перехода, а пользователя без роли ADMIN при параметре adminOnly – в библиотеку образов. Неизвестные пути и корень сайта перенаправляются на /library/images. Клиентская проверка служит только удобству: реальная защита данных обеспечивается сервером (раздел 3.2). Структура страниц приведена в таблице 14.

Таблица 14 – Страницы клиентского приложения

Маршрут	**Компонент**	**Доступ**	**Содержание**
/library/images	ImageLibrary	все	карточки образов, поиск, фильтры, скачивание; для ADMIN – «Изменить» и «Удалить»
/library/components	ComponentLibrary	все	каталог компонентов, поиск, фильтр по типу, сортировка по цене
/editor/image	ImageEditor	авторизованный	выбор компонентов и заказ образа; для ADMIN – изменение образа по параметру id
/editor/component	ComponentEditor	ADMIN	создание и изменение компонентов, список существующих
/admin/users	UserManagement	ADMIN	форма пользователя и постраничный список с поиском и фильтром по роли
/account	Account	авторизованный	профиль (имя, email, роли) и «Мои образы»
/login, /register	Login, Register	все	вход и регистрация

Верхнее меню формирует компонент Header, учитывающий роль пользователя. Всем посетителям показываются пункты «Образы» и «Компоненты» и кнопки «Вход» и «Регистрация»; авторизованному пользователю добавляются пункт «Создать образ», ссылка «Аккаунт» и кнопка «Выйти»; администратору – пункты «+ Компонент» и «Пользователи». Признак isAdmin вычисляется в AuthContext по наличию значения ADMIN в списке ролей профиля.

Страница библиотеки образов (рисунок 2) содержит панель фильтров и сетку карточек. Панель включает строку «Поиск образов…», выпадающие списки «Все статусы» (значения берутся из словаря меток статусов: «Создан», «В очереди», «Сборка», «Готов», «Ошибка»), «Все платы» и «Сначала новые» (альтернативы – сортировка по цене), а также компонент ComponentFilter «Фильтр по компонентам…»: он показывает подсказки из каталога, а выбранные компоненты отображаются удаляемыми метками. Карточка ImageCard выводит название, статус в виде цветной метки StatusBadge, описание, дистрибутив и плату, первые шесть компонентов состава (остальные сворачиваются в метку «+N») и стоимость. Кнопка «Скачать .img» появляется только у образа, готового к скачиванию; администратору доступны кнопки «Изменить» и «Удалить», блокируемые во время сборки (изменение недоступно для образа в очереди и на сборке, удаление – для собираемого). Нумерацию страниц выводит компонент Pagination в виде «Стр. 1 из 2 · всего 15» с кнопками перехода на первую, предыдущую, следующую и последнюю страницы; при единственной странице выводится только общее число записей.

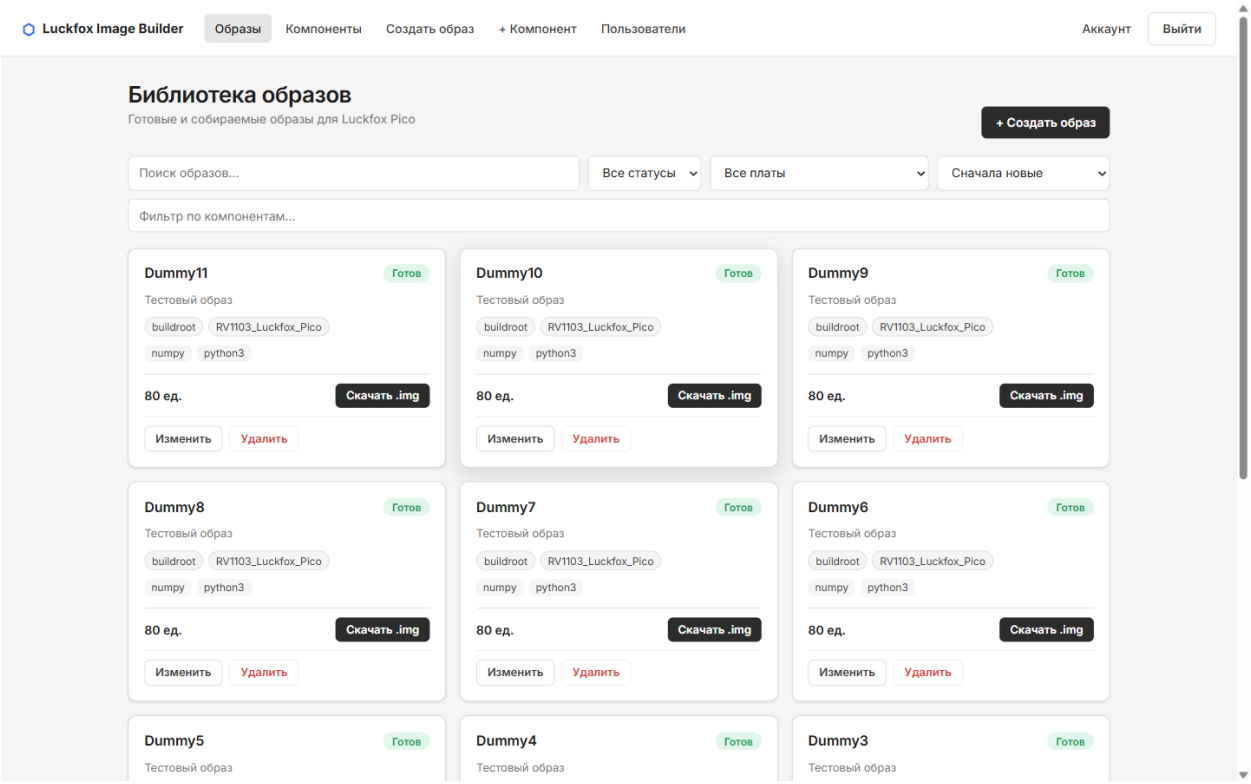


Рисунок 2 – Библиотека образов

Общая логика постраничных списков вынесена в хук usePagedList. Он принимает функцию запроса, объект фильтров и параметры (размер страницы, интервал опроса). При изменении любого фильтра страница сбрасывается на первую; ввод в строку поиска сглаживается задержкой 250 мс (debounce), чтобы не отправлять запрос на каждую букву; порядковый номер запроса позволяет отбрасывать устаревшие ответы, пришедшие позже более свежих; при удалении последней записи на странице происходит возврат на предыдущую. Для библиотеки образов включён периодический опрос с интервалом 5 с, благодаря которому статусы сборок обновляются без перезагрузки страницы. Один и тот же хук используется во всех постраничных списках интерфейса.

Библиотека компонентов (рисунок 3) построена по тому же шаблону: строка поиска, выпадающий список типов «Все типы», сортировка («По названию», «Цена: по возрастанию», «Цена: по убыванию») и сетка карточек ComponentCard, которая показывает название, тип, описание, метки зависимостей и стоимость. Администратору доступны кнопки «Изменить» и

«Удалить» и переход к редактору. Карточка поддерживает необязательную кнопку действия, что позволяет использовать её и в других сценариях выбора компонентов.

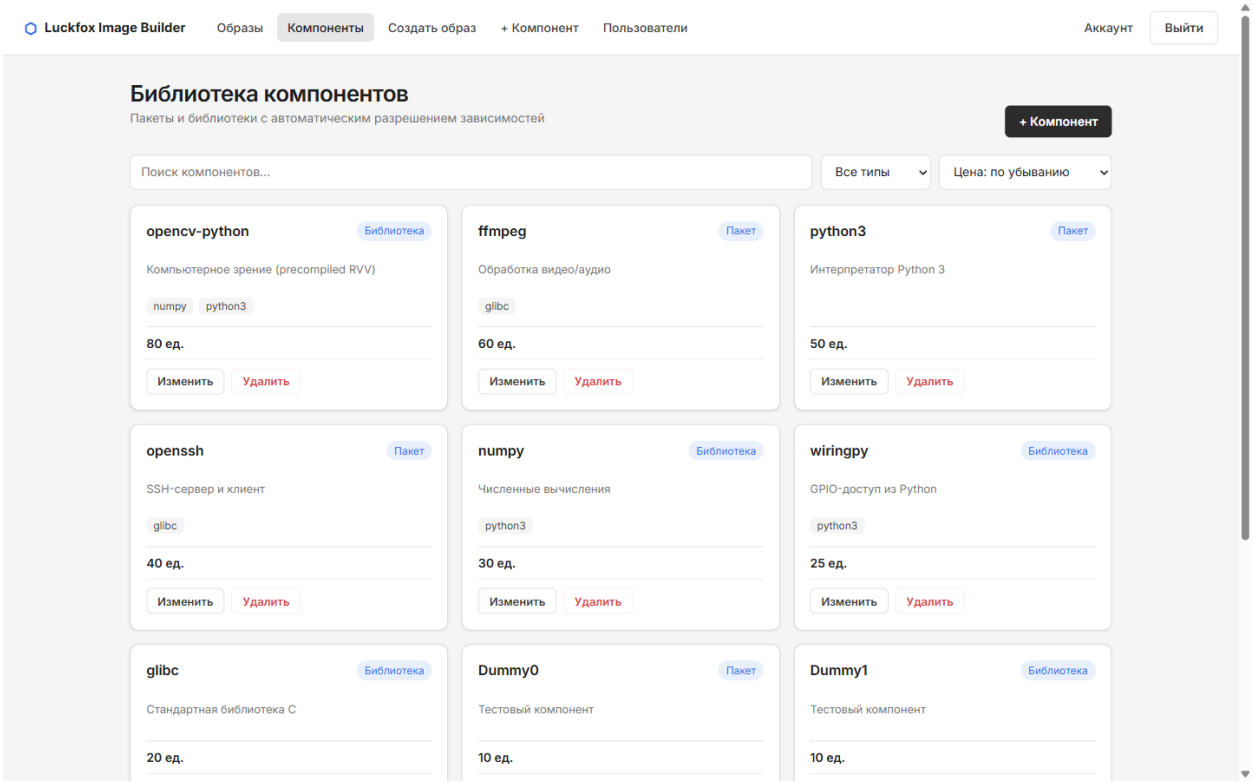


Рисунок 3 – Библиотека компонентов

Редактор образа (ImageEditor) хранит состояние в хранилище Zustand editorStore: название, описание, дистрибутив, плату, массив выбранных компонентов и идентификатор редактируемого образа. Хранилище содержит действия добавления и удаления компонента (повторное добавление игнорируется), вычисление суммарной стоимости выбранных компонентов и заполнение полей существующим образом для режима изменения. Использование хранилища вместо локального состояния компонента позволяет сохранять выбор при переходах между страницами. Кнопка отправки называется «Заказать образ», а в режиме изменения администратором – «Сохранить и пересобрать»; после успешной отправки редактор очищается и пользователь возвращается в библиотеку. Редактор компонентов (рисунок 4) содержит форму с названием, типом, описанием, стоимостью и выбором зависимостей и список

существующих компонентов с кнопками редактирования и удаления; кнопка отправки называется «Создать компонент» (при редактировании – «Сохранить изменения»). Серверные проверки, например запрет циклической зависимости, показываются над формой сообщением из ответа сервера.

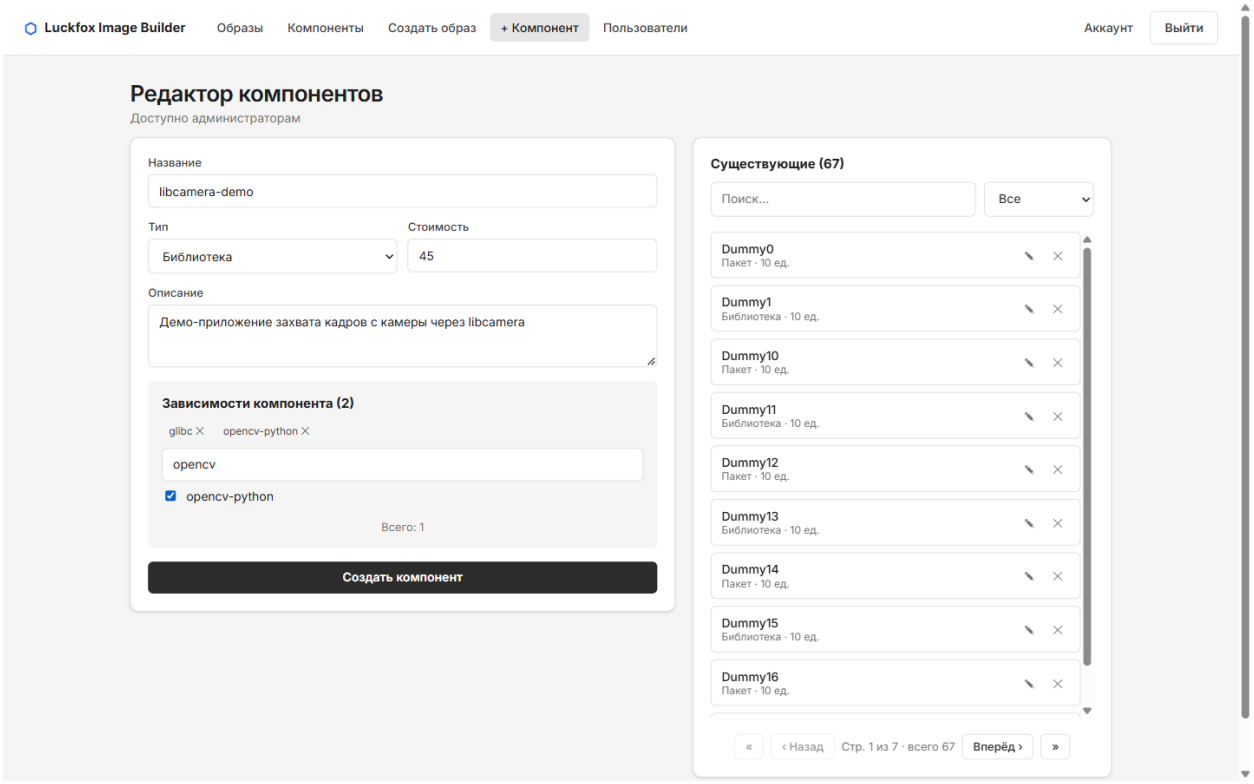


Рисунок 4 – Редактор компонентов

Страница управления пользователями (рисунок 5) состоит из формы создания и редактирования и списка. Форма содержит имя, фамилию, адрес электронной почты, пароль и флажки ролей (USER и ADMIN); при редактировании поле пароля озаглавлено «Новый пароль (пусто – не менять)», а под ним выводится подсказка о требованиях к паролю. Кнопка отправки называется «Создать пользователя» (при редактировании – «Сохранить»). Список пользователей выводится постранично, содержит строку поиска «Имя или email…» и фильтр «Все роли», а кнопка удаления собственной учётной записи заблокирована, что дублирует серверную защиту.

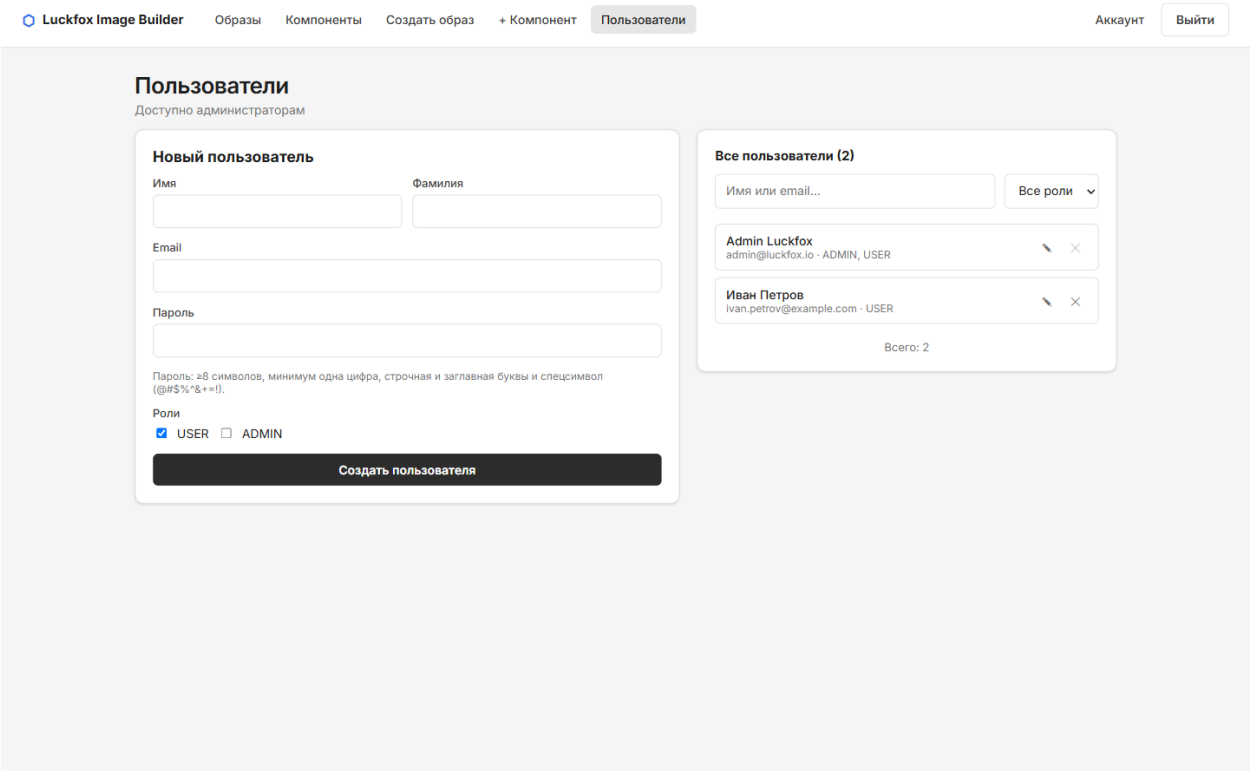


Рисунок 5 – Управление пользователями

3.5 Взаимодействие клиентского приложения с REST API

Обмен с сервером сосредоточен в каталоге frontend/src/api: модуль client.js создаёт настроенный экземпляр Axios, а модуль api.js предоставляет прикладные функции по одной на эндпоинт (apiLogin, apiRegister, apiGetMe, apiGetComponents, apiCreateComponent, apiGetImages, apiBuildImage, apiGetUsers и другие). Компоненты страниц не формируют URL самостоятельно, а вызывают эти функции, поэтому изменение контракта серверной части локализуется в одном файле. Базовый адрес берётся из переменной окружения VITE_API_URL, подставляемой Vite при сборке; если она не задана, используется относительный путь /api, который в режиме разработки проксируется на Spring Boot (порт 8080) правилом server.proxy файла vite.config.js, что избавляет разработчика от проблем CORS. Модуль client.js регистрирует два перехватчика (листинг 4).

Листинг 4 – Перехватчики Axios (client.js)

```
const baseURL = import.meta.env.VITE_API_URL || '/api';
const client = axios.create({ baseURL });
```

```
// Подставляем JWT в каждый запрос.
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// При 401 – сбрасываем сессию и уводим на вход.
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      if (!window.location.pathname.startsWith('/login')) {
        window.location.assign('/login');
      }
    }
    return Promise.reject(error);
  }
);
```

Перехватчик запросов добавляет заголовок Authorization: Bearer <токен> ко всем обращениям, если токен есть в localStorage. Перехватчик ответов при статусе 401 удаляет токен и перенаправляет пользователя на страницу входа. Однако сервер при отсутствии или недействительности токена, как и при неверном пароле, отвечает не 401, а 403 без тела: используется стандартная обработка Spring Security, которая при отсутствии настроенной точки входа (authenticationEntryPoint) возвращает 403. В результате ветка обработки 401 не срабатывает: пользователь с просроченным токеном остаётся на защищённой странице и видит ошибки запросов (дефект DEF-04, тест E2E-06), а форма входа при неверном пароле показывает техническое сообщение «Request failed with status code 403» вместо понятного «Неверный email или пароль» (дефект DEF-03, тест E2E-08). Дефекты не приводят к утечке данных, но ухудшают удобство работы; устранение состоит в настройке authenticationEntryPoint с ответом 401 (см. приложение Ж).

Состояние аутентификации хранит AuthContext. Токен сохраняется в localStorage под ключом token и восстанавливается при загрузке страницы. При появлении токена контекст запрашивает профиль GET /api/users/me и сохраняет его в состоянии user; значение isAuthenticated определяется наличием токена, а

isAdmin – наличием роли ADMIN в профиле. Функции login и register вызывают соответствующие методы API, сохраняют токен и возвращают объект { success, message }, из которого страницы формируют сообщение об ошибке; logout очищает токен и профиль. Читаемое сообщение об ошибке извлекает функция extractError: она берёт текст из поля error ответа, а при карте ошибок валидации объединяет её значения, и только при отсутствии тела ответа показывает исходное сообщение Axios. Именно из-за отсутствия тела у ответа 403 пользователю попадает техническая строка Axios.

Скачивание образа реализовано отдельно от Axios. Так как файл защищён токеном, простая ссылка на страницу не может передать заголовок Authorization, поэтому обработчик в ImageLibrary и Account вызывает fetch с заголовком Bearer, преобразует ответ в объект Blob, создаёт временную ссылку URL.createObjectURL, программно нажимает скрытый элемент a с атрибутом download (имя файла – название образа с расширением .img) и освобождает ссылку. При ответе, отличном от успешного, пользователю показывается сообщение «Загрузка недоступна».

3.6 Конфигурация и развёртывание приложения

Настройки серверной части хранятся в application.properties, а значения, различающиеся между средами, вынесены в переменные окружения (таблица 15). Для параметров сборки используется синтаксис ${ИМЯ:значение_по_умолчанию}, для источника данных – механизм свободной привязки Spring Boot, при котором переменные SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME и SPRING_DATASOURCE_PASSWORD переопределяют одноимённые свойства [8]. По умолчанию приложение обращается к MySQL на localhost:3306 (база luckfox_db, параметр createDatabaseIfNotExist=true создаёт её при отсутствии) с учётной записью root.

Таблица 15 – Переменные окружения и параметры запуска

Переменная	**Значение по умолчанию**	**Назначение**
SPRING_DATASOURCE_URL	jdbc:mysql://localhost:3306/luckfox_db	адрес базы данных (в Compose – хост db)
SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD	root, root	учётная запись СУБД
DB_POOL_SIZE	20	размер пула соединений
DB_POOL_TIMEOUT_MS	3000	время ожидания соединения из пула
LUCKFOX_BUILD_SCRIPT	../scripts/luckfox_build.sh	путь к сценарию сборки
LUCKFOX_OUTPUT_DIR	../output/images	каталог готовых образов
LUCKFOX_BUILD_TIMEOUT	1800	предельное время сборки, с
LUCKFOX_SDK_DIR, LUCKFOX_SIM_DELAY	в Compose: /workspace/luckfox-pico; 0	каталог SDK и задержка имитации сборки (используются сценарием сборки)
VITE_API_URL	/api (в Compose: http://localhost:8080/api)	адрес API, встраиваемый в клиент при сборке
VITE_BACKEND_URL	http://localhost:8080	цель прокси Vite в режиме разработки
PORT	5173	порт статического сервера клиента
MYSQL_DATABASE, MYSQL_ROOT_PASSWORD	luckfox_db, root	инициализация контейнера MySQL

Совместное использование клиента и сервера с разных адресов разрешено конфигурацией CORS в классе WebConfig: для путей /api/** допускается источник http://localhost:5173, методы GET, POST, PUT, DELETE и OPTIONS, любые заголовки и передача учётных данных. Список источников задан в коде, поэтому для доступа к клиенту с иного адреса или из иного домена его необходимо расширить, а лучше вынести в конфигурацию.

Развёртывание описано файлом docker-compose.yml [18] и включает три сервиса (листинг 5). Сервис db использует образ mysql:8.0 и хранит данные в именованном томе mysql_data. Сервис backend собирается из каталога backend,

публикует порт 8080 и получает параметры подключения к базе, пути к сценарию и каталогу вывода сборки через переменные окружения. Сервис frontend собирается из каталога frontend, публикует порт 5173 и принимает в аргументе сборки VITE_API_URL адрес API. Готовые образы сохраняются в именованном томе build_output, смонтированном в /workspace/output, поэтому файлы не пропадают при пересоздании контейнера; сценарии сборки и SDK подключаются только для чтения.

Листинг 5 – Основные части docker-compose.yml

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: luckfox_db
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-proot"]
      interval: 10s
      timeout: 5s
      retries: 10
  backend:
    build: ./backend
    ports: ["8080:8080"]
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/luckfox_db?...
      LUCKFOX_BUILD_SCRIPT: /workspace/scripts/luckfox_build.sh
      LUCKFOX_OUTPUT_DIR: /workspace/output/images
    volumes:
      - ./scripts:/workspace/scripts:ro
      - ./luckfox-pico:/workspace/luckfox-pico:ro
      - build_output:/workspace/output
  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_URL: http://localhost:8080/api
    ports: ["5173:5173"]
    depends_on: [backend]
volumes:
  mysql_data:
  build_output:
```

Порядок запуска обеспечивается двумя средствами. Проверка работоспособности (healthcheck) сервиса db выполняет команду mysqladmin ping каждые 10 с (до десяти повторов), а условие depends_on: condition:

service_healthy откладывает запуск серверной части до готовности СУБД: без него приложение при старте не смогло бы подключиться к базе и завершилось бы с ошибкой. Клиент запускается после серверной части, но готовность последней не проверяет.

Для каждого из двух образов предусмотрено два варианта описания. Основные файлы backend/Dockerfile и frontend/Dockerfile рассчитаны на среду, где Docker-демон внутри WSL2 не разрешает DNS-имена на этапах сборки образа, и потому не могут загружать зависимости из сети. Поэтому артефакты собираются на хосте (mvn package и npm run build), а образ лишь копирует готовый jar-файл в образ eclipse-temurin:17-jre-jammy, а каталог dist – в образ node:18-slim. Полноценные многоступенчатые варианты (Dockerfile.multistage) собирают приложения внутри контейнера образами maven и node и предназначены для сред с рабочим DNS; в серверном образе дополнительно устанавливаются bash, e2fsprogs и coreutils, необходимые сценарию сборки образов. В основном варианте серверного образа mkfs.ext4 отсутствует, и сценарий формирует артефакт в виде tar-архива (см. смежную подсистему сборки). В основной Dockerfile серверной части добавлена инструкция HEALTHCHECK, обращающаяся к /actuator/health, однако модуль Spring Boot Actuator в зависимости проекта не входит, поэтому эта проверка в текущей конфигурации не может завершаться успехом; на порядок запуска сервисов Compose это не влияет, так как он опирается на проверку сервиса db.

Статические файлы клиента отдаёт минималистичный сервер server.mjs на Node.js без внешних зависимостей: для запроса он определяет путь внутри каталога dist (с защитой от выхода за его пределы, ответ 403), возвращает файл с типом содержимого по расширению, а для неизвестных путей отдаёт index.html, обеспечивая работу клиентской маршрутизации при прямом открытии адресов вида /library/images. Тестирование выявило особенность такого решения: для несуществующих файлов в каталоге /assets сервер тоже отдаёт index.html со

статусом 200 вместо 404, из-за чего браузер получает HTML вместо скрипта (дефект DEF-01, низкая критичность).

Единый запуск всего стека выполняет сценарий scripts/run-docker.sh. Он задаёт JAVA_HOME для Java 17, собирает серверную часть командой mvn package с пропуском тестов и клиентскую командой npm run build с адресом API из переменной VITE_API_URL, после чего выполняет docker-compose up --build; допускаются ключи -d (фоновый режим), --skip-build и подкоманда down. Следует учитывать особенность сценария: первой командой он выполняет docker-compose down -v, которая останавливает контейнеры и удаляет тома Compose, то есть базу данных и собранные образы. Поэтому каждый запуск через этот сценарий возвращает систему в исходное состояние, что удобно для воспроизводимых испытаний, но недопустимо для среды с данными, которые необходимо сохранить; в такой среде следует запускать docker compose up напрямую.

После первого старта на пустой базе DataSeeder создаёт учётную запись администратора admin@luckfox.io с паролем Admin@123, который также записывается в журнал запуска. Эта запись предназначена для демонстрации и испытаний; в реальной эксплуатации пароль следует немедленно изменить через страницу управления пользователями. Для разработки серверная часть запускается командой mvn spring-boot:run (при доступной СУБД, которую поднимает docker compose up -d db), клиент – командой npm run dev, после чего приложение доступно по адресу http://localhost:5173.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ

4.1 Тестовый стенд, методика и критерии оценки

Тестирование прототипа выполнялось на четырёх уровнях: модульном (изолированная проверка классов серверной части), интеграционном (проверка развёрнутой системы по HTTP с реальной СУБД, файловой системой и внешним процессом сборки), сквозном (сценарии пользователя в браузере) и нагрузочном (поведение под заданной интенсивностью запросов) [23]. Для проверки эффекта оптимизации нагрузочные и интеграционные тесты выполнены дважды – до и после внесения исправлений.

Стенд развёрнут в WSL2 (Ubuntu 24.04) и повторяет топологию Docker Compose: MySQL 8.0 на порту 3306, серверная часть в виде jar-файла на порту 8080 и собранное клиентское приложение, раздаваемое сервером Node.js на порту 5173 и обращающееся к API по абсолютному адресу (с проверкой CORS). Характеристики стенда приведены в таблице 16.

Таблица 16 – Характеристики тестового стенда

Параметр	**Значение**
Процессор	AMD Ryzen 7 5800HS, 16 логических ядер, выделенных WSL2
Оперативная память	7 638 МБ (лимит WSL2)
Операционная система	Ubuntu 24.04 LTS в WSL2, ядро 6.6.87.2; хост – Windows 11 Enterprise LTSC 2024
JDK и параметры JVM	OpenJDK 17.0.20; -Xms512m -Xmx1g -XX:+UseG1GC
Серверная часть	Spring Boot 3.2.0, Hibernate 6, HikariCP, встроенный Tomcat (200 потоков)
СУБД	MySQL 8.0.46, параметры по умолчанию (innodb_buffer_pool_size 128 МБ)
Клиентская часть	React 18 + Vite 5, производственная сборка, Node.js 24
Сборка образов	luckfox_build.sh в режиме симуляции, задержка 3 с, один воркер
Инструменты	JUnit 5, Mockito, AssertJ; Python 3.12 + requests; Playwright (Chromium); Grafana k6 0.54
Мониторинг	CPU, RSS и потоки процессов (/proc), heap и сборка мусора (jstat), соединения и число SQL-запросов MySQL (SHOW GLOBAL STATUS) с шагом 1 с

Интеграционные тесты построены по принципу чёрного ящика: каждый тест создаёт собственные данные (уникальные адреса и названия) и проверяет не только HTTP-ответы, но и состояние базы данных (через information_schema и счётчики) и артефакты на диске. Финальные прогоны выполнялись на полностью пересозданной базе данных, наполненной начальными данными (67 компонентов, 15 образов, собранных через тот же конвейер, администратор).

Критерии успеха нагрузочного тестирования определены на основе нефункциональных требований: доля ошибок менее 1 %, 95-й процентиль (p95) времени отклика на чтение менее 500 мс, на запись менее 1 000 мс, на вход менее 1 000 мс, на скачивание образа размером 18 МБ менее 3 000 мс; для стрессовых профилей фиксировалась точка насыщения.

4.2 Модульное тестирование

Модульные тесты размещены в каталоге backend/src/test/java с повторением структуры пакетов приложения и выполняются командой mvn test. Использованы JUnit 5 [14], Mockito [15] (расширение MockitoExtension со строгой проверкой заглушек) и AssertJ. Каждый тест проверяет один класс, а его зависимости (репозитории, кодировщик паролей, другие сервисы) заменены заглушками, поэтому тесты не требуют базы данных и выполняются за несколько секунд. Проверки, обращающиеся к развёрнутому API, к модульным не относятся и описаны в разделе 4.3.

Всего разработано 84 теста в 13 классах: сервисы (ImageService, BuildService, BuildStateService, ComponentService, UserService, AuthenticationService, LoginAttemptService, JwtService), DTO и постраничный вывод, валидация запросов, глобальный обработчик ошибок и сущность User. Для подсистемы сборки разработано 29 тестов (таблица 17); полный состав тестов приведён в приложении Ж.

Таблица 17 – Модульные тесты подсистемы сборки

Класс	**Тестов**	**Основные проверки**
ImageServiceTest	11	Транзитивное разрешение зависимостей и однократный учёт стоимости; плата по умолчанию; привязка к заказчику и постановка в очередь; отказ при неизвестном компоненте, пользователе и циклическом графе; запрет изменения в QUEUED и BUILDING; пересборка с удалением старого артефакта; удаление с отвязкой заказчиков и файла
BuildServiceTest	6	Успешная сборка: READY, путь, журнал, уведомление; передача компонентов сценарию; FAILED при ненулевом коде и при отсутствии файла; принудительное завершение по таймауту; отказ до перехода в BUILDING для удалённого образа
BuildStateServiceTest	4	Отсортированный снимок входных данных; сохранение артефакта и журнала; сохранение журнала при переходе в BUILDING; ошибка для отсутствующего образа
ComponentServiceTest	8	Создание с типом и зависимостями; неизвестный тип и зависимость; запрет зависимости от себя и транзитивного цикла; запрет удаления используемого компонента

Тесты BuildService запускают настоящий процесс: вместо luckfox_build.sh во временном каталоге (@TempDir) создаётся сценарий-заглушка, который печатает строки журнала, создаёт или не создаёт выходной файл, завершается с нужным кодом или «зависает». Это позволяет проверить обработку вывода, кодов возврата и таймаута без SDK; так как тесты требуют bash, они выполняются только в Linux и macOS (аннотация @EnabledOnOs). Постановка в очередь после фиксации транзакции проверяется косвенно: вне транзакции действие afterCommit выполняется сразу, и тест фиксирует вызов buildService.build с идентификатором нового образа.

Результат прогона в WSL (Ubuntu 24.04, JDK 17, Maven): выполнено 84 теста, ошибок и пропусков нет (BUILD SUCCESS). Строки уровня ERROR в выводе прогона являются ожидаемыми сообщениями сценариев неуспешной сборки.

4.3 Интеграционное тестирование серверной части

Интеграционные тесты реализованы сценарием на Python (tests/integration/run_integration.py) с библиотекой requests и выполняются против развёрнутого стенда. Тесты сгруппированы по этапам: развёртывание (статические файлы, CORS, схема БД, начальное наполнение), аутентификация и доступ, каталог и пагинация, компоненты, заказ и сборка образа, управление пользователями и отказоустойчивость. Всего 47 API-тестов; результаты по этапам приведены в таблице 18, полный протокол – в приложении Ж.

Таблица 18 – Результаты интеграционного и сквозного тестирования (исходный прогон)

Этап	**Всего**	**Успешно**	**Выявлены отклонения**
1. Развёртывание	7	5	2
2. Аутентификация и доступ	11	4	7
3. Каталог и пагинация	5	2	3
4. Компоненты (ADMIN)	5	4	1
5. Заказ и сборка образа	12	8	4
6. Управление пользователями	3	2	1
7. Отказоустойчивость	4	2	2
8. Сквозные тесты в браузере	9	7	2
Итого	56	34	22

Основные бизнес-сценарии работают корректно: регистрация и вход, каталоги с поиском, фильтрами, сортировкой и пагинацией, защита графа зависимостей, заказ, сборка и скачивание валидного образа, администрирование, очередь сборок с одним воркером, оптимистичная блокировка при одновременном редактировании (из 10 одновременных изменений компонента принято одно, остальные получили 409) и защита от гонки при одновременной регистрации одного адреса. Большинство отклонений этапа аутентификации имеют общую причину: Spring Security отвечает кодом 403 без тела там, где ожидаются 401, 400 или 503, так как не настроена точка входа аутентификации, а путь /error закрыт для анонимных запросов. Из-за этого клиентское приложение не перенаправляет пользователя на страницу входа при истёкшем токене.

4.4 Сквозное тестирование пользовательских сценариев

Сквозные тесты выполнены средствами Playwright [17] в браузере Chromium с хоста Windows против того же стенда. Проверены девять сценариев: открытие главной страницы с перенаправлением в библиотеку и отрисовкой карточек (первая карточка – через 412 мс), регистрация через форму, фильтр по компоненту с сортировкой по цене, заказ сборки в редакторе, скачивание образа из интерфейса, поведение при недействительном токене, поиск пользователя администратором, сообщение при неверном пароле и отсутствие необработанных ошибок JavaScript за весь сценарий.

Ключевой для подсистемы сборки сценарий E2E-04 воспроизводит полный путь пользователя: в редакторе образа задаётся название, в каталоге находится и добавляется компонент ffmpeg, выполняется заказ, после чего в разделе «Мои образы» образ доходит до статуса «Готов», а зависимость glibc оказывается добавленной автоматически. Сценарий E2E-05 нажимает «Скачать .img» и проверяет, что браузер сохраняет файл «E2E образ.img» размером 18 874 368 байт, что соответствует 16 МБ базового образа и 2 МБ на два компонента.

Два сценария выявили отклонения, связанные с упомянутой обработкой ошибок безопасности: при подменённом токене пользователь остаётся на странице аккаунта с сообщением «Request failed with status code 403» вместо перенаправления на вход (DEF-04), а при неверном пароле показывается техническое сообщение вместо «Неверный email или пароль» (DEF-03).

4.5 Проверка конвейера сборки и отказоустойчивости

Проверки конвейера сборки входят в этапы 5 и 7 интеграционного тестирования и приведены в таблице 19.

Таблица 19 – Проверки конвейера сборки и отказоустойчивости

Тест	**Проверка**	**Результат**
INT-07	Начальное наполнение: 15 старто-вых образов проходят реальную	PASS: 15 из 15 READY

Тест	Проверка	Результат
	очередь сборки	
INT-25	Зависимость компонента от себя и цикл A → B → A	PASS: 400 в обоих случаях
INT-29	Заказ [opencv-python] с транзитивными зависимостями	PASS: 202, QUEUED, состав {numpy, opencv-python, python3}, стоимость 160
INT-30	Сборка до READY и появление в «Мои образы»	PASS: READY за 2,1 с
INT-31	Содержимое скачанного образа	PASS: ext4 (сигнатура 0xEF53), 19 МБ, манифест с составом
INT-36	Очередь: 6 последовательных и 6 одновременных заказов	PASS: не более одной сборки одновременно, порядок FIFO, 12 сборок за 35,5 с
INT-37	Изменение образа администратором	PASS: пересборка за 3,6 с, новый состав; изменение во время сборки – 400
INT-38	Удаление образа	PASS: запись, связь и файл удалены
INT-40	Файл артефакта удалён с диска вне приложения	FAIL: кнопка скачивания остаётся, скачивание – 400 (DEF-17)
INT-44	Штатный перезапуск серверной части	PASS: данные сохранены, дублей нет, токен действителен
INT-45	Аварийное завершение (kill -9) во время сборки	FAIL: образы остались в BUILDING и QUEUED и не обрабатываются (DEF-19)
INT-46	Недоступность MySQL	FAIL: ответ 403 через 28,3 с (DEF-20); после запуска СУБД работа восстановилась без перезапуска

Важно разделять два свойства, проверенных тестами INT-44 и INT-45. Сохранность данных при перезапуске обеспечена: все заказы, образы и артефакты хранятся в MySQL и файловой системе. Восстановление очереди не обеспечено: очередь существует только в памяти процесса, поэтому после аварийного завершения образ, находившийся в сборке, навсегда остаётся в статусе BUILDING, а ожидавшие заказы – в QUEUED, и их нельзя ни изменить, ни (для BUILDING) удалить. Для исправления предложено при запуске приложения переводить образы из BUILDING в FAILED и повторно ставить в очередь образы в статусе QUEUED либо хранить очередь в базе данных.

Проверка артефакта в режиме симуляции подтверждает корректность конвейера (передача состава, формирование файла, сохранение пути, выдача), но не является доказательством загрузки операционной системы на устройстве: для этого требуется реальная сборка SDK и прошивка физического Luckfox Pico.

4.6 Нагрузочное тестирование и анализ производительности

Нагрузочное тестирование выполнено инструментом Grafana k6 [16]. Сценарии описаны на JavaScript (tests/load/scenarios.js); использовались модель закрытой нагрузки (виртуальные пользователи с паузами «на чтение») и модель открытой нагрузки (заданная интенсивность запросов). Операции размечены тегами, что позволило получать метрики отдельно для каждого эндпоинта и для чтения и записи. Разработано 12 профилей (таблица 20).

Таблица 20 – Профили нагрузочного тестирования

ID	**Профиль**	**Нагрузка**	**Результат до оптимизации**
L-01	Дымовой	1 пользователь, 1 мин, все пользовательские пути	0 ошибок, p95 111 мс
L-02	Просмотр каталогов	до 50 гостей	p95 15 мс
L-03	Смешанный (целевой)	100 пользователей: 70 гостей, 20 пользователей, 8 заказчиков, 2 администратора	0 ошибок из 22 489, p95 17,5 мс
L-04	Стресс	ступени 50–600 итераций/с	до 755 запросов/с без деградации
L-04b	Поиск предела	ступени 1 000–4 000 итераций/с	предел ≈1 300–1 470 запросов/с
L-05	Всплеск	10 → 400 пользователей за 10 с	без ошибок и роста p95
L-06	Шторм входов	10–100 входов/с	на 100/с p95 1 050 мс
L-07	Пакет заказов	60 одновременных заказов	все 60 собраны, последний ждал 189 с
L-08	Скачивание	20 потоков, 2 мин	361 МБ/с, p95 104 мс
L-09	Длительный	40 пользователей, 15 мин	без утечек и роста задержки
L-10a/b	Объёмный	40 запросов/с на обычной БД и на БД ×100	p95 9 → 28 мс

Во всех профилях (около 879 тыс. запросов) не зафиксировано ни одной HTTP-ошибки. Целевой профиль L-03 выполнен с многократным запасом: p95 чтения 17,5 мс, записи менее 60 мс при загрузке машины около 5 %. Длительный профиль L-09 (23 230 запросов, 173 сборки) не выявил утечек памяти (heap стабильно около 200 МБ) и дрейфа задержки.

Для подсистемы сборки наиболее показательны профили L-07 и L-08. В L-07 60 пользователей одновременно заказали сборку: все заказы приняты (p95

ответа на заказ 277 мс), все 60 образов собраны без потерь и сбоев, однако последний заказ ожидал 189 с при длительности одной сборки около 3,2 с – очередь строго последовательна. В смешанном профиле среднее время от заказа до READY составило 12,5 с, то есть ожидание в очереди в 3–4 раза дольше самой сборки. При реальной сборке SDK длительностью 1–3 ч один воркер обеспечит лишь 8–24 образа в сутки, а очередь из 60 заказов будет обрабатываться 2,5–7,5 суток; пользователь при этом не видит свою позицию в очереди (DEF-24). В L-08 отдача файлов потоком обеспечила 361 МБ/с при 20 одновременных скачиваниях без роста потребления памяти.

Узким местом чтения оказалась база данных. В профиле L-04b пропускная способность перестала расти на уровне 1 300–1 470 запросов/с; при большей интенсивности система не отказывала, а накапливала запросы в очередях Tomcat и пула соединений, и медиана задержки выросла до 2–3 с. Мониторинг показал около 12,5 SQL-запроса на один HTTP-запрос (проблема N+1 при загрузке составов образов и зависимостей) и исчерпание пула из 10 соединений. В профиле входа L-06 при 100 входах в секунду медиана выросла в 11 раз (с 80 до 912 мс). Объёмный тест L-10b на базе, увеличенной в 100 раз (20 388 образов, 2 071 компонент), показал рост p95 в 3 раза, а страница из 100 образов порождала 389 SQL-запросов.

4.7 Анализ выявленных дефектов и оптимизация системы

По результатам тестирования составлен реестр из 22 дефектов (полный реестр – в приложении Ж): 1 критический, 4 высокой, 9 средней и 8 низкой серьёзности. Критический дефект DEF-07 – ключ подписи JWT, записанный в исходном коде и хранящийся в репозитории: зная его, можно выпустить токен администратора. Дефекты высокой серьёзности: отсутствие проверки владельца при скачивании образа (DEF-16), потеря очереди сборок после аварийного перезапуска (DEF-19), необработанные ошибки разбора JWT (DEF-05) и

отсутствие ограничения попыток входа (DEF-21). К подсистеме сборки относятся DEF-15, DEF-16, DEF-17, DEF-19 и DEF-24.

По итогам нагрузочного тестирования выполнена оптимизация, затрагивающая производительность и устойчивость (таблица 21).

Таблица 21 – Изменения, выполненные по итогам нагрузочного тестирования

Изменение	**Дефекты**
Пакетная догрузка ленивых связей (hibernate.default_batch_fetch_size = 100): составы образов, зависимости и типы компонентов загружаются одним запросом IN на страницу	DEF-12, DEF-22
Отключение Open Session In View: соединение с БД не удерживается до конца HTTP-запроса, в том числе на время проверки пароля BCrypt	DEF-21, DEF-22
Загрузка пользователя вместе с ролями одним запросом (@EntityGraph) в фильтре JWT	DEF-22
Соединения пула без autocommit: исключены команды SET autocommit вокруг каждой транзакции	DEF-22
Пул HikariCP увеличен с 10 до 20 соединений; таймаут получения соединения – 3 с с ответом 503 и заголовком Retry-After	DEF-20, DEF-22
Индексы Image(state, cost), Image(cost), Component(name, id), Component(type, cost)	DEF-23
Ограничение неудачных входов: 5 на адрес и 20 на IP за 5 мин, сверх лимита – ответ 429 без проверки BCrypt	DEF-21
84 модульных теста на сервисы, DTO, валидацию и обработку ошибок	–

После изменений те же профили нагрузки повторены на том же стенде и на тех же данных. Сравнение ключевых метрик приведено в таблице 22, изменение пропускной способности и задержки на ступенях профиля L-04b – на рисунке 6, время входа по ступеням профиля L-06 – на рисунке 7.

Таблица 22 – Ключевые метрики до и после оптимизации

Метрика	**До**	**После**	**Изменение**
Предел пропускной способности чтения (L-04b), запросов/с	1 468	3 830	×2,6
Медиана задержки на ступени 2 000 итераций/с (L-04b), мс	2 692	3,4	в 790 раз
Пропущенные итерации k6 (L-04b), тыс.	526	81	−85 %
p95 стресса до 780 запросов/с (L-04), мс	15,0	6,9	−54 %
SQL-запросов в секунду в L-04	4 291	1 970	−54 %
CPU mysqld / backend в L-04, %	97 / 80	56 / 56	−42 % / −30 %
p95 целевого профиля (L-03), мс	17,5	5,6	−68 %
p95 входа на 100 входах/с (L-06), мс	1 050	127	−88 %
SQL-запросов на страницу из 100 образов (БД ×100)	389	11	−97 %

Метрика	До	После	Изменение
p95 страницы из 100 образов (БД ×100), мс	102,8	11,8	−89 %
p95 объёмного профиля (L-10b), мс	27,7	29,8	без улучшения

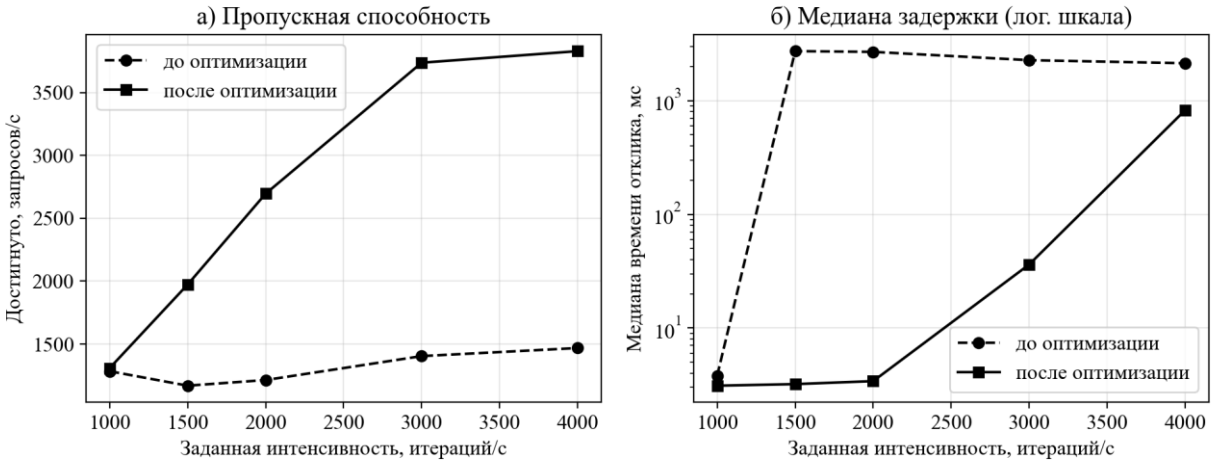


Рисунок 6 – Профиль L-04b до и после оптимизации: а – достигнутая пропускная способность, б – медиана времени отклика

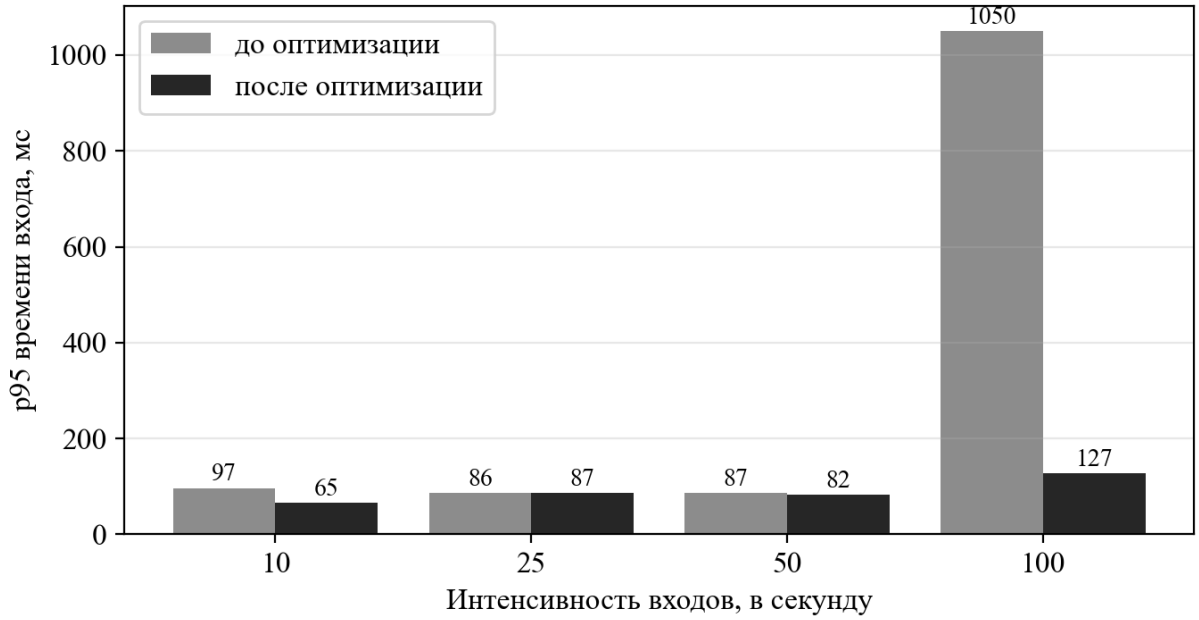


Рисунок 7 – p95 времени входа по ступеням профиля L-06 до и после оптимизации

Главный эффект дало устранение проблемы N+1 и удержания соединения: число SQL-запросов на страницу больше не зависит от её размера (9–11 запросов на любую страницу), на одинаковой нагрузке ресурсов требуется вдвое меньше,

а новый предел пропускной способности определяется процессором машины, а не пулом соединений. Причиной «колена» при входе оказалось не само хеширование BCrypt, а режим Open Session In View: соединение с базой данных удерживалось всё время проверки пароля, и 100 входов в секунду исчерпывали пул из 10 соединений.

Не улучшился p95 объёмного профиля под нагрузкой (27,7 → 29,8 мс, в пределах разброса), хотя число SQL-запросов снизилось на 66 %. На 20 тыс. образов время ответа определяют поиск по подстроке LIKE '%…%' и подсчёт общего числа строк для пагинации – полный проход таблицы, который обычными индексами не ускоряется; поэтому DEF-23 исправлен частично, а дальнейшее улучшение требует полнотекстового индекса и пагинации по ключу.

Регрессий не выявлено. Повторный интеграционный прогон на чистой базе данных дал 30 успешных тестов из 48: результаты INT-23 (число SQL-запросов на страницу) и INT-46 (недоступность БД: теперь 503 за 3,0 с) изменились с FAIL на PASS, новый тест INT-48 (защита от перебора паролей) пройден, остальные результаты совпали с первым прогоном. Ошибок ленивой инициализации после отключения Open Session In View не возникло, поскольку все DTO формируются внутри транзакций сервисов. Итог: исправлены DEF-12, DEF-20, DEF-21 и DEF-22, частично – DEF-23.

Не исправлялись дефекты безопасности и обработки ошибок (DEF-03, DEF-04, DEF-05, DEF-07, DEF-16) и архитектурные ограничения очереди сборок (DEF-19, DEF-24): они выходят за рамки оптимизации производительности. Для них в реестре зафиксированы рекомендации: чтение ключа JWT из переменной окружения, точка входа аутентификации с ответом 401, проверка владельца образа при скачивании, восстановление очереди при запуске, лимит активных заказов на пользователя с отображением позиции в очереди и несколько воркеров с изолированными копиями SDK.

4.8 Оценка результатов и ограничения тестирования

Проведённое тестирование подтвердило работоспособность основных сценариев системы и корректность подсистемы сборки: разрешение зависимостей, защита от циклов, последовательная очередь без потерь заказов, формирование валидного артефакта и его выдача. Целевая нагрузка выполняется с многократным запасом, а после оптимизации предел пропускной способности чтения вырос в 2,6 раза.

При интерпретации результатов необходимо учитывать ограничения:

– сборка образов в нагрузочных и интеграционных тестах выполнялась в режиме симуляции (около 3 с) вместо реальной сборки SDK длительностью 1–3 ч, поэтому оценки очереди для реального режима получены экстраполяцией;

– генератор нагрузки, серверная часть и СУБД работали на одном компьютере и делили процессор, поэтому абсолютные пределы занижены относительно выделенного сервера (в профиле L-04b k6 достиг предела в 3 000 виртуальных пользователей);

– сеть – петлевой интерфейс без сетевых задержек и ограничений пропускной способности;

– MySQL работал с параметрами по умолчанию;

– проверка ext4-артефакта подтверждает корректность конвейера, но не загрузку операционной системы на физическом Luckfox Pico, которая не проверялась;

– автоматизированные тесты клиентской части ограничены сквозными сценариями; модульных тестов компонентов React нет.

5 ОРГАНИЗАЦИЯ РАБОТЫ НАД ПРОЕКТОМ

5.1 Распределение задач и зон ответственности

Разработка прототипа системы автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico выполнялась командой из двух человек: Поздняковым Н.Э. и Ждановым А.И. Распределение работ строилось по функциональным подсистемам, что позволило вести параллельную разработку серверной и клиентской частей с минимальным числом пересечений в коде.

За Поздняковым Н.Э. закреплены подсистемы конфигурирования и сборки образов: серверная обработка состава образа, разрешение транзитивных зависимостей компонентов и проверка графа зависимостей на циклы, расчёт стоимости образа, очередь сборок (BuildService, AsyncConfig, BuildStateService) с единственным фоновым воркером, интеграция с Luckfox SDK, сценарии luckfox_build.sh и real-build.sh, режим симуляции сборки, хранение и выдача артефактов, а также программный интерфейс заказа и скачивания образов. Помимо этого, в его зону ответственности вошли разработка проверок и тестирование: модульные, интеграционные, сквозные (E2E) и нагрузочные тесты, а также оптимизация обращений к базе данных по результатам измерений.

За Ждановым А.И. закреплены пользовательский интерфейс на React, проектирование и наполнение базы данных, управление пользователями и ролями, аутентификация на основе JWT, каталоги компонентов и образов, административные функции (редактирование компонентов и образов, управление пользователями) и общая инфраструктура проекта (конфигурация Spring Boot, docker-compose, сборка frontend).

Общие для обеих частей решения принимались и фиксировались совместно. К ним относятся формат запроса на сборку BuildRequest и представление образа ImageDto, набор статусов образа (QUEUED, BUILDING, READY, FAILED), состав REST-интерфейса, права доступа ролей USER и ADMIN, а также структура таблиц базы данных. Согласование общих

контрактов до начала реализации исключало расхождения между серверной частью и интерфейсом. Итоговое распределение приведено в таблице 23.

Таблица 23 – Распределение зон ответственности

Направление	**Поздняков Н.Э.**	**Жданов А.И.**	**Совместно**
Конфигурирование образа	Разрешение зависимостей, проверка циклов, расчёт стоимости	Редактор образа и форма выбора компонентов в интерфейсе	Формат BuildRequest и ImageDto
Сборка образов	Очередь сборок, скрипты luckfox_build.sh и real-build.sh, режим симуляции, интеграция с SDK	Отображение статусов и автообновление библиотеки образов	Набор статусов образа
Артефакты и заказ	API заказа и скачивания, хранение и удаление .img-файлов	Кнопка скачивания и обработка ошибок в интерфейсе	Права доступа к операциям
Пользователи и безопасность	Ограничение доступа к API заказа, ограничение попыток входа (по итогам нагрузочного тестирования)	Роли, JWT-аутентификация, управление пользователями	Модель ролей USER и ADMIN
База данных	Проверка целостности и оптимизация запросов по результатам тестов	Проектирование схемы, сущности и репозитории	Схема таблиц образов и компонентов
Каталоги и администрирование	Проверка правил удаления компонентов и образов	Библиотеки, поиск, фильтры, пагинация, редакторы для администратора	Правила удаления связанных записей
Тестирование	Модульные, интеграционные, E2E- и нагрузочные тесты, отчёт	Проверка интерфейса в ходе разработки	Анализ найденных дефектов
Документация	Разделы о сборке, тестировании и оптимизации	Разделы об интерфейсе, базе данных и ролях	Введение, анализ предметной области, заключение

5.2 Организация совместной разработки и контроль версий

Для контроля версий использовалась система Git. Общий репозиторий создан на платформе GitHub средствами GitHub Classroom, что обеспечивает преподавателю доступ к истории изменений. Разработка велась в основной ветке main, изменения передавались в общий репозиторий небольшими законченными порциями.

Структура каталогов репозитория отражает разделение подсистем. Каталог backend содержит серверное приложение на Spring Boot: контроллеры, сервисы, репозитории, сущности и объекты передачи данных. Каталог frontend содержит клиентское приложение на React и Vite: страницы, компоненты, обращения к API, контекст аутентификации и хранилище состояния. В каталоге scripts размещены вспомогательные сценарии сборки, включая luckfox_build.sh. Каталог DOCs включает материалы по этапам проекта, диаграммы, описание базы данных, макеты интерфейса, техническую документацию и результаты тестирования. Каталог tests содержит сценарии интеграционных и нагрузочных испытаний и средства формирования отчёта. В корне репозитория расположены файл docker-compose.yml и краткое описание проекта README.md.

Не все файлы, необходимые для работы системы, хранятся в Git. Правила исключения заданы в файле .gitignore. В репозиторий не включаются:

– внешний SDK luckfox-pico (около 12 ГБ), который имеет собственную историю версий и загружается отдельно из репозитория производителя;

– файлы .env с настройками окружения и учётными данными;

– результаты сборки и служебные каталоги (target, node_modules, dist, output);

– сгенерированные образы с расширением .img.

Такой подход сохраняет репозиторий компактным, исключает попадание секретов в общий доступ и не допускает подмену исходных текстов производными файлами.

Сообщения коммитов формулировались кратко и описательно, в повелительном наклонении, например «Add component filter and price sorting to libraries» или «Add unit, integration and load tests; optimize DB access». Первые коммиты этапов проектирования, оформлявшиеся на русском языке, содержат названия этапов («Второй этап + диограммы», «3-4 этапы»). Каждый коммит

объединяет изменения одной логической задачи, что упрощает просмотр истории и поиск причин дефектов.

5.3 Календарный план выполнения работ

Работа над проектом была разбита на этапы в соответствии с ходом курса. Фактические сроки определены по истории коммитов репозитория и датам материалов этапов. Календарный план приведён в таблице 24.

Таблица 24 – Календарный план

Этап	**Содержание работ**	**Сроки**	**Результат**
1. Концепция и прототип	Формулировка идеи, обоснование актуальности, эскиз прототипа системы	Ноябрь 2025	Презентация этапа 1 (каталог DOCs/этап1)
2. Анализ и проектирование	Анализ предметной области, диаграмма прецедентов, модели BPMN, диаграммы классов и состояний	Декабрь 2025	Материалы этапа 2, диаграммы
3–4. Интерфейс и данные	Макеты интерфейса, схема навигации, проектирование схемы базы данных	Декабрь 2025	Макеты интерфейса, схема БД, описание базы данных
Реализация	Backend с аутентификацией, очередь сборок и разрешение зависимостей, тулчейн Luckfox, новый frontend, docker-compose	Июнь 2026	Работающий прототип системы
Доработки	Исправление найденных замечаний, пагинация, фильтры и сортировка, администрирование и управление пользователями	Сентябрь 2026	Расширенная версия прототипа
Тестирование и оптимизация	84 модульных теста, интеграционные и нагрузочные испытания, оптимизация доступа к БД, отчёт о тестировании	Сентябрь 2026	Отчёт тестирования, наборы тестов
Оформление ПЗ	Подготовка пояснительной записки и приложений	Сентябрь 2026	Пояснительная записка

Как следует из таблицы, этапы анализа и проектирования были выполнены в срок, в ноябре и декабре 2025 года. Основная реализация системы была зафиксирована в репозитории только в июне 2026 года, а доработки, тестирование и оформление пояснительной записки перенесены на сентябрь 2026 года. Таким образом, фактические сроки реализации сместились относительно первоначального плана; на объём и содержание итоговых

результатов это не повлияло, однако сжало время, отводимое на тестирование и оптимизацию.

5.4 Использование инструментов искусственного интеллекта

При выполнении работы применялся инструмент Claude Code – агент на основе больших языковых моделей Anthropic, работающий в среде командной строки с доступом к файлам проекта. Инструмент применялся в следующих сценариях:

– анализ кодовой базы: получение обзора структуры проекта и связей между слоями приложения (контроллер, сервис, репозиторий) перед внесением изменений;

– поиск причин ошибок: разбор сообщений об ошибках и трассировок, выдвижение гипотез о причине дефекта и указание участков кода, требующих проверки;

– подготовка заготовок модульных тестов и сценариев нагрузочного тестирования на k6, которые затем дорабатывались и уточнялись вручную;

– автоматизация прогонов на стенде: запуск наборов тестов и нагрузочных сценариев, сбор и агрегирование результатов измерений;

– подготовка черновиков документации: технической документации проекта и отчёта о тестировании.

Все результаты, полученные с помощью инструмента, проверялись вручную. Корректность кода подтверждалась запуском тестов (команда mvn test, 84 теста из 84 пройдены), а результаты измерений – повторными прогонами сценариев. Предложенные исправления принимались только после просмотра изменений и подтверждения их работоспособности.

Использование инструмента имело ограничения. Проектные решения (состав подсистем, архитектура, структура данных, правила предметной области) принимались разработчиками; инструмент использовался как вспомогательное средство. Предлагаемые инструментом решения и тексты

могли содержать ошибки и неточности: например, неверные предположения о поведении кода или несоответствие описания фактической реализации, поэтому каждое утверждение документации сопоставлялось с исходным кодом. Приёмка результатов без верификации не допускалась; ответственность за содержание работы, включая код, тесты и текст пояснительной записки, несут авторы проекта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработан прототип распределённой программной системы Luckfox Image Builder для автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico. Система позволяет выбрать готовый образ из библиотеки или сконфигурировать собственный из каталога компонентов, заказать удалённую сборку и получить готовый файл образа без развёртывания сборочной среды на компьютере пользователя.

В рамках личного вклада автора спроектированы и реализованы пользовательский интерфейс, модель данных и подсистема управления пользователями:

– разработана информационная модель и реализована база данных MySQL на основе JPA-сущностей с отношениями «многие ко многим», самоссылочным графом зависимостей компонентов, ограничениями целостности и оптимистичной блокировкой;

– реализованы регистрация и вход с хранением паролей в виде хешей BCrypt, stateless-аутентификация по токенам JWT и ролевое разграничение доступа USER/ADMIN с защитой администратора от самоблокировки;

– реализованы каталоги образов и компонентов с поиском, фильтрами, сортировкой и постраничным выводом, а также административные функции управления компонентами, образами и пользователями;

– разработано одностраничное клиентское приложение на React с маршрутизацией по ролям, редакторами образов и компонентов, личным кабинетом с автоматическим обновлением статусов сборки;

– определены контракты взаимодействия клиентской и серверной частей (DTO, формат ошибок, постраничные ответы) и обеспечено развёртывание системы средствами Docker Compose.

Работоспособность системы подтверждена комплексом проверок: 84 модульных теста, 48 интеграционных тестов API, 9 сквозных сценариев в браузере и 12 профилей нагрузочного тестирования. Целевая нагрузка в 100 одновременных пользователей выполняется с многократным запасом; оптимизация доступа к базе данных по итогам тестирования (пакетная загрузка связей, индексы, настройка пула соединений, ограничение попыток входа) увеличила пропускную способность чтения каталогов в 2,6 раза.

Тестирование выявило и ряд недостатков, относящихся к подсистемам автора: ключ подписи JWT хранится в исходном коде, при ошибках аутентификации сервер отвечает кодом 403 вместо 401, из-за чего клиент не перенаправляет пользователя на страницу входа, разрешённый источник CORS не вынесен в конфигурацию. Для каждого недостатка в реестре дефектов зафиксированы рекомендации по исправлению.

Дальнейшее развитие предполагает вынесение секретов и CORS в переменные окружения, единую обработку ошибок безопасности с ответами 401, отображение позиции заказа в очереди, полнотекстовый поиск по каталогу, а также реализацию оплаты и уведомлений по электронной почте, рассматривавшихся на этапе анализа.

В процессе работы закреплены знания в области проектирования распределённых программных систем, моделирования на языке UML, проектирования реляционных баз данных, разработки веб-приложений на Spring Boot и React, обеспечения безопасности веб-приложений и тестирования программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017. – 32 с.

2. Фаулер М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования. – 3-е изд. – СПб.: Символ-Плюс, 2004. – 192 с.

3. Буч Г., Рамбо Дж., Якобсон И. Язык UML. Руководство пользователя. – 2-е изд. – М.: ДМК Пресс, 2006. – 496 с.

4. Luckfox Pico Wiki [Электронный ресурс]. – Режим доступа: https://wiki.luckfox.com/Luckfox-Pico/Luckfox-Pico-quick-start (дата обращения: 20.09.2026).

5. LuckfoxTECH/luckfox-pico: Luckfox Pico SDK [Электронный ресурс]. – Режим доступа: https://github.com/LuckfoxTECH/luckfox-pico (дата обращения: 20.09.2026).

6. The Buildroot user manual [Электронный ресурс]. – Режим доступа: https://buildroot.org/downloads/manual/manual.html (дата обращения: 20.09.2026).

7. Yocto Project Documentation [Электронный ресурс]. – Режим доступа: https://docs.yoctoproject.org/ (дата обращения: 21.09.2026).

8. Spring Boot Reference Documentation [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-boot/ (дата обращения: 22.09.2026).

9. Spring Framework Reference: Task Execution and Scheduling [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-framework/reference/integration/scheduling.html (дата обращения: 22.09.2026).

10. Hibernate ORM User Guide [Электронный ресурс]. – Режим доступа: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate_User_Guide.html (дата обращения: 28.09.2026).

11. MySQL 8.0 Reference Manual [Электронный ресурс]. – Режим доступа: https://dev.mysql.com/doc/refman/8.0/en/ (дата обращения: 28.09.2026).

12. React Documentation [Электронный ресурс]. – Режим доступа: https://react.dev/ (дата обращения: 22.09.2026).

13. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.

14. JUnit 5 User Guide [Электронный ресурс]. – Режим доступа: https://junit.org/junit5/docs/current/user-guide/ (дата обращения: 29.09.2026).

15. Mockito Documentation [Электронный ресурс]. – Режим доступа: https://site.mockito.org/ (дата обращения: 29.09.2026).

16. Grafana k6 Documentation [Электронный ресурс]. – Режим доступа: https://grafana.com/docs/k6/latest/ (дата обращения: 29.09.2026).

17. Playwright Documentation [Электронный ресурс]. – Режим доступа: https://playwright.dev/docs/intro (дата обращения: 29.09.2026).

18. Docker Compose Documentation [Электронный ресурс]. – Режим доступа: https://docs.docker.com/compose/ (дата обращения: 25.09.2026).

19. Armbian Documentation: Build Framework [Электронный ресурс]. – Режим доступа: https://docs.armbian.com/Developer-Guide_Build-Preparation/ (дата обращения: 21.09.2026).

20. Raspberry Pi Imager [Электронный ресурс]. – Режим доступа: https://www.raspberrypi.com/software/ (дата обращения: 21.09.2026).

21. OpenWrt Firmware Selector [Электронный ресурс]. – Режим доступа: https://firmware-selector.openwrt.org/ (дата обращения: 21.09.2026).

22. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2018. – 352 с.

23. Майерс Г., Баджетт Т., Сандлер К. Искусство тестирования программ. – 3-е изд. – М.: Диалектика, 2012. – 272 с.

ПРИЛОЖЕНИЕ А

РАСШИРЕННЫЕ ОПИСАНИЯ ПРЕЦЕДЕНТОВ

Прецедент: «Заказать сборку образа»

Действующее лицо: пользователь (роль USER или ADMIN);

Предусловие: пользователь авторизован в системе, в каталоге существует хотя бы один компонент;

Основной поток: «Заказать сборку образа». 1. Пользователь выбирает пункт «Создать образ» в главном меню; 2. Система отображает редактор образа с полями «Название», «Базовый дистрибутив», «Плата», «Описание» и каталогом компонентов с поиском и фильтром по типу; 3. Пользователь вводит название образа, выбирает дистрибутив (buildroot, ubuntu или busybox) и плату и при необходимости вводит описание; 4. Пользователь добавляет нужные компоненты из каталога кнопкой «+», система отображает выбранные компоненты в списке «Зависимости» и суммарную стоимость в условных единицах; 5. Пользователь нажимает кнопку «Заказать образ»; 6. Система проверяет данные, находит выбранные компоненты и добавляет к ним все транзитивные зависимости, проверяет граф зависимостей на циклы, рассчитывает стоимость как сумму стоимостей всех компонентов состава, сохраняет образ в статусе «В очереди», привязывает его к пользователю и ставит сборку в очередь; 7. Система возвращает ответ о принятии заказа, редактор очищается, и пользователь перенаправляется в библиотеку образов;

Альтернативный поток 1: «Не заполнены обязательные данные». На шаге 5 название образа не указано или не выбран ни один компонент. Система не отправляет запрос и отображает сообщение «Укажите название образа» или «Добавьте хотя бы один компонент», после чего пользователь возвращается к шагу 3 основного потока;

Альтернативный поток 2: «Компонент не найден или обнаружена циклическая зависимость». На шаге 6 выбранный компонент отсутствует в

каталоге либо в графе зависимостей обнаружен цикл. Система отклоняет заказ и отображает сообщение об ошибке, например «Обнаружена циклическая зависимость с участием компонента '…'», после чего пользователь возвращается к шагу 4 основного потока;

Альтернативный поток 3: «Сборка завершилась ошибкой». После шага 7 фоновая сборка прервана по ошибке или превышено время ожидания. Система переводит образ в статус «Ошибка», сохраняет журнал сборки, файл образа не создаётся и кнопка «Скачать .img» не отображается;

Постусловие: образ сохранён в библиотеке образов и привязан к заказавшему его пользователю; сборка поставлена в очередь и выполняется в фоновом режиме, при этом одновременно собирается только один образ.

Прецедент: «Отследить статус и скачать образ»

Действующее лицо: пользователь (роль USER или ADMIN);

Предусловие: пользователь авторизован в системе, им заказан хотя бы один образ;

Основной поток: «Отследить статус и скачать образ». 1. Пользователь открывает пункт «Образы» главного меню; 2. Система отображает библиотеку образов в виде карточек с названием, статусом, дистрибутивом, платой, составом и стоимостью; 3. Система автоматически обновляет список каждые 5 секунд, статус образа последовательно меняется: «В очереди», «Сборка», «Готов»; 4. Пользователь при необходимости уточняет список с помощью поиска, фильтров по статусу, плате и компонентам и сортировки по цене; 5. Для образа в статусе «Готов» система отображает кнопку «Скачать .img»; 6. Пользователь нажимает кнопку «Скачать .img»; 7. Система проверяет статус образа и наличие файла артефакта и передаёт файл, который сохраняется на устройство пользователя под именем «название образа».img;

Альтернативный поток 1: «Образ не готов к загрузке». На шаге 6 образ не имеет статуса «Готов» либо файл артефакта отсутствует на сервере (например, был удалён). Система отклоняет запрос, и на странице отображается сообщение

«Загрузка недоступна»; при обращении к интерфейсу напрямую сервер возвращает сообщение «Образ ещё не готов к загрузке (статус: …)» или «Файл артефакта не найден на сервере»;

Альтернативный поток 2: «Сборка завершилась ошибкой». На шаге 3 образ получает статус «Ошибка». Кнопка «Скачать .img» не отображается; администратор может изменить состав образа и запустить сборку повторно;

Постусловие: образ в статусе «Готов» передан пользователю в виде файла .img; состояние образа и его артефакт в системе не изменены.

Прецедент: «Изменить образ с пересборкой»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, образ существует и не находится в статусе «В очереди» или «Сборка»;

Основной поток: «Изменить образ с пересборкой». 1. Администратор открывает библиотеку образов и нажимает кнопку «Изменить» на карточке нужного образа; 2. Система открывает редактор с заголовком «Редактирование образа #N» и подгружает параметры и состав образа, отображая пояснение, что после сохранения образ будет пересобран; 3. Администратор изменяет название, дистрибутив, плату, описание или состав компонентов; 4. Администратор нажимает кнопку «Сохранить и пересобрать»; 5. Система повторно проверяет данные, пересчитывает транзитивные зависимости и стоимость, переводит образ в статус «В очереди», сбрасывает прежний журнал сборки и удаляет ранее созданный файл образа; 6. Система ставит пересборку в очередь, редактор закрывается, и администратор перенаправляется в библиотеку образов, где отслеживает статус образа;

Альтернативный поток 1: «Образ находится в очереди или собирается». На шаге 4 образ имеет статус «В очереди» или «Сборка». Система отклоняет изменение и отображает сообщение «Образ сейчас в очереди или собирается — дождитесь окончания сборки»; кнопка «Изменить» на карточке образа в таких статусах недоступна (подсказка «Дождитесь окончания сборки»);

Альтернативный поток 2: «Некорректные данные». На шаге 4 не указано название образа или не выбран ни один компонент. Система отображает сообщение «Укажите название образа» или «Добавьте хотя бы один компонент» и остаётся в редакторе;

Альтернативный поток 3: «Отмена изменений». На шаге 3 или 4 администратор нажимает кнопку «Отмена». Система очищает редактор и возвращает администратора в библиотеку образов, образ остаётся без изменений;

Постусловие: параметры образа обновлены, предыдущий артефакт удалён, образ находится в очереди на пересборку и после её завершения получает статус «Готов» или «Ошибка».

Прецедент: «Добавить компонент»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, в системе существует хотя бы один тип компонента;

Основной поток: «Добавить компонент». 1. Администратор выбирает пункт «+ Компонент» главного меню; 2. Система отображает редактор компонентов с полями «Название», «Тип», «Стоимость», «Описание», блоком выбора зависимостей и списком существующих компонентов; 3. Администратор вводит название, выбирает тип, указывает стоимость (неотрицательное число) и описание; 4. Администратор при необходимости находит в блоке зависимостей нужные компоненты и отмечает их; 5. Администратор нажимает кнопку «Создать компонент»; 6. Система проверяет данные, находит тип и зависимости, сохраняет компонент и отображает сообщение «Компонент «…» создан», форма очищается, а список существующих компонентов обновляется;

Альтернативный поток 1: «Не заполнены обязательные поля». На шаге 5 не указано название компонента, тип или стоимость либо стоимость отрицательна. Система отображает сообщение о нарушенном требовании,

например «Название компонента обязательно» или «Стоимость не может быть отрицательной», и остаётся в форме;

Альтернативный поток 2: «Зависимость не найдена». На шаге 6 указанный тип или зависимость отсутствуют в базе данных (например, были удалены другим администратором). Система отображает сообщение «Тип компонента не найден: …» или «Зависимость не найдена: …», компонент не создаётся;

Постусловие: новый компонент сохранён в каталоге, отображается в библиотеке компонентов и доступен для выбора в редакторе образов.

Прецедент: «Удалить компонент»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, компонент существует в каталоге;

Основной поток: «Удалить компонент». 1. Администратор открывает библиотеку компонентов или редактор компонентов и нажимает кнопку «Удалить» (в редакторе – значок «✕») у нужного компонента; 2. Система запрашивает подтверждение «Удалить компонент «…»?»; 3. Администратор подтверждает действие; 4. Система проверяет, что компонент не входит ни в один образ и что от него не зависят другие компоненты; 5. Система удаляет компонент из каталога и обновляет список;

Альтернативный поток 1: «Компонент входит в образы». На шаге 4 компонент содержится в составе хотя бы одного образа. Удаление запрещено, система отображает сообщение «Компонент «…» входит в образы — сначала измените или удалите эти образы», компонент остаётся в каталоге;

Альтернативный поток 2: «От компонента зависят другие компоненты». На шаге 4 компонент указан как зависимость у другого компонента. Удаление запрещено, система отображает сообщение «От компонента «…» зависят другие компоненты — сначала уберите эту зависимость»;

Альтернативный поток 3: «Отказ от удаления». На шаге 3 администратор отказывается от подтверждения. Система отменяет действие, компонент остаётся без изменений;

Постусловие: компонент удалён из каталога и недоступен для выбора в образах либо, при запрете удаления, остался в каталоге без изменений.

Прецедент: «Зарегистрироваться и войти»

Действующее лицо: посетитель (неавторизованный пользователь);

Предусловие: посетитель не авторизован в системе и не имеет учётной записи;

Основной поток: «Зарегистрироваться и войти». 1. Посетитель нажимает кнопку «Регистрация» в шапке приложения; 2. Система отображает форму регистрации с полями «Имя», «Фамилия», «Email» и «Пароль» и подсказкой о требованиях к паролю (не менее 8 символов, цифра, строчная и заглавная буквы, спецсимвол); 3. Посетитель заполняет поля и нажимает кнопку «Зарегистрироваться»; 4. Система проверяет данные и уникальность адреса электронной почты, сохраняет пользователя с ролью USER, хранит пароль в виде хэша и выдаёт токен доступа; 5. Система выполняет вход, перенаправляет пользователя в библиотеку образов и отображает в меню пункты «Создать образ», «Аккаунт» и «Выйти». Для входа при последующих посещениях: 6. Пользователь нажимает кнопку «Вход», вводит email и пароль и нажимает кнопку «Войти»; 7. Система проверяет учётные данные, выдаёт токен доступа и перенаправляет пользователя на запрошенную ранее страницу или в библиотеку образов;

Альтернативный поток 1: «Данные не прошли проверку». На шаге 4 адрес электронной почты имеет неверный формат либо пароль не соответствует требованиям. Система отображает сообщения проверки (например, «Password must be at least 8 characters long» или «Invalid email format»), регистрация не выполняется, пользователь возвращается к шагу 3;

Альтернативный поток 2: «Адрес уже зарегистрирован». На шаге 4 пользователь с таким адресом уже существует. Система отображает сообщение «Пользователь с таким email уже существует»;

Альтернативный поток 3: «Неверные учётные данные или превышен лимит попыток». На шаге 7 данные не подходят, и попытка входа учитывается как неудачная. После превышения допустимого числа неудачных попыток (по умолчанию 5 для одного адреса почты и 20 для одного адреса клиента за 300 секунд) система отклоняет вход до проверки пароля и отображает сообщение «Слишком много неудачных попыток входа. Повторите через N с»;

Постусловие: учётная запись создана с ролью USER, пользователь авторизован, токен доступа сохранён в браузере, разделы, требующие авторизации, доступны; при выходе токен удаляется.

ПРИЛОЖЕНИЕ Б

ДИАГРАММЫ ПРЕЦЕДЕНТОВ И БИЗНЕС-ПРОЦЕССОВ

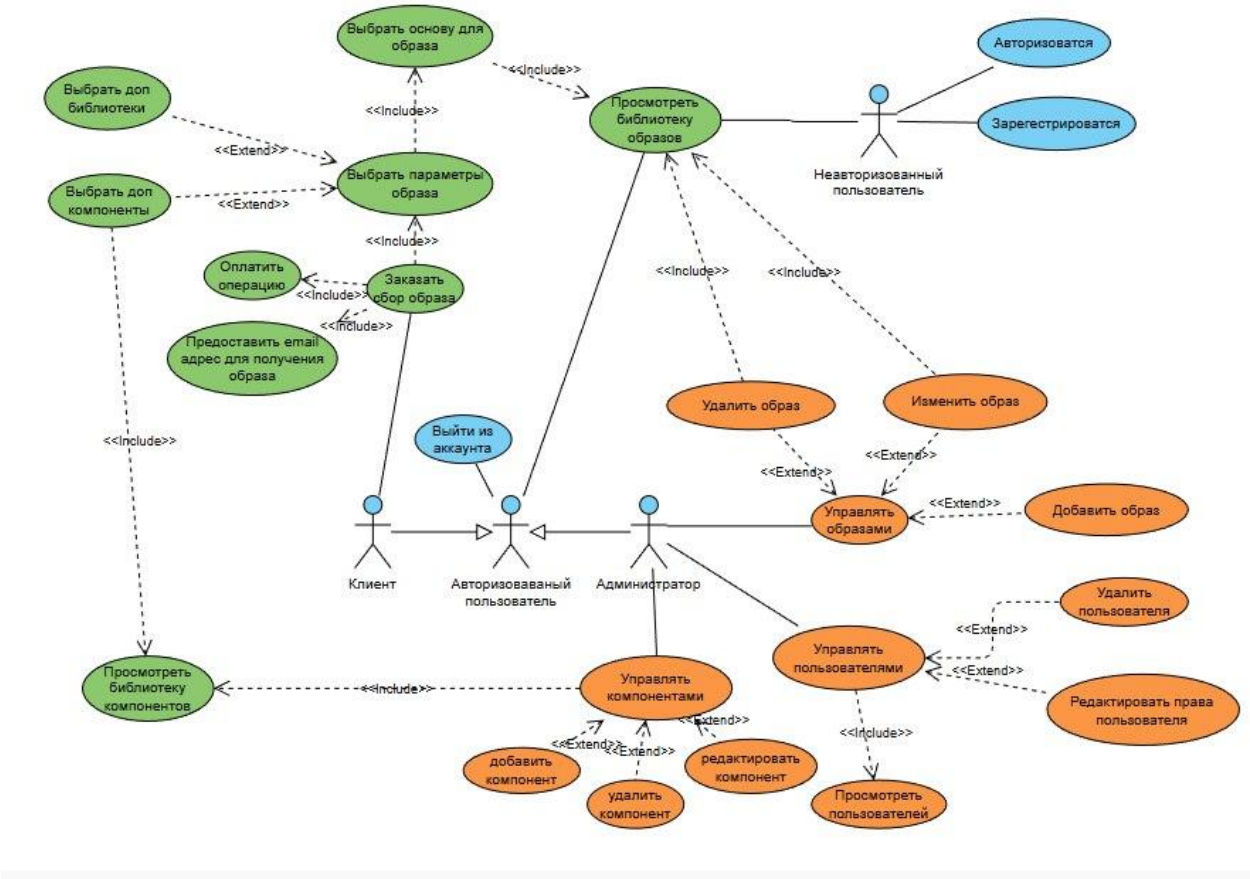


Рисунок Б.1 – Диаграмма прецедентов

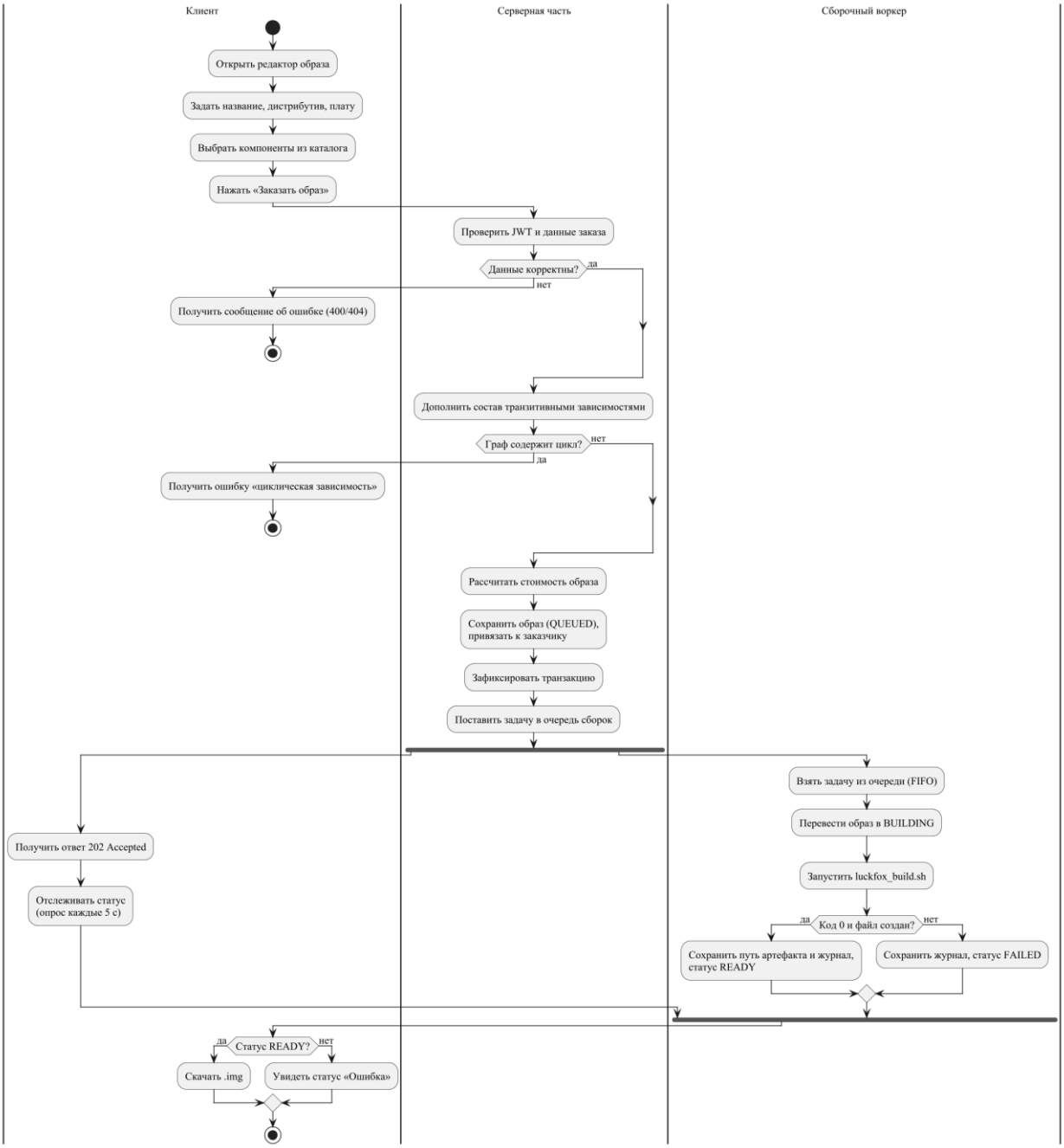


Рисунок Б.2 – Диаграмма деятельности бизнес-процесса «Заказ и сборка образа»

ПРИЛОЖЕНИЕ В

ДИАГРАММЫ КЛАССОВ И СОСТОЯНИЙ

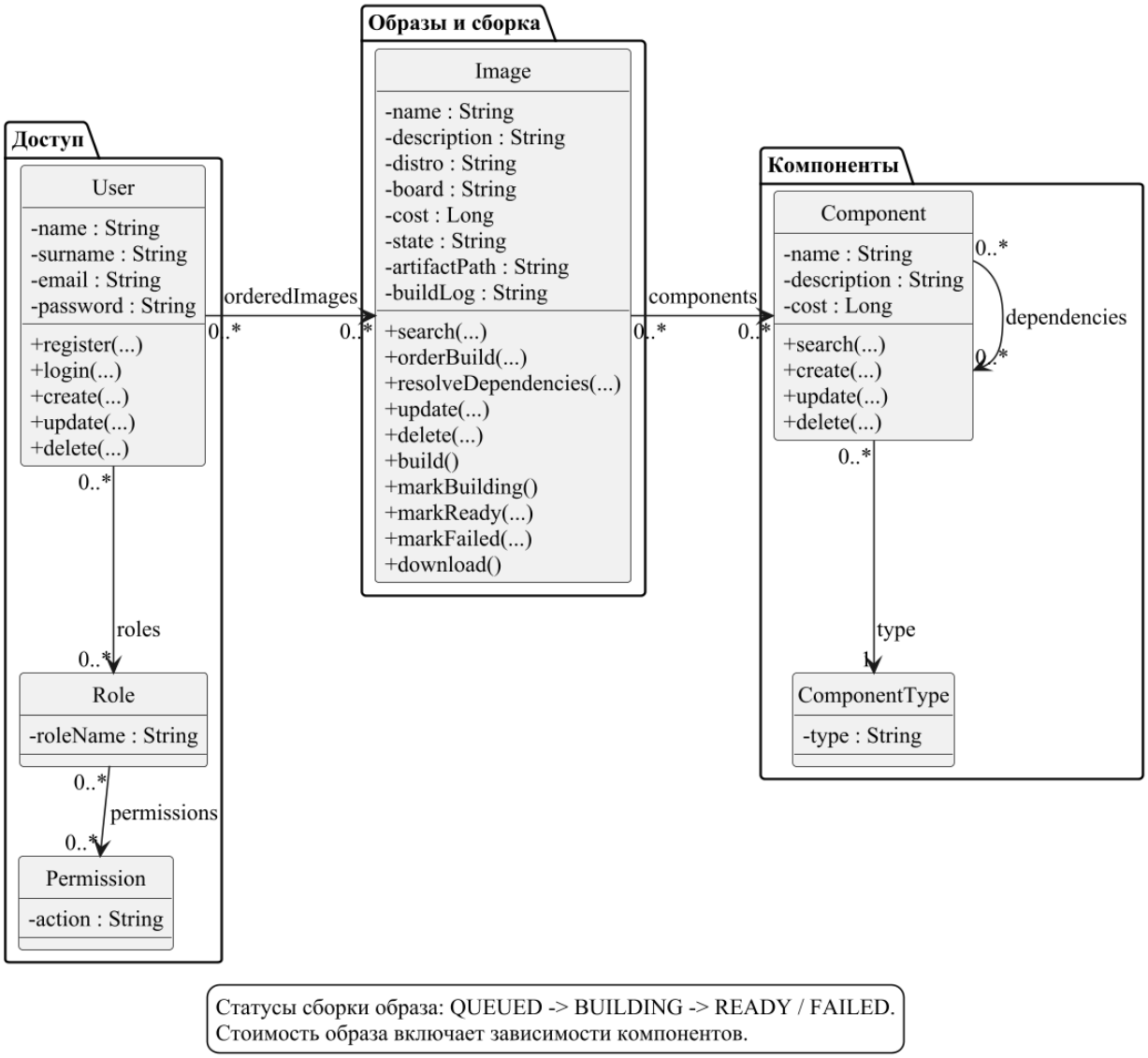


Рисунок В.1 – Диаграмма классов предметной области

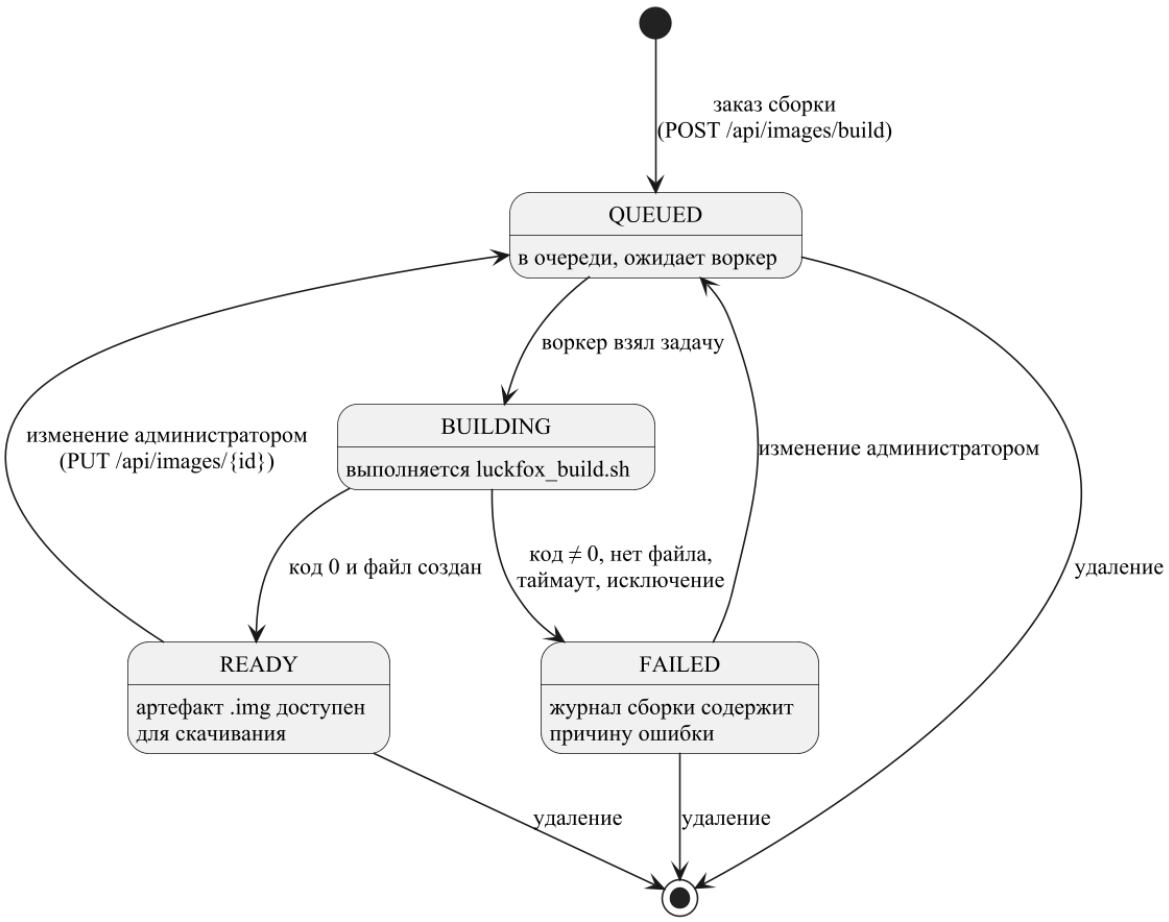


Рисунок В.2 – Диаграмма состояний сущности «Образ»

ПРИЛОЖЕНИЕ Г

СХЕМЫ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ И РАЗВЁРТЫВАНИЯ СИСТЕМЫ

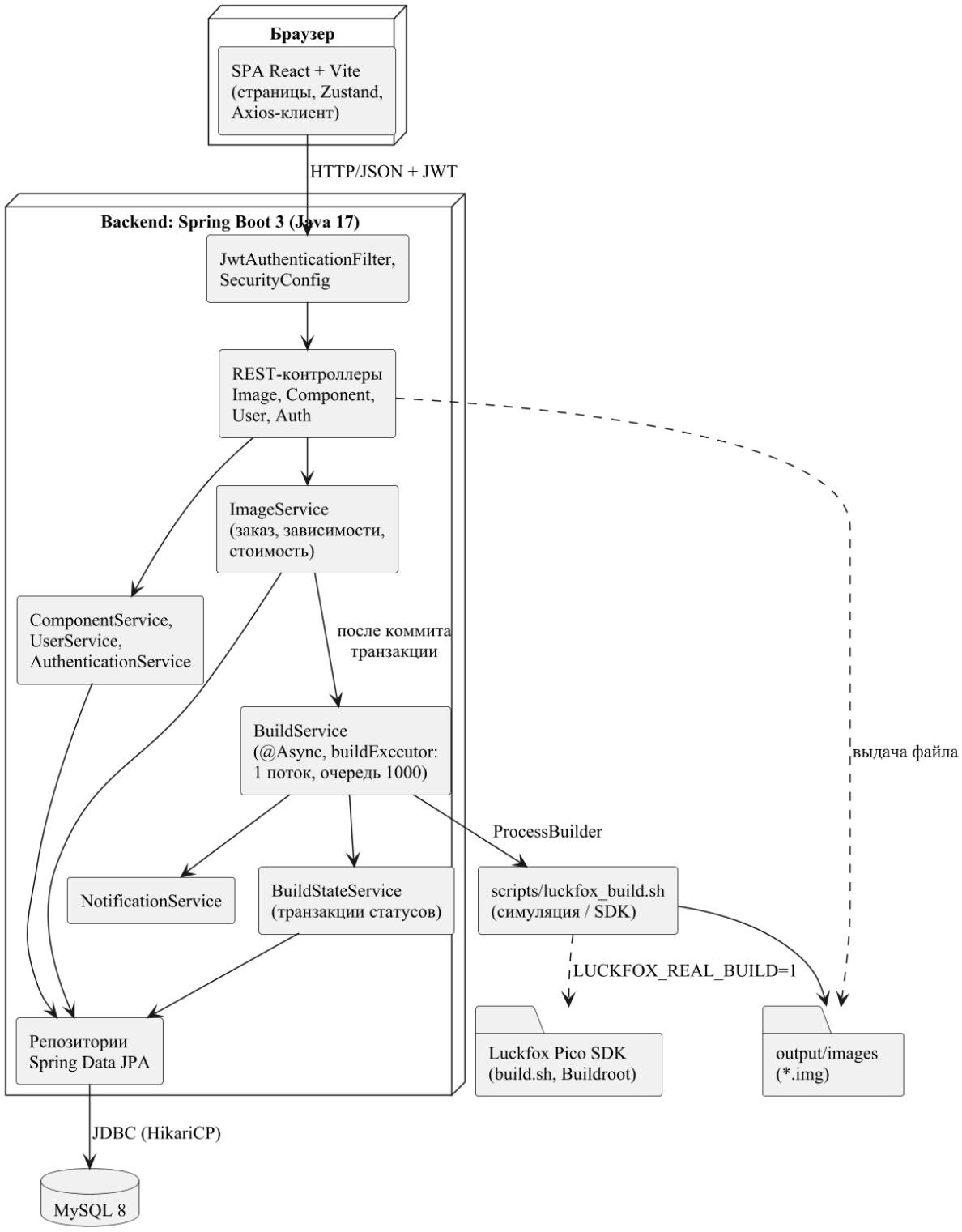


Рисунок Г.1 – Диаграмма компонентов системы

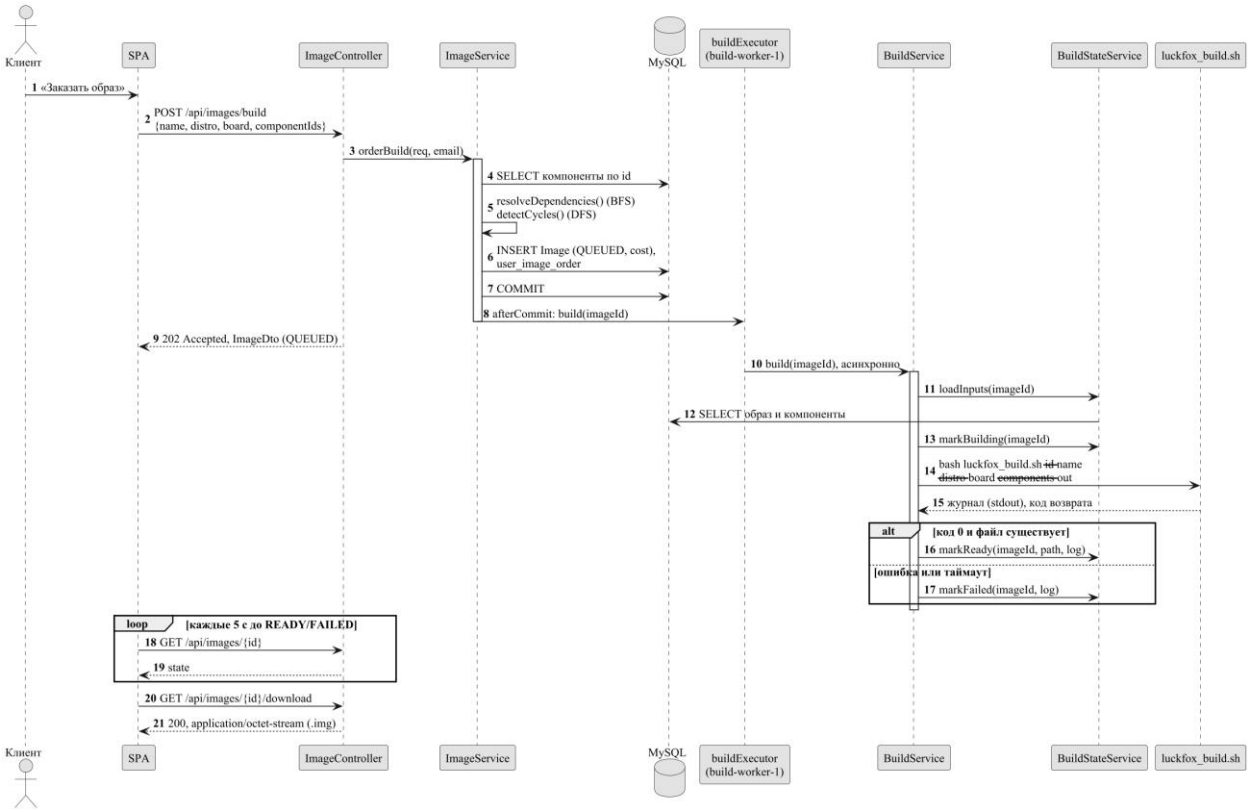


Рисунок Г.2 – Диаграмма последовательности «Заказ, сборка и скачивание образа»

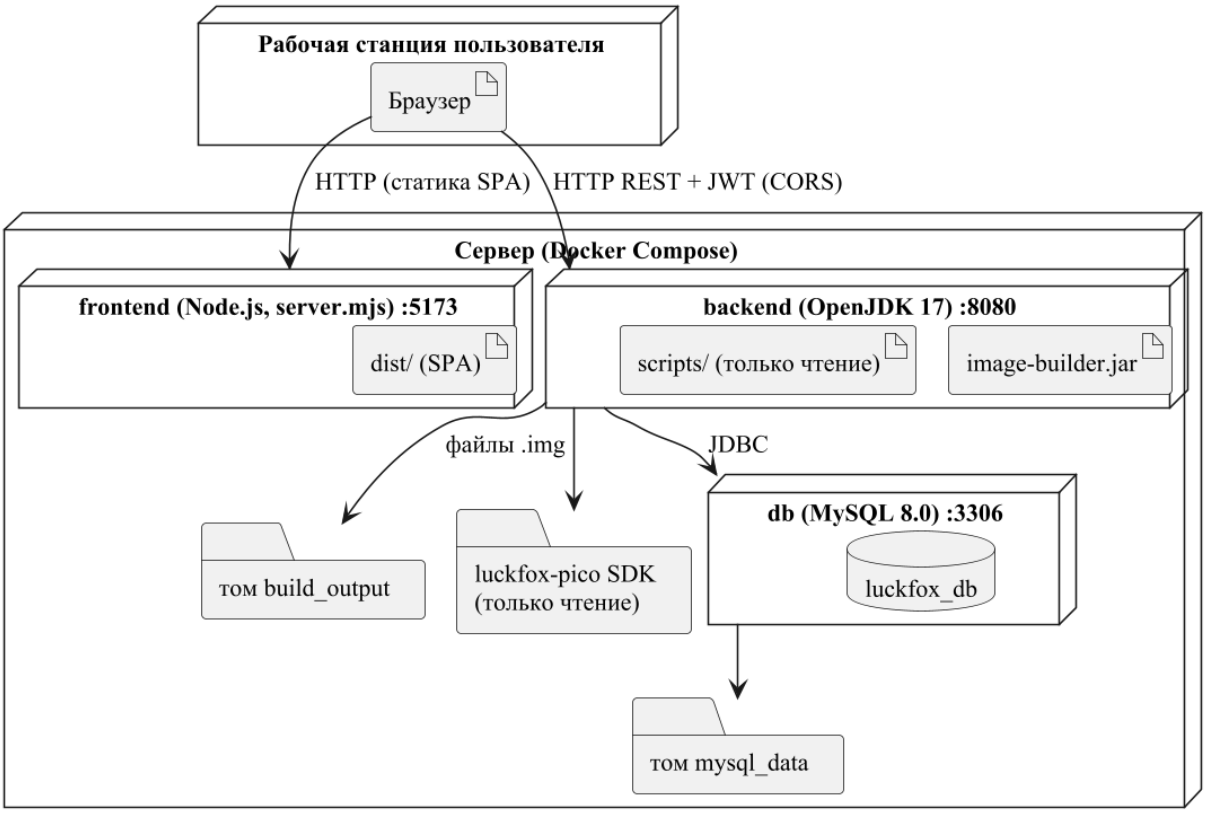


Рисунок Г.3 – Диаграмма развёртывания системы

ПРИЛОЖЕНИЕ Д

СХЕМА БАЗЫ ДАННЫХ И ОПИСАНИЕ ТАБЛИЦ

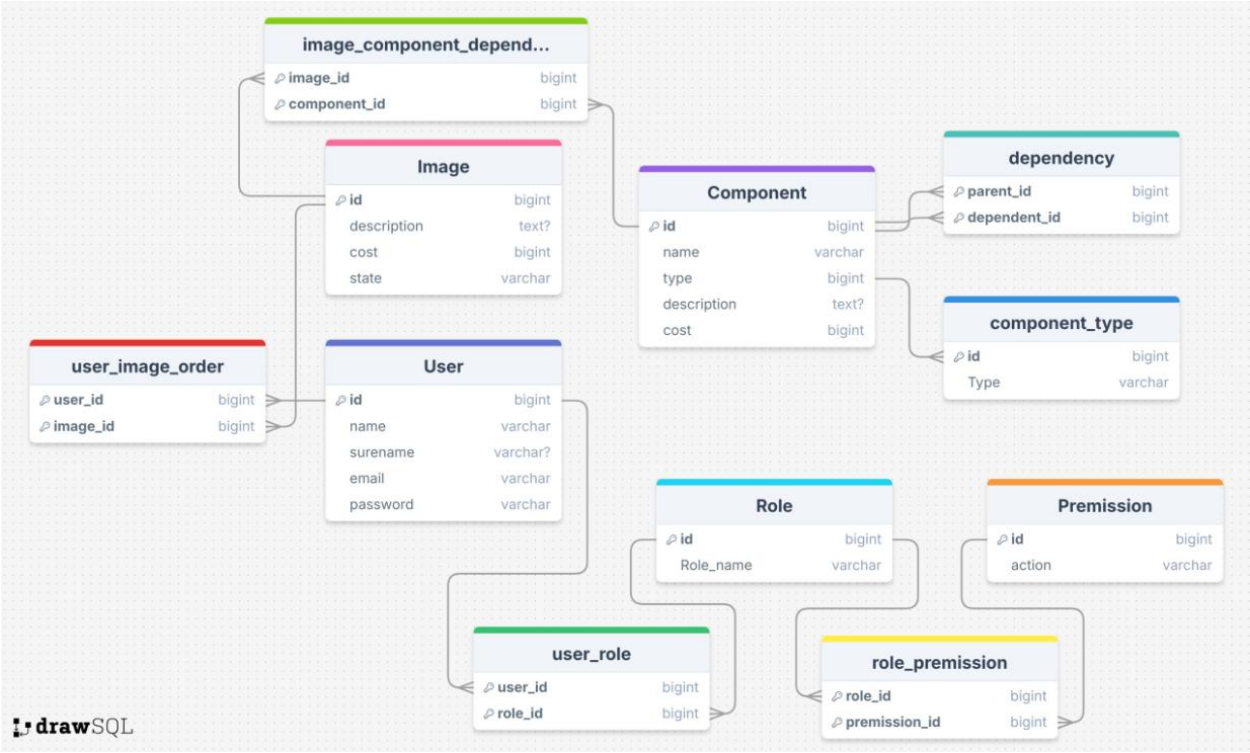


Рисунок Д.1 – ER-диаграмма базы данных (модель этапа проектирования)

На рисунке приведена ER-диаграмма, разработанная на этапе проектирования. В реализации состав таблиц и связей сохранён, а таблицы расширены: в Image добавлены поля name, distro, board, artifact_path, build_log и version, в Component – поле version; фактическая структура всех таблиц приведена ниже.

Рабочую схему базы данных создаёт Hibernate (параметр spring.jpa.hibernate.ddl-auto=update) по JPA-сущностям пакета entity; при первом запуске на пустой MySQL создаются все 11 таблиц, приведённых ниже, а при последующих запусках схема расширяется без удаления данных. Файл DOCs/этап3/java_working.sql является ранней документной моделью и служит только справочным материалом. Имена таблиц и столбцов в приложении указаны так, как они заданы аннотациями @Table, @Column и @JoinTable (в том числе с историческими написаниями Premission и surename); таблица «User» в

запросах заключается в обратные кавычки, так как User – зарезервированное слово MySQL.

Таблица Д.1 – Таблица «User»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор пользователя
name	VARCHAR(255)	NOT NULL	Имя пользователя
surename	VARCHAR(255)	NULL	Фамилия пользователя
email	VARCHAR(255)	NOT NULL, UNIQUE	Адрес электронной почты, используется как логин
password	VARCHAR(255)	NOT NULL	Хеш пароля (BCrypt)

Таблица Д.2 – Таблица «Role»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор роли
Role_name	VARCHAR(255)	NOT NULL	Название роли (USER, ADMIN)

Таблица Д.3 – Таблица «Premission»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор разрешения
action	VARCHAR(255)	NOT NULL	Название действия, которое разрешено выполнять

Таблица Д.4 – Таблица «user_role»

Поле	**Тип**	**Ограничения**	**Описание**
user_id	BIGINT	PK (составной), FK → User.id	Пользователь
role_id	BIGINT	PK (составной), FK → Role.id	Роль, назначенная пользователю

Таблица Д.5 – Таблица «role_premission»

Поле	**Тип**	**Ограничения**	**Описание**
role_id	BIGINT	PK (составной), FK → Role.id	Роль
premission_id	BIGINT	PK (составной), FK → Premission.id	Разрешение, входящее в роль

Таблица Д.6 – Таблица «Image»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор образа
name	VARCHAR(255)	NULL	Название образа
description	TEXT	NULL	Описание образа
distro	VARCHAR(255)	NULL	Дистрибутив (например, buildroot)
board	VARCHAR(255)	NULL	Целевая плата (по умолчанию RV1103_Luckfox_Pico)
cost	BIGINT	NOT NULL, индекс idx_image_cost, входит в idx_image_state_cost	Стоимость образа – сумма стоимостей компонентов
state	VARCHAR(255)	NOT NULL, входит в idx_image_state_cost	Состояние сборки: CREATED, QUEUED, BUILDING, READY, FAILED
artifact_path	VARCHAR(255)	NULL	Путь к собранному файлу образа на сервере
build_log	TEXT	NULL	Журнал сборки
version	BIGINT	NULL, @Version	Номер версии записи для оптимистичной блокировки

Таблица Д.7 – Таблица «Component»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор компонента
name	VARCHAR(255)	NOT NULL, входит в idx_component_name	Название компонента
type	BIGINT	NOT NULL, FK → component_type.id, входит в idx_component_type_cost	Тип компонента
description	TEXT	NULL	Описание компонента
cost	BIGINT	NOT NULL, входит в idx_component_type_cost	Стоимость компонента
version	BIGINT	NULL, @Version	Номер версии записи для оптимистичной блокировки

Таблица Д.8 – Таблица «component_type»

Поле	**Тип**	**Ограничения**	**Описание**
id	BIGINT	PK, AUTO_INCREMENT	Идентификатор типа компонента
Type	VARCHAR(255)	NOT NULL	Название типа (категория) компонента

Таблица Д.9 – Таблица «dependency»

Поле	Тип	Ограничения	Описание
parent_id	BIGINT	PK (составной), FK → Component.id	Компонент, у которого есть зависимость
dependent_id	BIGINT	PK (составной), FK → Component.id	Компонент, от которого зависит parent_id

Таблица Д.10 – Таблица «image_component_dependency»

Поле	Тип	Ограничения	Описание
image_id	BIGINT	PK (составной), FK → Image.id	Образ
component_id	BIGINT	PK (составной), FK → Component.id	Компонент, входящий в состав образа

Таблица Д.11 – Таблица «user_image_order»

Поле	Тип	Ограничения	Описание
user_id	BIGINT	PK (составной), FK → User.id	Пользователь, заказавший образ
image_id	BIGINT	PK (составной), FK → Image.id	Заказанный образ

Для ускорения выборок в каталогах определены следующие индексы (заданы в @Table сущностей): idx_image_state_cost по полям (state, cost) – фильтр образов по состоянию с сортировкой по цене; idx_image_cost по полю (cost) – сортировка образов по цене; idx_component_name по полям (name, id) – сортировка и постраничный вывод компонентов по названию; idx_component_type_cost по полям (type, cost) – фильтр компонентов по типу с сортировкой по цене. Кроме того, на поле email таблицы «User» создан уникальный индекс, который гарантирует отсутствие дубликатов адресов при одновременной регистрации. В таблицах «Image» и «Component» предусмотрено поле version (аннотация @Version): при сохранении Hibernate сравнивает номер версии и отклоняет конкурентную запись устаревшей копии, что защищает от потерянных обновлений при одновременном редактировании; в этом случае API возвращает код 409.

ПРИЛОЖЕНИЕ Е

ДИАГРАММА НАВИГАЦИИ И ЭКРАНЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

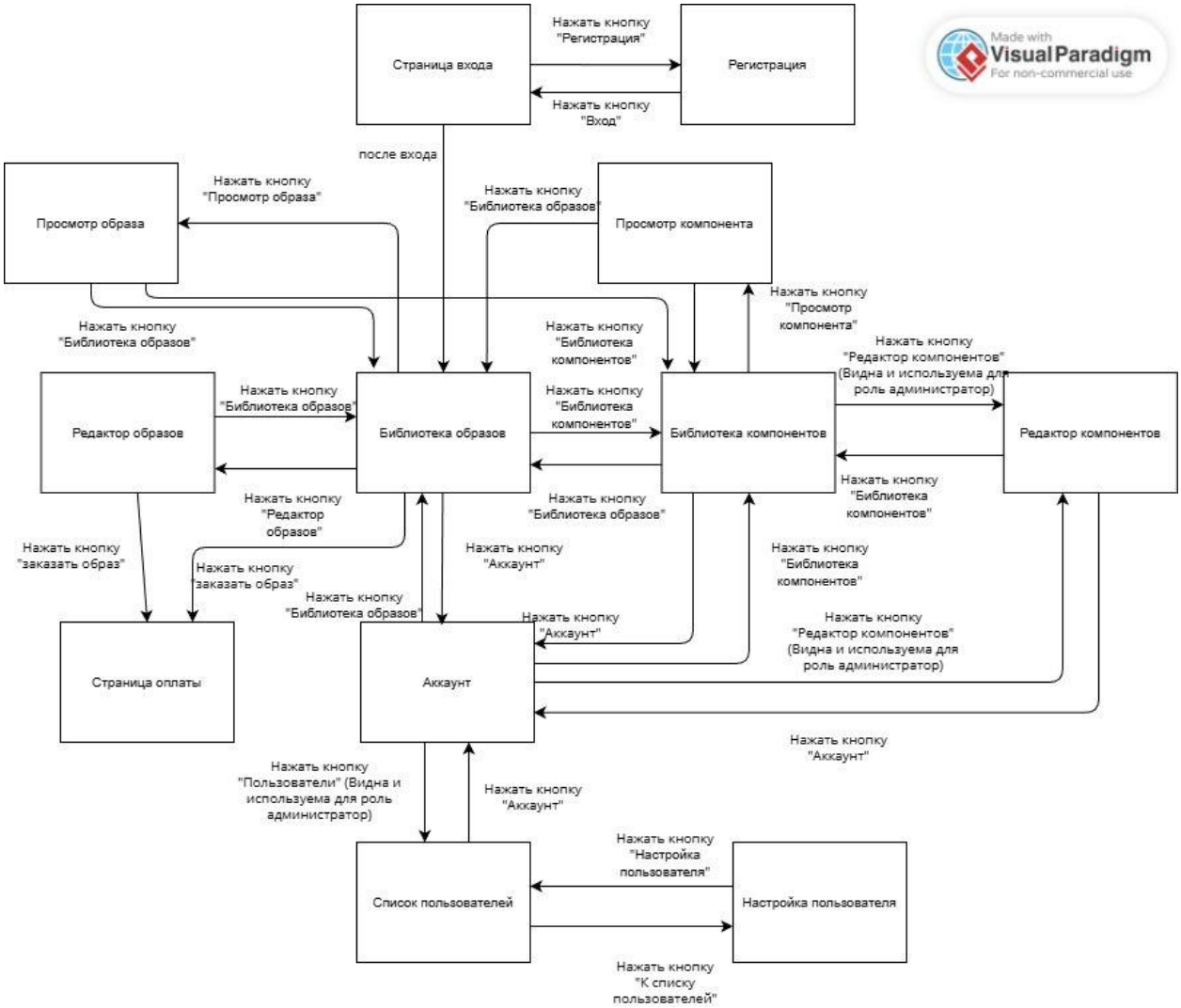


Рисунок Е.1 – Диаграмма навигации

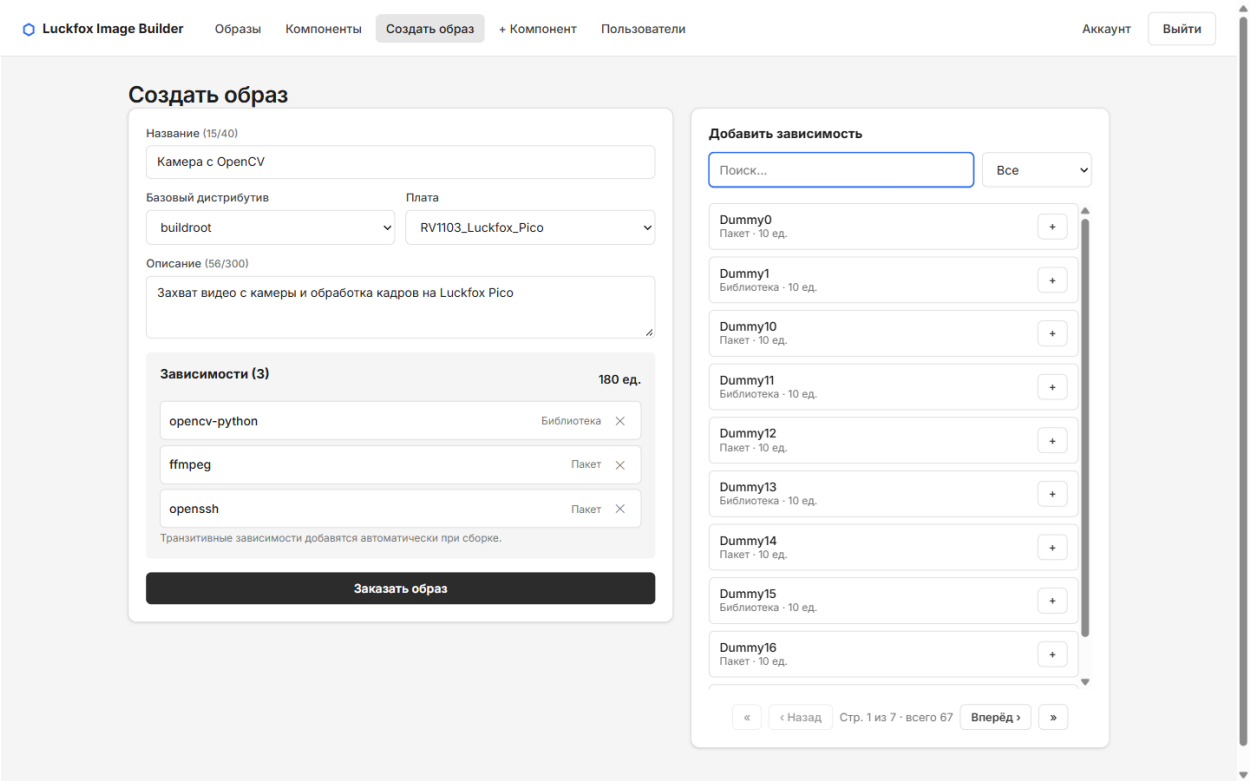


Рисунок Е.2 – Редактор образа

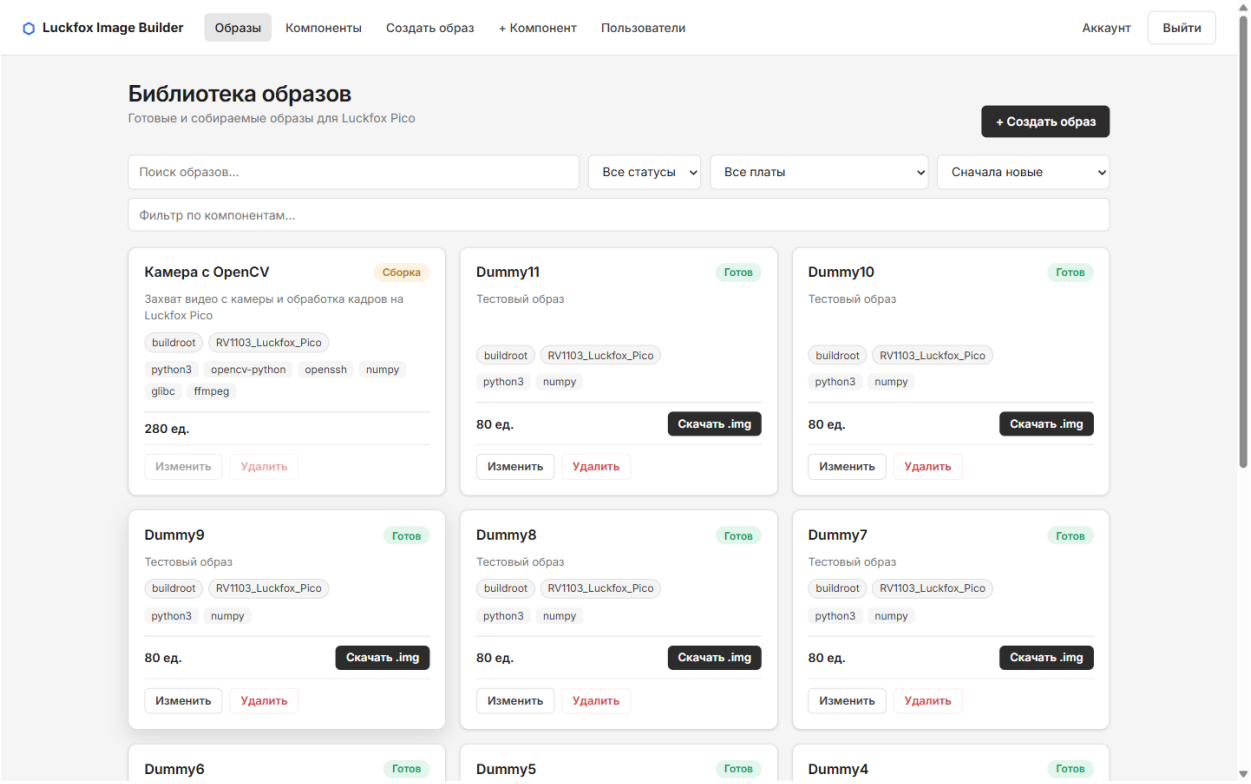


Рисунок Е.3 – Библиотека образов после заказа сборки

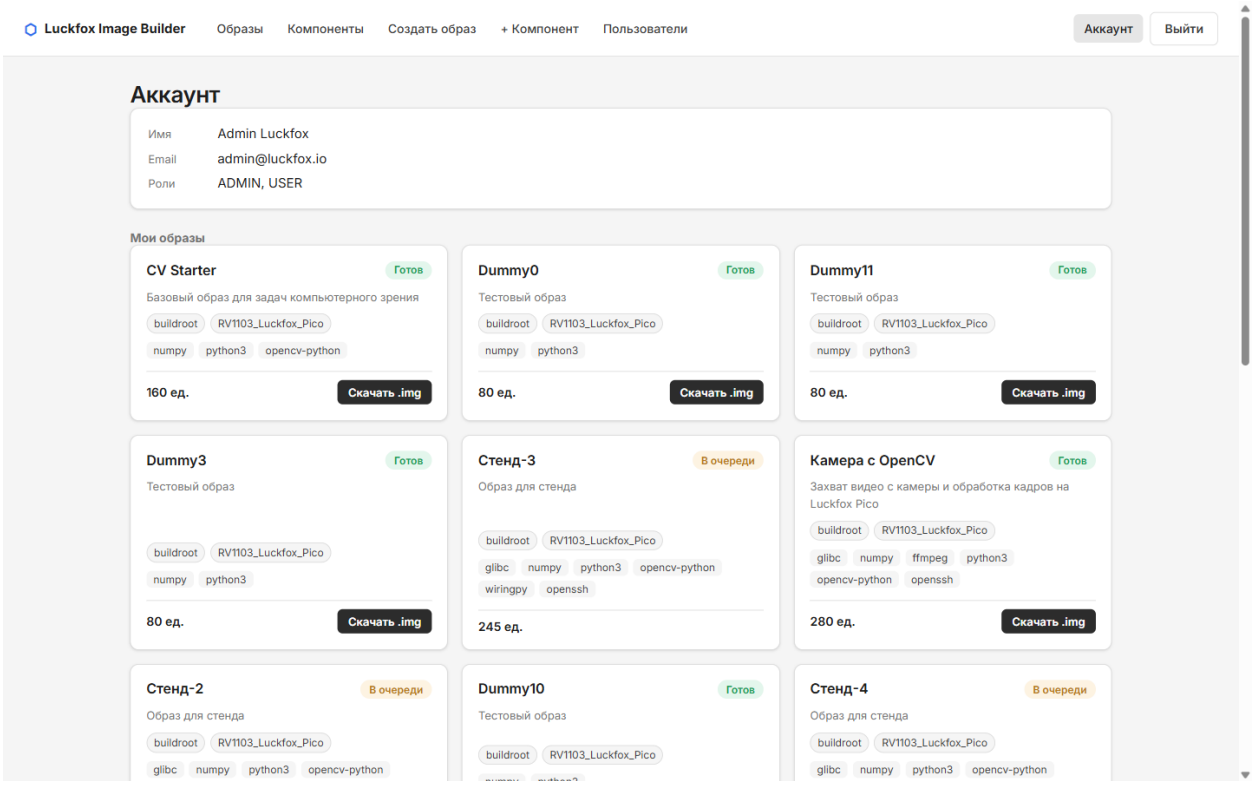


Рисунок Е.4 – Аккаунт: образы в очереди сборки

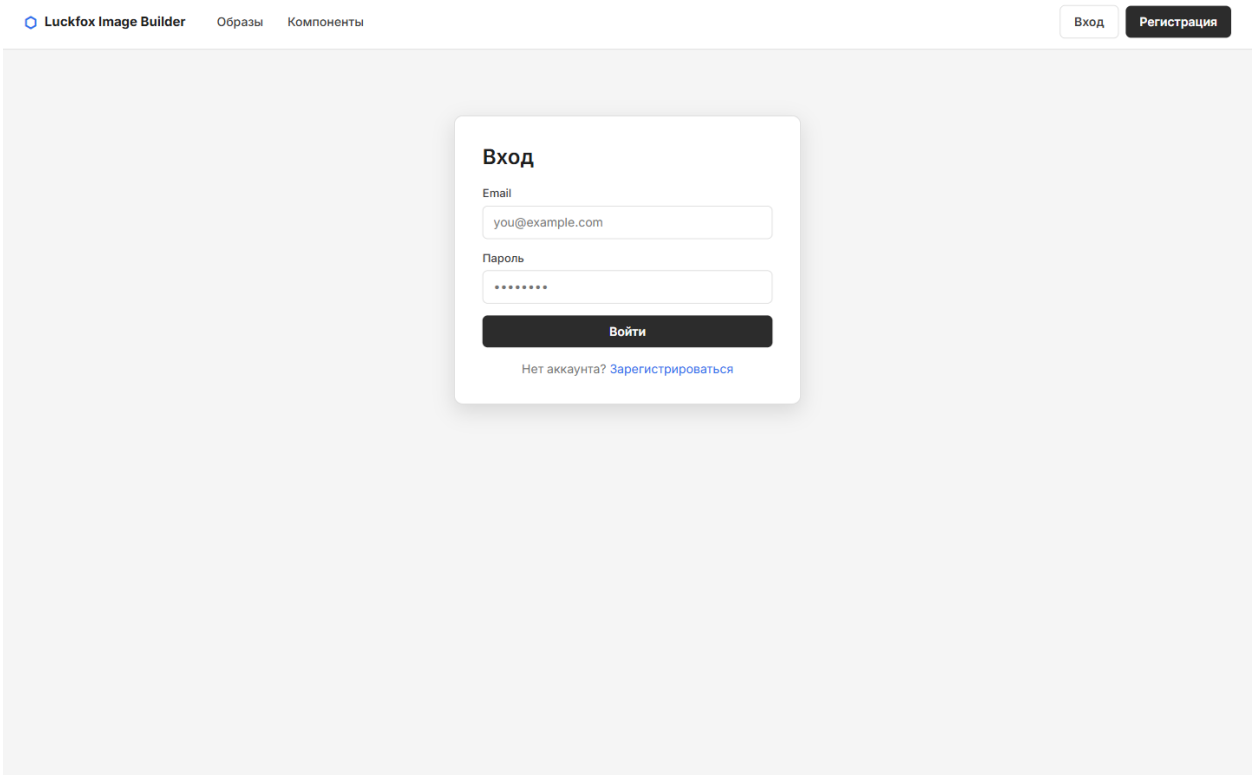


Рисунок Е.5 – Страница входа

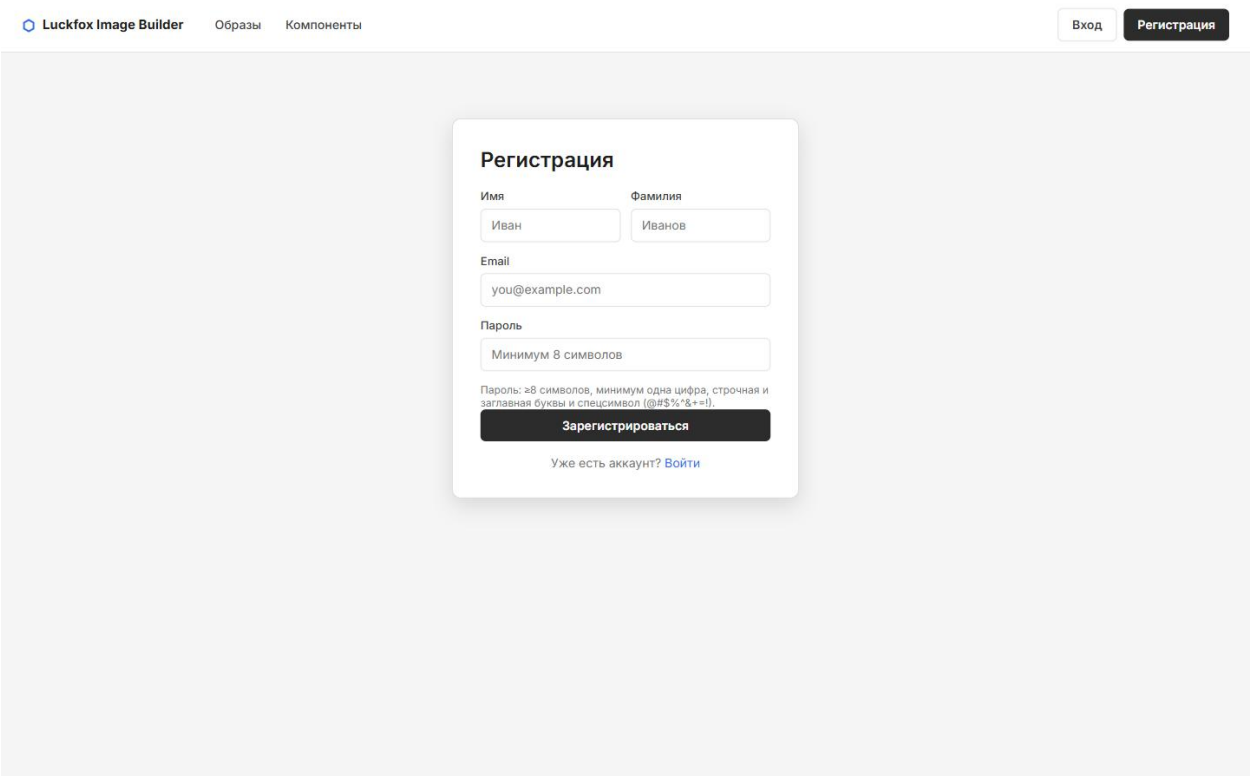


Рисунок Е.6 – Страница регистрации

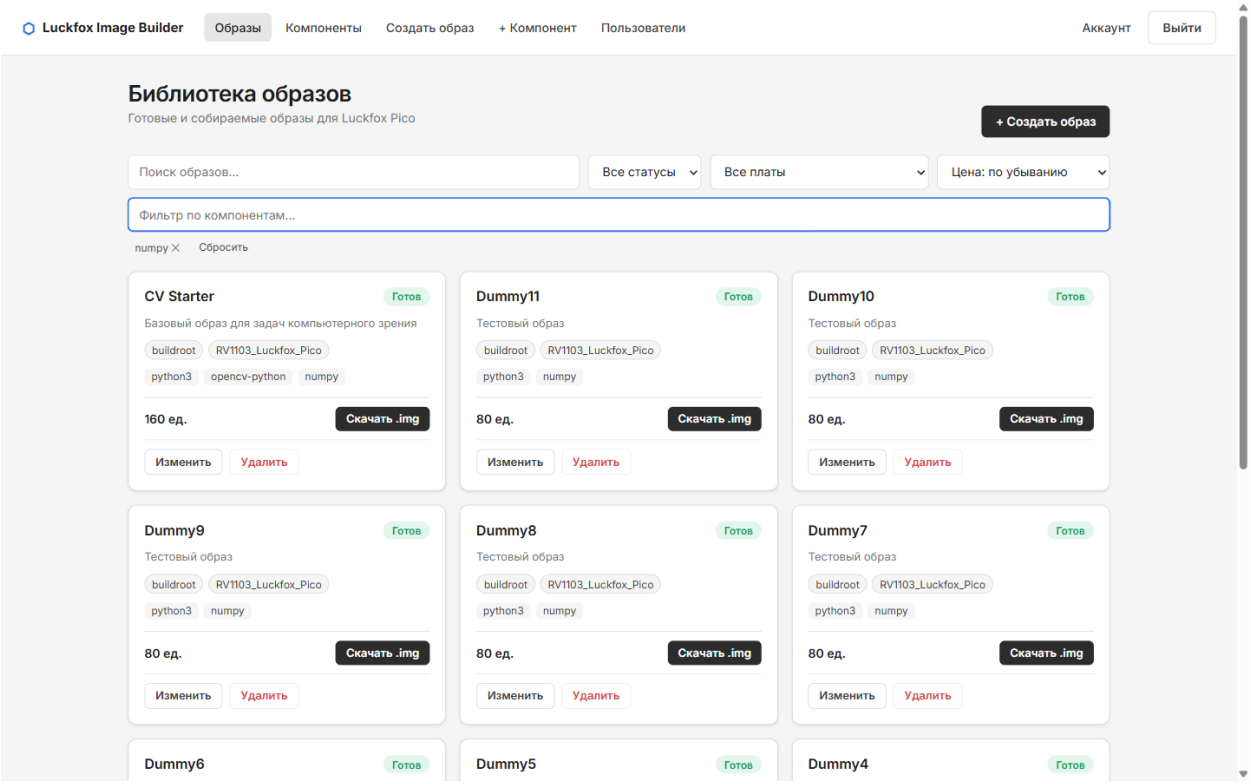


Рисунок Е.7 – Библиотека образов с фильтром по компоненту и сортировкой по цене

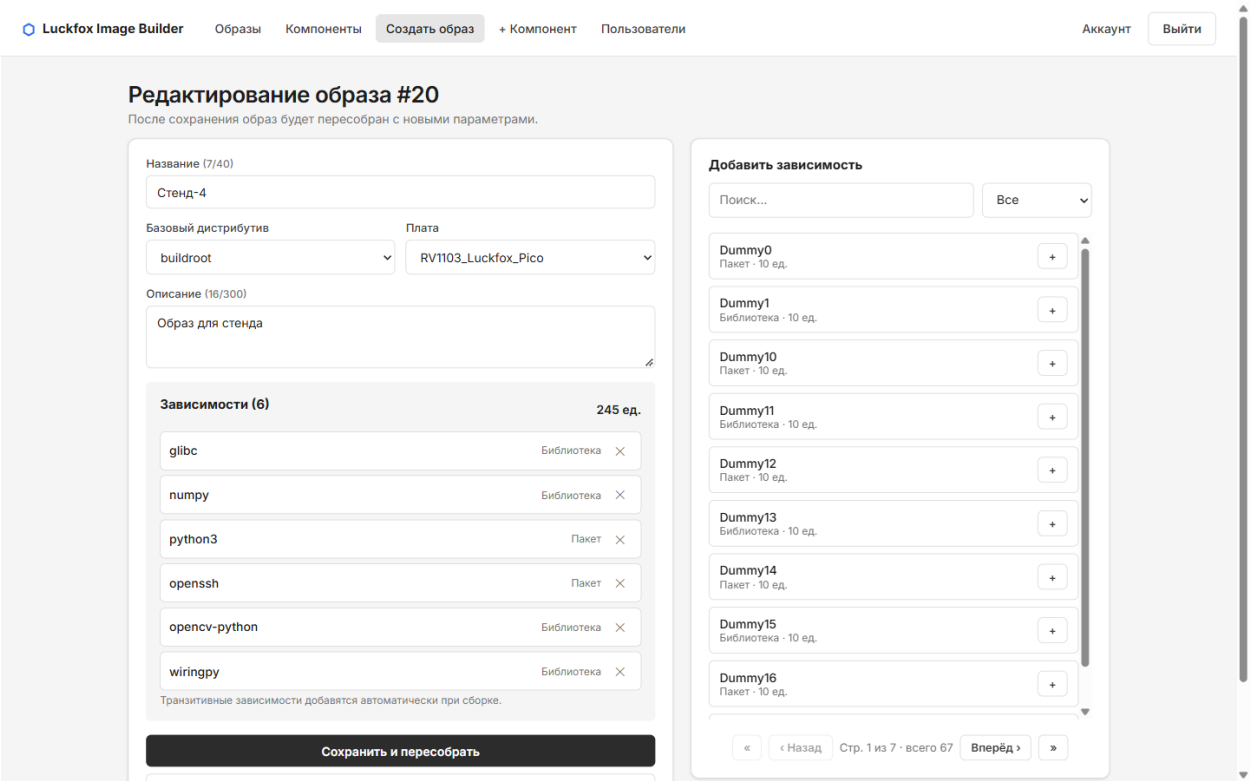


Рисунок Е.8 – Изменение образа администратором с пересборкой

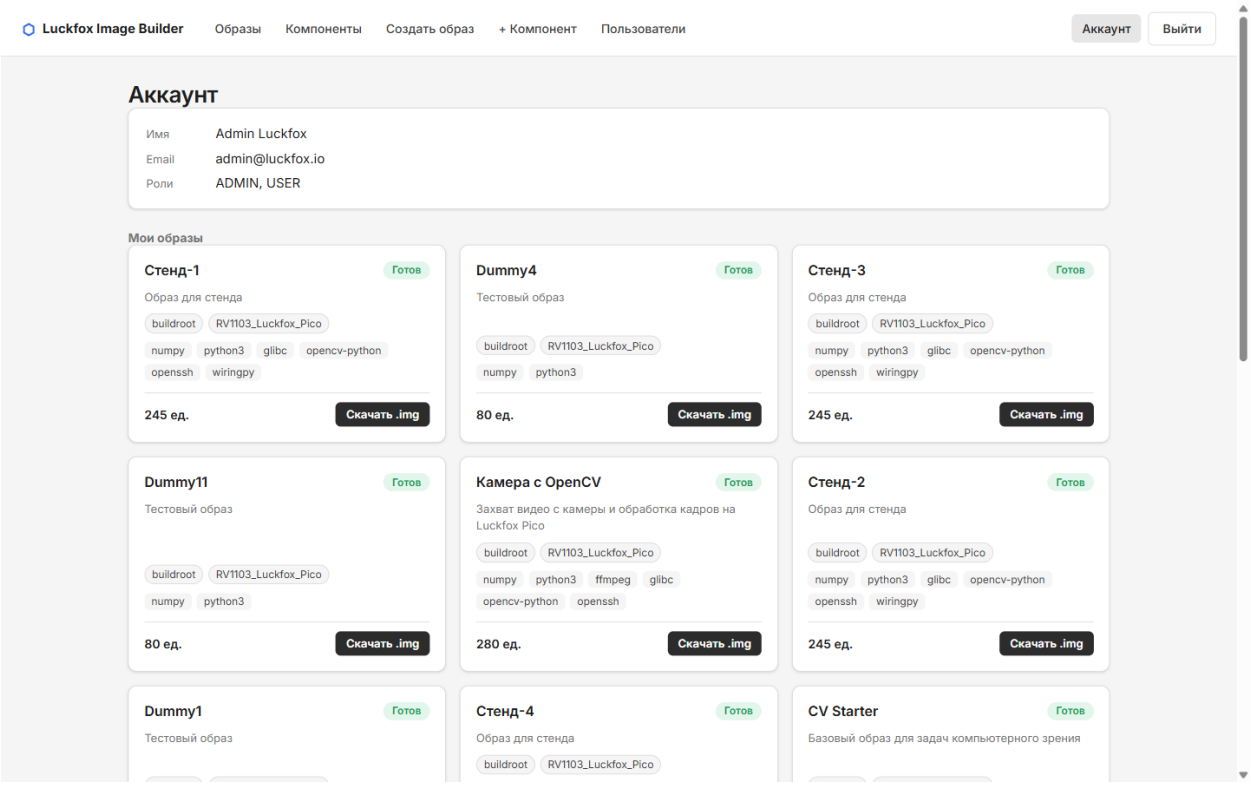


Рисунок Е.9 – Аккаунт и раздел «Мои образы» с готовыми образами

ПРИЛОЖЕНИЕ Ж

ПРОТОКОЛЫ И СВОДНЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Ж.1 Модульные тесты

Модульные тесты написаны на JUnit 5 с использованием Mockito и AssertJ и размещены в каталоге backend/src/test/java. Они проверяют сервисы, DTO, валидацию запросов, сущность пользователя и обработчик ошибок без обращения к реальной базе данных. Состав тестовых классов приведён в таблице Ж.1.

Таблица Ж.1 – Состав модульных тестов

Класс теста	**Тестируемый объект**	**Кол-во**	**Проверяемые свойства**
DtoMappingTest	Преобразование сущностей в DTO	5	Разворачивание прямых зависимостей компонента, обработка пустого списка зависимостей, признак доступности образа для скачивания, скрытие пути к артефакту и журнала, отсутствие пароля в ответе о пользователе
PageResponseTest	Постраничная выдача (PageRequest, PageResponse)	5	Ограничение номера и размера страницы, сортировка по цене с сохранением сортировки по умолчанию, замена неизвестной сортировки, перенос данных страницы в ответ
RequestValidationTest	Валидация запросов (регистрация, сборка, компонент, пользователь)	10	Приём корректной регистрации, отклонение 5 вариантов слабого пароля, некорректный email и пустое имя, обязательность названия, дистрибутива и компонентов, запрет отрицательной цены, обязательность роли
UserTest	Сущность User (UserDetails)	2	Префикс ROLE_ у полномочий, использование email в качестве имени пользователя и активность учётной записи
GlobalExceptionHandlerTest	Глобальный обработчик ошибок	5	Ошибки валидации по полям, код 404 с сообщением, код 429 с заголовком Retry-After, код 503 при исчерпании пула соединений, код 409 при конфликте оптимистичной блокировки
AuthenticationServiceTest	Сервис регистрации и входа	6	Регистрация с хешированием пароля и ролью USER, создание отсутствующей роли, отказ при занятом email, успешный вход

Класс теста	Тестируемый объект	Кол-во	Проверяемые свойства
			со сбросом счётчика, учёт неудачных попыток, пропуск проверки пароля при блокировке
BuildServiceTest	Сервис сборки образов	6	Успешная сборка, передача компонентов в сценарий, ненулевой код возврата, отсутствие артефакта, принудительное завершение по таймауту, отсутствие образа
BuildStateServiceTest	Сервис состояний сборки	4	Чтение входных данных с сортировкой компонентов, фиксация готовности, сохранение журнала при начале сборки, ошибка при отсутствии образа
ComponentServiceTest	Сервис компонентов	8	Создание с типом и зависимостями, неизвестные тип и зависимость, запрет зависимости от самого себя, транзитивный цикл, запрет удаления используемых компонентов, удаление свободного компонента
ImageServiceTest	Сервис образов	11	Заказ с транзитивными зависимостями и подсчётом стоимости, целевая плата по умолчанию, привязка к заказчику и запуск сборки, неизвестные компонент и пользователь, циклический граф, изменение, удаление и получение образа
JwtServiceTest	Сервис JWT-токенов	6	Выпуск токена и проверка его для владельца, отклонение для другого пользователя, дополнительные утверждения, срок жизни, поддельный и просроченный токены
LoginAttemptServiceTest	Ограничение частоты попыток входа	7	Допуск попыток ниже лимита, блокировка email и IP-адреса с Retry-After, нечувствительность ключа к регистру и пробелам, сброс счётчика после успеха, истечение окна, обработка пустого email
UserServiceTest	Сервис управления пользователями	9	Создание с хешированием пароля и назначением ролей, отказ без пароля, дубликат email, неизвестная роль, сохранение прежних данных при обновлении, запрет самоблокировки администратора, удаление и получение пользователя

Всего выполнено 84 модульных теста (сумма по таблице Ж.1), ошибок и пропусков нет: команда mvn test завершилась со статусом BUILD SUCCESS.

Тесты подсистемы сборки образов (заказ, изменение и удаление образов, запуск сборочного сценария, смена состояний, каталог компонентов) приведены отдельно в таблице Ж.2.

Таблица Ж.2 – Модульные тесты подсистемы сборки

Тест	Проверка
ImageServiceTest.orderResolvesTransitiveDependenciesAndSumsCostOnce	Заказ образа с транзитивными зависимостями: в состав попадают все зависимости, стоимость общих компонентов учитывается один раз (80 + 20 + 30 + 10), состояние QUEUED, скачивание недоступно
ImageServiceTest.orderUsesDefaultBoardWhenNotGiven	Если плата не указана, используется RV1103_Luckfox_Pico
ImageServiceTest.orderLinksImageToCustomerAndSchedulesBuild	Образ привязывается к заказчику, сборка ставится в очередь
ImageServiceTest.orderWithUnknownComponentFails	Заказ с несуществующим компонентом отклоняется (ResourceNotFoundException), сборка не запускается
ImageServiceTest.orderByUnknownUserFails	Заказ от неизвестного пользователя отклоняется
ImageServiceTest.cyclicDependencyGraphIsRejected	Циклический граф зависимостей отклоняется ошибкой 400, образ не сохраняется
ImageServiceTest.updateOfQueuedOrBuildingImageIsRejected	Изменение образа в состоянии QUEUED или BUILDING запрещено
ImageServiceTest.updateRequeuesImageAndDropsOldArtifact	Изменение готового образа: пересчёт стоимости, состояние QUEUED, очистка пути и журнала, удаление старого файла, повторный запуск сборки
ImageServiceTest.deleteOfBuildingImageIsRejected	Удаление образа в состоянии BUILDING запрещено
ImageServiceTest.deleteUnlinksCustomersAndRemovesArtifact	При удалении образа снимаются связи с заказчиками и удаляется файл артефакта
ImageServiceTest.getByIdOfMissingImageFails	Запрос несуществующего образа даёт ResourceNotFoundException
BuildServiceTest.successfulBuildMarksReadyWithArtifactAndNotifies	Успешная сборка: образ помечается READY с путём к артефакту, отправляется уведомление, отметки FAILED нет
BuildServiceTest.componentsArePassedToScript	Сборочному сценарию передаются идентификатор, название, дистрибутив, плата и список компонентов
BuildServiceTest.nonZeroExitMarksFailedWithExitCode	Ненулевой код возврата: образ помечается FAILED, в журнале вывод сценария и код возврата, уведомление не отправляется

Тест	Проверка
BuildServiceTest.zeroExitWithoutArtifactIsFailure	Нулевой код возврата без созданного файла считается сбоем
BuildServiceTest.hangingBuildIsKilledByTimeout	Зависшая сборка принудительно завершается по таймауту и помечается FAILED
BuildServiceTest.missingImageStopsBeforeBuilding	Если образ не найден, сборка не начинается
BuildStateServiceTest.loadInputsReturnsSortedComponentNames	Входные данные сборки: названия компонентов отсортированы, переданы дистрибутив и плата
BuildStateServiceTest.markReadyStoresArtifactAndLog	Состояние READY сохраняет путь к артефакту и журнал
BuildStateServiceTest.markBuildingKeepsPreviousLog	Начало сборки переводит образ в BUILDING и не стирает прежний журнал
BuildStateServiceTest.markFailedOnMissingImageThrows	Перевод в FAILED несуществующего образа вызывает ResourceNotFoundException
ComponentServiceTest.createSetsFieldsTypeAndDependencies	Создание компонента: поля, тип и зависимости сохраняются и отражаются в DTO
ComponentServiceTest.createWithUnknownTypeFails	Неизвестный тип компонента отклоняется, сохранения нет
ComponentServiceTest.createWithUnknownDependencyFails	Неизвестная зависимость отклоняется
ComponentServiceTest.componentCannotDependOnItself	Компонент не может зависеть сам от себя (ошибка 400)
ComponentServiceTest.transitiveCycleIsRejectedOnUpdate	Обновление, замыкающее транзитивный цикл зависимостей, отклоняется
ComponentServiceTest.deleteOfComponentUsedInImagesIsRejected	Нельзя удалить компонент, входящий в образы
ComponentServiceTest.deleteOfComponentOthersDependOnIsRejected	Нельзя удалить компонент, от которого зависят другие компоненты
ComponentServiceTest.deleteOfUnusedComponentSucceeds	Неиспользуемый компонент удаляется

Ж.2 Интеграционные и сквозные тесты

Интеграционное тестирование выполнено методом чёрного ящика по HTTP на прод-подобном стенде (47 API-тестов, скрипт tests/integration/run_integration.py) с дополнением сквозными E2E-тестами в браузере (9 тестов, Playwright). Полный протокол приведён в таблице Ж.3;

результат: 56 тестов, из них 34 PASS и 22 FAIL. Каждое отклонение связано с дефектом из реестра (см. раздел Ж.4).

Таблица Ж.3 – Протокол интеграционного и сквозного тестирования

ID	Тест	Ожидаемый результат	Фактический результат	Статус	Дефект
INT-01	Фронтенд (prod-сборка) отдаёт SPA и ассеты	200, text/html; бандлы 200 с корректным Content-Type	index-CTuQi8U5.js text/javascript; charset=utf-8 244 КБ; index-CYtgMNis.css text/css; charset=utf-8 8 КБ	PASS	–
INT-02	Прямой переход по клиентскому маршруту (deep link)	200 и index.html (SPA fallback)	/editor/image → 200, /admin/users → 200, /no/such/route → 200	PASS	–
INT-03	Отдача несуществующего ассета	404 для отсутствующего файла со статикой	HTTP 200, Content-Type text/html; charset=utf-8 – браузер получит HTML вместо JS	FAIL	DEF-01
INT-04	CORS: preflight с origin фронтенда	200, Access-Control-Allow-Origin = origin	HTTP 200, ACAO=http://localhost:5173	PASS	–
INT-05	CORS: origin боевого домена	Разрешённые origin задаются конфигурацией (переменной окружения)	HTTP 403 «Invalid CORS request»: фронтенд на другом адресе не сможет обращаться к API, origin без пересборки не изменить	FAIL	DEF-02
INT-06	Схема БД создана Hibernate на чистой MySQL	Все таблицы есть, уникальный индекс на User.email	11 таблиц, уникальный индекс email есть	PASS	–
INT-07	Начальное наполнение (DataSeeder) через конвейер сборки	67 компонентов, 15 образов; все стартовые образы READY и скачиваемы	67 компонентов, 15/15 образов READY (очередь опустела за 0 с), вход admin OK	PASS	–
INT-08	Регистрация и вход	200, JWT, профиль с ролью USER	токен выдан, /users/me → it-c28bab-117d46@test.io ['USER']	PASS	–
INT-09	Валидация регистрации	400 с понятным сообщением для каждого случая	weak password→400, bad email→400, empty name→400, duplicate→400	PASS	–
INT-10	Вход с неверным паролем / несуществующим email	401 Unauthorized с сообщением об ошибке	неверный пароль → HTTP 403 «»; нет пользователя → HTTP 403	FAIL	DEF-03
INT-11	Доступ к защищённым ресурсам без токена	401 (фронтенд перенаправляет на вход только по 401)	/users/me → 403, /images/build → 403	FAIL	DEF-04
INT-12	Повреждённый и чужой по подписи токен	401, без 500 и стектрейсов	коды: {'мусор': 403, 'чужая подпись': 403}	FAIL	DEF-05
INT-13	Просроченный токен	401 → фронтенд отправляет на вход	HTTP 403 (ExpiredJwtException не обработано)	FAIL	DEF-05
INT-	Время жизни JWT	24 часа (так указано в коде/документации)	4.0 ч	FAIL	DEF-

ID	Тест	Ожидаемый результат	Фактический результат	Статус	Дефект
14					06
INT-15	Ключ подписи JWT в исходном коде	Ключ берётся из секрета окружения; такой токен отклоняется	GET /users (ADMIN) с токеном из известного ключа → HTTP 200	FAIL	DEF-07
INT-16	Разграничение ролей USER / ADMIN	403 для всех	GET /users→403, POST /components→403, PUT /images→403, DELETE /images→403, DELETE /users/1→403	PASS	–
INT-17	Токен удалённого пользователя	401	HTTP 403	FAIL	DEF-08
INT-18	Гонка: одновременная регистрация одного email	Ровно 1 успех, остальные 400/409, дублей в БД нет, 5xx нет	коды {200: 1, 409: 9, 400: 10}, строк в БД 1	PASS	–
INT-19	Пагинация: граничные значения	size обрезается до 100, page<0 → 0, пустая страница за пределами	size 1000→100, 0→1, page -1→0, page 999 → пусто	PASS	–
INT-20	Нечисловые параметры (анонимный пользователь)	400 Bad Request	{'page=abc': 403, 'components=x': 403, '/components/abc': 403}	FAIL	DEF-10
INT-21	Спецсимволы и инъекции в поиске	Нет 5xx; спецсимволы трактуются буквально (% и _ не шаблоны)	Инъекции нет (параметризованные запросы), но % и _ работают как шаблоны: найдено по 67 записей	FAIL	DEF-11
INT-22	Комбинация поиска, типа, сортировки и страниц	Без дублей и пропусков, порядок по цене сохраняется между страницами	30 записей на 5 стр., дублей нет	PASS	–
INT-23	Число SQL-запросов на одну страницу образов	≤ 10 запросов (без проблемы N+1)	GET /images?size=12 → 23 SQL-запросов; GET /components?size=20 → 31	FAIL	DEF-12
INT-24	Создание компонента с зависимостями и чтение	201/200, зависимости сохранены	HTTP 200, id=68, deps=[1]	PASS	–
INT-25	Защита графа зависимостей	400 в обоих случаях, граф не меняется	self→400, cycle→400	PASS	–
INT-26	Удаление используемого и свободного компонента	400 с сообщением; 204	used→400 «Компонент «numpy» входит в образы – сначала измени», free→204, затем GET→404	PASS	–
INT-27	Валидация полей компонента	400 для некорректных полей (404 допустим для несуществующего типа)	{'empty name': 400, 'negative cost': 400, 'name 300': 409, 'bad type': 404}	FAIL	DEF-13
INT-28	Параллельное редактирование компонента двумя админами	Без 5xx; итог – одно из отправленных значений	коды {200: 1, 409: 9}, итоговая цена 104: оптимистичная блокировка (@Version) отсекла конкурентные записи	PASS	–
INT-29	Заказ сборки с разрешением транзи-	202, QUEUED, в составе numpy и python3, цена = сумма компонентов	HTTP 202, state=QUEUED, состав=['numpy', 'opencv-python', 'python3'], cost=160	PASS	–

ID	Тест	Ожидаемый результат	Фактический результат	Статус	Дефект
	тивных зависимостей				
INT-30	Сборка до READY и появление в «Мои образы»	READY, downloadable=true, образ в /users/me/images	READY за 2.1 с	PASS	–
INT-31	Скачивание артефакта и проверка содержимого	200, octet-stream, валидный ext4 (магия 0xEF53), размер 16+N МБ, манифест – валидный JSON с составом	ext4 OK, 19 МБ, пакеты ['numpy', 'opencv-python', 'python3'], манифест: IT Образ «Тест» c28bab	PASS	–
INT-32	Имя файла при скачивании (кириллица)	Корректное имя файла (filename*=UTF-8"… по RFC 6266)	HTTP 200, заголовок Content-Disposition отсутствует (значение с не-Latin-1 символами отброшено), файл сохранится как «download»	FAIL	DEF-15
INT-33	Скачивание чужого образа	403 (скачивать может владелец или ADMIN)	HTTP 200, получено 19 МБ чужого образа	FAIL	DEF-16
INT-34	Скачивание без токена и неготового образа	401; 400 «не готов»	аноним → HTTP 403; неготовый → 400	FAIL	DEF-04
INT-35	Валидация заказа	400/404 без 5xx	{'empty components': 400, 'unknown component': 404, 'empty name': 400}	PASS	–
INT-36	Очередь сборок: один воркер, FIFO	Одновременно BUILDING не более 1; последовательные заказы завершаются в порядке заказа; все READY	max BUILDING = 1, FIFO соблюдён, 12 сборок за 35,5 с (3,0 с на сборку); одновременные заказы идут в порядке фиксации транзакций	PASS	–
INT-37	Изменение образа администратором с пересборкой	202 → QUEUED → READY; новый состав в артефакте; старый файл заменён	202, пересобран за 3.6 с, состав ['glibc', 'openssh'], cost 60; PUT во время сборки → 400	PASS	–
INT-38	Удаление образа администратором	204; GET → 404; файл удалён с диска; пропал из «Мои образы»	204, 404, файл удалён, связь с пользователем снята	PASS	–
INT-39	Удаление пользователя, у которого есть образы	204; образы остаются в библиотеке	204, образ сохранён	PASS	–
INT-40	Артефакт удалён с диска вне приложения	downloadable=false либо понятная ошибка; без 5xx	downloadable=True, download → HTTP 400 {"error":"Файл артефакта не найден на сервере"}	FAIL	DEF-17
INT-41	CRUD пользователей администратором	201/200 на каждом шаге, новые права действуют	создан, повышен до ADMIN, вход с новым паролем, поиск+фильтр, удалён	PASS	–
INT-42	Защита от самоблокировки администратора	400 в обоих случаях	400 / 400	PASS	–
INT-43	Смена email администратором самому себе	Сессия продолжает работать (или выдаётся новый токен)	PUT → 200, затем /users/me со старым токеном → 403	FAIL	DEF-18

ID	Тест	Ожидаемый результат	Фактический результат	Статус	Дефект
INT-44	Перезапуск backend: сохранность данных	Данные сохранены, дублей каталога нет, токен остаётся валидным	компонентов/образов (71, 31) = до рестарта; простой 12 с; токен валиден	PASS	–
INT-45	Падение backend (kill -9) во время сборки	Прерванные сборки возобновляются или помечаются FAILED; их можно пересобрать/удалить	Через 15 с после рестарта статусы {33: BUILDING, 34: QUEUED, 35: QUEUED}; PUT/DELETE зависших: (400, 400), (400, 204), (400, 204)	FAIL	DEF-19
INT-46	Недоступность БД	Во время сбоя – быстрый 5xx (≤ 5 с) без утечки деталей; после восстановления – 200 без рестарта	во время сбоя HTTP 403 за 28.3 с; восстановление без рестарта за 2.9 с	FAIL	DEF-20
INT-47	Состояние стенда после всех тестов	API отвечает, очередь пуста	API отвечает	PASS	–
E2E-01	Открытие главной, редирект в библиотеку, отрисовка карточек	Редирект на /library/images, карточки отрисованы < 2 с	/library/images, карточек 12, первая карточка через 412 мс (DOMContentLoaded 50 мс)	PASS	–
E2E-02	Регистрация через форму	Вход выполнен, токен сохранён, переход в библиотеку	после регистрации → /library/images, токен в localStorage: true	PASS	–
E2E-03	Фильтр по компоненту и сортировка по цене	IoT Edge, Media Box (по убыванию цены)	IoT Edge, Media Box	PASS	–
E2E-04	Заказ сборки в редакторе	Образ появляется в «Мои образы», статус доходит до «Готов», зависимость glibc добавлена автоматически	В кабинете: образ «E2E образ», Готов, buildroot, RV1103_Luckfox_Pico, glibc + ffmpeg, 80 ед., кнопка «Скачать .img»	PASS	–
E2E-05	Скачивание .img из интерфейса	Браузер сохраняет файл <название>.img корректного размера	файл «E2E образ.img», 18 874 368 байт (18 МБ = 16 + 2 компонента)	PASS	–
E2E-06	Недействительный токен в браузере	Сессия сбрасывается, редирект на /login	Пользователь остался на /account: «Request failed with status code 403»; API отвечает 403, а клиент перехватывает только 401	FAIL	DEF-04
E2E-07	Админ: страница пользователей и поиск	Зарегистрированный в E2E-02 пользователь найден	пользователь найден	PASS	–
E2E-08	Неверный пароль в форме входа	Понятное сообщение «Неверный email или пароль»	показано техническое сообщение «Request failed with status code 403»	FAIL	DEF-03
E2E-09	JS-ошибки за весь сценарий	Нет необработанных исключений JS	нет	PASS	–

После оптимизации (индексы, пакетная загрузка связей, отключение Open Session In View, быстрый отказ при недоступной БД, ограничение неудачных

входов) интеграционный набор был выполнен повторно на чистой базе данных. Изменились результаты только двух тестов: INT-23 (число SQL-запросов на страницу образов: 9 вместо 23) и INT-46 (недоступность БД: ответ 503 за 3,0 с вместо 403 через 28,3 с) перешли из FAIL в PASS. Добавлен тест INT-48 «Защита от перебора паролей» – PASS (после 5 неудачных попыток ответ 429 с Retry-After 300 с, другой пользователь входит без ограничений). Итого 30 PASS из 48 API-тестов (было 27 из 47), регрессий нет; изменения сведены в таблицу Ж.4.

Таблица Ж.4 – Изменения результатов после оптимизации

ID	**До**	**После**	**Комментарий**
INT-23	FAIL	PASS	Страница образов: 9 SQL-запросов (было 23), устранена проблема N+1 (DEF-12)
INT-46	FAIL	PASS	Недоступная БД: ответ 503 за 3,0 с, восстановление без рестарта за 1,0 с (DEF-20)
INT-48	– (новый)	PASS	Защита от перебора паролей: 429 после 5 неудачных попыток, Retry-After 300 с (DEF-21)

Ж.3 Нагрузочное тестирование

Нагрузочное тестирование выполнено с помощью k6 (скрипт tests/load/scenarios.js) на стенде WSL2 Ubuntu 24.04, 16 vCPU, 7,5 ГБ, MySQL 8, JDK 17; сборка образов – в режиме симуляции. Профили нагрузки приведены в таблице Ж.5.

Таблица Ж.5 – Профили нагрузочного тестирования

ID	**Профиль**	**Модель нагрузки**	**Критерии**
L-01	Дымовой (baseline)	1 VU, 1 мин; все пользовательские пути, каждая 5-я итерация – заказ и скачивание	Ошибки < 1 %; p95 < 300 мс
L-02	Просмотр каталогов (гости)	Рост 0→50 VU за 1 мин, удержание 3 мин, спад 30 с; чтение каталогов с поиском, фильтрами и сортировкой	Ошибки < 1 %; p95 чтения < 500 мс
L-03	Смешанный продакшен-профиль (целевая нагрузка)	100 одновременных пользователей: гости 70, пользователи 20, заказчики 8, администраторы 2; разгон 1 мин, удержание 5 мин	Ошибки < 1 %; p95 чтения < 500 мс; p95 записи < 1000 мс
L-04	Стресс (поиск предела)	Только чтение, ступени 50, 100, 200, 300, 450, 600 итераций/с по 1 мин, до 1500 VU	Ошибки < 1 %; p95 < 500 мс (нарушение ожидается на верхних ступенях)

ID	Профиль	Модель нагрузки	Критерии
L-04b	Стресс до отказа (breakpoint)	Ступени 1000, 1500, 2000, 3000, 4000 итераций/с по 1 мин, до 3000 VU	Фиксация точки насыщения (ошибки ≥ 1 % или p95 ≥ 500 мс)
L-05	Всплеск (spike)	10 VU, за 10 с рост до 400 VU, удержание 1 мин, возврат к 10 VU	Ошибки < 1 %; p95 < 500 мс; задержка возвращается к исходной
L-06	Шторм авторизаций	Ступени 10, 25, 50, 100 входов/с по 30 с, 50 учётных записей	Ошибки < 1 %; p95 входа < 1000 мс
L-07	Пакет заказов сборки (очередь)	60 VU одновременно заказывают сборку, опрос статуса каждые 2 с до READY	Ошибки < 1 %; p95 заказа < 1000 мс; все 60 образов READY
L-08	Скачивание артефактов	20 VU, 2 мин; скачивание готовых образов размером 17–19 МБ	Ошибки < 1 %; p95 < 3000 мс
L-09	Длительная нагрузка (soak)	40 VU (гости 28, пользователи 8, заказчики 3, администратор 1), 15 мин	Ошибки < 1 %; p95 чтения < 500 мс; нет роста памяти и задержки
L-10a	Объём: базовая БД	Чтение каталогов, 40 запросов/с, 3 мин, стартовые данные	Ошибки < 1 %; p95 < 500 мс
L-10b	Объём: БД ×100	То же на БД ×100: 20 000 образов, 2 000 компонентов, 5 000 пользователей	Ошибки < 1 %; p95 < 500 мс

Сводные результаты прогона до оптимизации приведены в таблице Ж.6. Все 12 профилей (около 879 тыс. запросов) выполнены без HTTP-ошибок; критерии нарушены только в профиле L-04b, целью которого был поиск предела.

Таблица Ж.6 – Сводные результаты нагрузочного тестирования (до оптимизации)

Профиль	Запросов	RPS	Ошибки, %	p95, мс	p99, мс	SLA
L-01 Дымовой (baseline)	320	5,7	0,0	111,5	194,5	PASS
L-02 Просмотр каталогов (гости)	7 772	30,3	0,0	15,3	17,2	PASS
L-03 Смешанный продакшен-профиль (целевая нагрузка)	22 489	60,3	0,0	17,5	34,4	PASS
L-04 Стресс (поиск предела)	176 498	343,8	0,0	15,0	16,4	PASS
L-04b Стресс до отказа (breakpoint)	560 700	1 294,4	0,0	2 949,5	3 170,0	FAIL
L-05 Всплеск (spike)	55 548	292,9	0,0	15,4	18,0	PASS
L-06 Шторм авторизаций	9 618	42,4	0,0	987,9	1 049,8	PASS
L-07 Пакет заказов сборки (очередь)	3 059	16,1	0,0	35,7	157,3	PASS
L-08 Скачивание артефактов	2 256	19,6	0,0	103,9	234,6	PASS
L-09 Длительная нагрузка (soak)	23 230	26,4	0,0	18,6	73,4	PASS
L-10a Объём: базовая БД	8 845	51,5	0,0	9,2	10,3	PASS
L-10b Объём: БД ×100	8 815	50,9	0,0	27,7	31,8	PASS

Очередь сборок под нагрузкой (таблица Ж.7) работает строго последовательно, по одной сборке за раз, без потерь и сбоев; ожидание в очереди в 3–4 раза дольше самой сборки (около 3,2 с в режиме симуляции). Время до статуса READY отсчитывается от ответа на заказ.

Таблица Ж.7 – Очередь сборок под нагрузкой

Профиль	**Го-тово**	**Сбое в**	**Среднее, с**	**Меди-ана, с**	**p95, с**	**Макси-мум, с**	**Сбо-рок/мин**
L-01 Дымовой (baseline)	6	0	4,1	4,1	4,1	4,1	6,4
L-03 Смешанный продакшен-профиль (целевая нагрузка)	116	0	12,5	14,1	16,1	18,1	18,6
L-07 Пакет заказов сборки (очередь)	60	0	98,1	98,8	181,1	189,5	19,0
L-09 Длительная нагрузка (soak)	173	0	4,4	4,0	6,3	10,1	11,8

После оптимизации те же профили были выполнены повторно; ключевые показатели до и после сведены в таблицу Ж.8. Предел пропускной способности чтения вырос с 1 468,1 до 3 829,6 запросов/с, число SQL-запросов на HTTP-запрос снизилось более чем вдвое, HTTP-ошибок по-прежнему нет. Единственные ухудшения – p95 и p99 профиля L-10b (БД ×100) в пределах разброса измерений.

Таблица Ж.8 – Сравнение до и после оптимизации

Метрика	**До**	**После**	**Изменение**
L-04b: Макс. пропускная способность, запросов/с	1 468,1	3 829,6	+160,9 %
L-04b: Медиана на ступени 1 500 ит/с, мс	2 732,2	3,2	−99,9 %
L-04b: Медиана на ступени 2 000 ит/с, мс	2 692,0	3,4	−99,9 %
L-04b: Медиана на ступени 4 000 ит/с, мс	2 138,2	826,7	−61,3 %
L-04b: Ошибки, %	0	0	без изменений
L-04b: SQL-запросов на HTTP-запрос	12,6	5,6	−55,6 %
L-04: p95 за профиль, мс	15,0	6,9	−54,3 %
L-04: p95 на ступени 600 ит/с, мс	12,4	6,3	−49,2 %
L-04: CPU mysqld средн., %	97,4	55,7	−42,8 %
L-04: CPU java средн., %	80,1	55,8	−30,3 %
L-04: SQL-запросов на HTTP-запрос	12,5	5,6	−55,2 %
L-10a: p95, мс (стартовая БД, 40 запросов/с)	9,2	7,9	−13,7 %
L-10b: p95, мс (БД ×100, 40 запросов/с)	27,7	29,8	+7,6 %
L-10b: p99, мс (БД ×100)	31,8	40,5	+27,1 %
L-10b: CPU mysqld средн., % (БД ×100)	30,9	28,3	−8,4 %
L-10b: SQL-запросов на HTTP-запрос (БД ×100)	17,1	5,7	−66,7 %

Метрика	До	После	Изменение
L-03: p95, мс (100 VU, целевой профиль)	17,5	5,6	−68,2 %
L-03: p99, мс	34,4	16,3	−52,7 %
L-03: Ошибки, %	0	0	без изменений
L-06: p95 входа, мс (до 100 входов/с)	987,9	121,5	−87,7 %
L-06: Ошибки, %	0	0	без изменений

Ж.4 Реестр дефектов

По результатам интеграционного и нагрузочного тестирования выявлено 22 дефекта: 1 критический, 4 высоких, 9 средних и 8 низких. Реестр приведён в таблице Ж.9. Статус «Исправлен» означает, что дефект устранён при оптимизации и подтверждён повторным прогоном; остальные дефекты на момент составления отчёта остаются открытыми.

Таблица Ж.9 – Реестр дефектов

ID	Серьёзность	Описание	Тесты	Статус
DEF-07	Критическая	Секрет JWT захардкожен в JwtService и лежит в Git: любой, кто видел репозиторий, может выпустить токен администратора	INT-15	Открыт
DEF-05	Высокая	Некорректный/поддельный/просроченный JWT приводит к исключению в JwtAuthenticationFilter (ответ 403/500 вместо 401), в лог пишется стектрейс	INT-12, INT-13	Открыт
DEF-16	Высокая	Нет проверки владельца при скачивании: любой авторизованный пользователь скачивает чужие образы по id (IDOR)	INT-33	Открыт
DEF-19	Высокая	Очередь сборок хранится только в памяти: после перезапуска или падения backend образы навсегда остаются в QUEUED/BUILDING и не могут быть изменены или удалены	INT-45	Открыт
DEF-21	Высокая	Нет ограничения частоты попыток входа: за 4 мин с одного адреса принято 9 618 входов; при 100 попытках/с BCrypt занимает около 8 ядер, вход остальных замедляется в 11 раз	L-06	Исправлен: 429 после 5 неудач (INT-48)
DEF-02	Средняя	Разрешённый CORS-origin захардкожен (http://localhost:5173) в WebConfig	INT-05	Открыт
DEF-03	Средняя	Неверный логин/пароль возвращает 403 без тела вместо 401	INT-10, E2E-08	Открыт
DEF-04	Средняя	Запрос без токена/с недействительным токеном получает 403 вместо 401 – фронтенд (interceptor по 401) не выкидывает пользователя на страницу входа	INT-11, INT-34, E2E-	Открыт

ID	Серьёзность	Описание	Тесты	Статус
			06	
DEF-08	Средняя	Запрос с токеном удалённого пользователя: UsernameNotFoundException в фильтре → 403/500	INT-17	Открыт
DEF-10	Средняя	Ошибки разбора параметров у анонимного пользователя возвращают 403 вместо 400 (ошибка уходит на /error, закрытый Spring Security)	INT-20	Открыт
DEF-12	Средняя	N+1 запросов: список из 12 образов порождает десятки SQL-запросов (ленивые components и dependencies грузятся поштучно)	INT-23	Исправлен: 389 → 11 SQL (L-10b повторно)
DEF-20	Средняя	При недоступной БД запрос висит ~30 с (connection-timeout Hikari по умолчанию), анонимный клиент получает 403 вместо 503	INT-46	Исправлен: 503 за 3 с (INT-46)
DEF-22	Средняя	Чтение ограничено около 1 300 запросов/с из-за БД (около 12,5 SQL на запрос, пул 10 соединений); при превышении запросы копятся до 2–3 с вместо быстрого отказа 503	L-04, L-04b	Исправлен: предел 1 470 → 3 830 rps
DEF-24	Средняя	Очередь с одним воркером без лимита заказов и без ETA: в L-07 60 заказов ждали до 189 с; при реальной сборке 1–3 ч ожидание составит 60–180 ч	INT-36, L-07	Открыт
DEF-01	Низкая	server.mjs отдаёт index.html со статусом 200 вместо 404 для отсутствующих /assets/*	INT-03	Открыт
DEF-06	Низкая	Срок жизни JWT 4 ч вместо заявленных 24 ч (1000*60*24*10 мс)	INT-14	Открыт
DEF-11	Низкая	Символы % и _ в поиске не экранируются и работают как шаблоны LIKE (поиск «%» возвращает весь каталог)	INT-21	Открыт
DEF-13	Низкая	Имя компонента длиннее 255 символов даёт 409 с вводящим в заблуждение сообщением «запись используется в других объектах» вместо 400 валидации	INT-27	Открыт
DEF-15	Низкая	Для образов с кириллическим названием теряется заголовок Content-Disposition: файл сохраняется без имени и расширения (интерфейс не затронут)	INT-32	Открыт
DEF-17	Низкая	Если файл артефакта пропал с диска, карточка продолжает показывать «Скачать» (downloadable=true), а скачивание даёт 400	INT-40	Открыт
DEF-18	Низкая	После смены собственного email старый JWT (sub=старый email) перестаёт работать – админ внезапно получает ошибки вместо перелогина	INT-43	Открыт
DEF-23	Низкая	Нет индексов для поиска, фильтра по статусу и сортировки по цене; на 20 тыс. образов page=1500 замедлилась в 7,5 раза, LIKE '%…%' – полный проход	L-10b	Частично: индексы добавлены, поиск по подстроке – полный проход

ПРИЛОЖЕНИЕ И
ОСНОВНЫЕ ЛИСТИНГИ ПРОГРАММНОГО КОДА

Листинг И.1 – Класс SecurityConfig (правила доступа)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final AuthenticationProvider authenticationProvider;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**").permitAll()
                        // Каталоги доступны для чтения без авторизации.
                        .requestMatchers(HttpMethod.GET, "/api/components/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/component-types/**").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/images").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/images/*").permitAll()
                        // Заказ сборки, скачивание и мутации требуют авторизации.
                        .anyRequest().authenticated()
                )
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .authenticationProvider(authenticationProvider)
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Листинг И.2 – Класс JwtAuthenticationFilter

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(
            @NonNull HttpServletRequest request,
            @NonNull HttpServletResponse response,
            @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }
        jwt = authHeader.substring(7);
        userEmail = jwtService.extractUsername(jwt);
        if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null)
{
            UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);
            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthentica-
tionToken(
                        userDetails,
```

```
                        null,
                        userDetails.getAuthorities()
                );
                authToken.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request)
                );
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Листинг И.3 – Класс UserService (управление пользователями)

```
/** Управление пользователями (администратор). */
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;

    /** Поиск по имени/фамилии/email + фильтр по роли, пагинация в SQL. */
    @Transactional(readOnly = true)
    public PageResponse<UserResponse> search(String search, String role, int page, int size) {
        Specification<User> spec = (root, query, cb) -> {
            List<Predicate> where = new ArrayList<>();
            if (search != null && !search.isBlank()) {
                String like = "%" + search.trim().toLowerCase() + "%";
                where.add(cb.or(
                        cb.like(cb.lower(root.get("name")), like),
                        cb.like(cb.lower(root.get("surname")), like),
                        cb.like(cb.lower(root.get("email")), like)));
            }
            if (role != null && !role.isBlank()) {
                where.add(cb.equal(root.join("roles").get("roleName"), role));
                query.distinct(true);
            }
            return cb.and(where.toArray(new Predicate[0]));
        };
        return PageResponse.from(
                userRepository.findAll(spec, PageResponse.request(page, size, Sort.by("id"))),
                UserResponse::from);
    }

    @Transactional(readOnly = true)
    public UserResponse get(Long id) {
        return UserResponse.from(getById(id));
    }

    @Transactional
    public UserResponse create(UserRequest req) {
        if (req.getPassword() == null) {
            throw new BadRequestException("Пароль обязателен");
        }
        User user = new User();
        apply(user, req);
        return UserResponse.from(userRepository.save(user));
    }

    @Transactional
    public UserResponse update(Long id, UserRequest req, String currentEmail) {
        User user = getById(id);
        if (user.getEmail().equals(currentEmail) && !req.getRoles().contains("ADMIN")) {
            throw new BadRequestException("Нельзя снять роль ADMIN с самого себя");
        }
        apply(user, req);
        return UserResponse.from(userRepository.save(user));
    }

    @Transactional
    public void delete(Long id, String currentEmail) {
```

```
        User user = getById(id);
        if (user.getEmail().equals(currentEmail)) {
            throw new BadRequestException("Нельзя удалить самого себя");
        }
        // Заказанные образы остаются в библиотеке; удаляются только связи пользователя.
        userRepository.delete(user);
    }

    private User getById(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Пользователь не найден: " +
id));
    }

    private void apply(User user, UserRequest req) {
        String email = req.getEmail().trim();
        userRepository.findByEmail(email)
                .filter(other -> !other.getId().equals(user.getId()))
                .ifPresent(other -> {
                    throw new BadRequestException("Пользователь с таким email уже существует");
                });

        Set<Role> roles = new HashSet<>();
        for (String name : req.getRoles()) {
            roles.add(roleRepository.findByRoleName(name)
                    .orElseThrow(() -> new BadRequestException("Неизвестная роль: " + name)));
        }

        user.setName(req.getName());
        user.setSurname(req.getSurname());
        user.setEmail(email);
        user.setRoles(roles);
        if (req.getPassword() != null) {
            user.setPassword(passwordEncoder.encode(req.getPassword()));
        }
    }
}
```

Листинг И.4 – Класс ComponentService (каталог компонентов)

```
@Service
@RequiredArgsConstructor
public class ComponentService {

    private final ComponentRepository componentRepository;
    private final ComponentTypeRepository componentTypeRepository;
    private final ImageRepository imageRepository;

    /**
     * Страница каталога. Поиск по названию и фильтр по типу применяются совместно
     * (AND); пагинация выполняется в SQL (LIMIT/OFFSET + COUNT).
     */
    @Transactional(readOnly = true)
    public PageResponse<ComponentDto> search(String name, Long typeId, String sort, int page, int
size) {
        Specification<Component> spec = (root, query, cb) -> {
            List<Predicate> where = new ArrayList<>();
            if (name != null && !name.isBlank()) {
                where.add(cb.like(cb.lower(root.get("name")), "%" + name.trim().toLowerCase() +
"%"));
            }
            if (typeId != null) {
                where.add(cb.equal(root.get("type").get("id"), typeId));
            }
            return cb.and(where.toArray(new Predicate[0]));
        };
        return PageResponse.from(
                componentRepository.findAll(spec, PageResponse.request(page, size,
                        PageResponse.sort(sort, Sort.by("name", "id")))),
                ComponentDto::from);
    }

    @Transactional(readOnly = true)
    public ComponentDto getDtoById(Long id) {
```

```
        return ComponentDto.from(getById(id));
    }

    @Transactional(readOnly = true)
    public Component getById(Long id) {
        return componentRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Компонент не найден: " + id));
    }

    @Transactional
    public ComponentDto create(ComponentRequest req) {
        Component c = new Component();
        apply(c, req);
        return ComponentDto.from(componentRepository.save(c));
    }

    @Transactional
    public ComponentDto update(Long id, ComponentRequest req) {
        Component c = getById(id);
        apply(c, req);
        return ComponentDto.from(componentRepository.save(c));
    }

    @Transactional
    public void delete(Long id) {
        Component c = getById(id);
        if (imageRepository.existsByComponentsContains(c)) {
            throw new BadRequestException("Компонент «" + c.getName()
                    + "» входит в образы — сначала измените или удалите эти образы");
        }
        if (componentRepository.existsByDependenciesContains(c)) {
            throw new BadRequestException("От компонента «" + c.getName()
                    + "» зависят другие компоненты — сначала уберите эту зависимость");
        }
        componentRepository.delete(c);
    }

    private void apply(Component c, ComponentRequest req) {
        ComponentType type = componentTypeRepository.findById(req.getTypeId())
                .orElseThrow(() -> new ResourceNotFoundException("Тип компонента не найден: " +
req.getTypeId()));
        c.setName(req.getName());
        c.setDescription(req.getDescription());
        c.setCost(req.getCost());
        c.setType(type);

        Set<Component> deps = new HashSet<>();
        if (req.getDependencyIds() != null) {
            for (Long depId : req.getDependencyIds()) {
                if (c.getId() != null && depId.equals(c.getId())) {
                    throw new BadRequestException("Компонент не может зависеть сам от себя");
                }
                Component dep = componentRepository.findById(depId)
                        .orElseThrow(() -> new ResourceNotFoundException("Зависимость не найдена:
" + depId));
                if (c.getId() != null && dependsOn(dep, c.getId())) {
                    throw new BadRequestException("Циклическая зависимость: «" + dep.getName()
                            + "» уже зависит от «" + c.getName() + "»");
                }
                deps.add(dep);
            }
        }
        c.setDependencies(deps);
    }

    /** true, если from транзитивно зависит от компонента targetId. */
    private boolean dependsOn(Component from, Long targetId) {
        Set<Long> seen = new HashSet<>();
        Deque<Component> queue = new ArrayDeque<>(List.of(from));
        while (!queue.isEmpty()) {
            Component cur = queue.poll();
            if (cur.getDependencies() == null) continue;
            for (Component dep : cur.getDependencies()) {
                if (dep.getId().equals(targetId)) return true;
                if (seen.add(dep.getId())) queue.add(dep);
            }
```

```
        }
        return false;
    }
}
```

Листинг И.5 – Класс PageResponse (постраничные ответы)

```
/**
 * Страница списка: элементы + сведения для пагинатора на фронтенде.
 */
@Data
@AllArgsConstructor
@NoArgsConstructor
public class PageResponse<T> {
    public static final int MAX_SIZE = 100;

    private List<T> content;
    private int page;
    private int size;
    private long totalElements;
    private int totalPages;

    public static <E, T> PageResponse<T> from(Page<E> page, Function<E, T> mapper) {
        return new PageResponse<>(page.getContent().stream().map(mapper).toList(),
                page.getNumber(), page.getSize(), page.getTotalElements(), page.getTotalPages());
    }

    /**
     * Сортировка из параметра запроса: price_asc / price_desc — по стоимости
     * (при равной цене — по умолчанию), иначе — сортировка по умолчанию.
     */
    public static Sort sort(String sort, Sort byDefault) {
        if ("price_asc".equals(sort)) return Sort.by(Sort.Direction.ASC, "cost").and(byDefault);
        if ("price_desc".equals(sort)) return Sort.by(Sort.Direction.DESC, "cost").and(byDe-
fault);
        return byDefault;
    }

    /** Запрос страницы с защитой от отрицательного номера и слишком большого размера. */
    public static Pageable request(int page, int size, Sort sort) {
        return PageRequest.of(Math.max(page, 0), Math.min(Math.max(size, 1), MAX_SIZE), sort);
    }
}
```

Листинг И.6 – Модуль api/client.js (HTTP-клиент)

```
// Базовый URL: в dev используется vite-прокси (/api). Можно переопределить
// через VITE_API_URL (например, http://localhost:8080/api).
const baseURL = import.meta.env.VITE_API_URL || '/api';

const client = axios.create({ baseURL });

// Подставляем JWT в каждый запрос.
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// При 401 — сбрасываем сессию и уводим на вход.
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      if (!window.location.pathname.startsWith('/login')) {
        window.location.assign('/login');
      }
    }
    return Promise.reject(error);
```

```
  }
);

export default client;
```

Листинг И.7 – Модуль App.jsx (маршрутизация)

```
export default function App() {
  const { loading } = useAuth();

  if (loading) {
    return <div className="page-loading">Загрузка…</div>;
  }

  return (
    <>
      <Header />
      <main className="app-main">
        <Routes>
          <Route path="/" element={<Navigate to="/library/images" replace />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/library/images" element={<ImageLibrary />} />
          <Route path="/library/components" element={<ComponentLibrary />} />
          <Route
            path="/editor/image"
            element={
              <ProtectedRoute>
                <ImageEditor />
              </ProtectedRoute>
            }
          />
          <Route
            path="/editor/component"
            element={
              <ProtectedRoute adminOnly>
                <ComponentEditor />
              </ProtectedRoute>
            }
          />
          <Route
            path="/admin/users"
            element={
              <ProtectedRoute adminOnly>
                <UserManagement />
              </ProtectedRoute>
            }
          />
          <Route
            path="/account"
            element={
              <ProtectedRoute>
                <Account />
              </ProtectedRoute>
            }
          />
          <Route path="*" element={<Navigate to="/library/images" replace />} />
        </Routes>
      </main>
    </>
  );
}
```

Листинг И.8 – Хук usePagedList.js (постраничные списки с автообновлением)

```
// Постраничный список с поиском/фильтрами.
// fetchPage(params) → axios-ответ с { content, totalPages, totalElements }.
// filters — объект { search, type, ... }; пустые значения в запрос не попадают.
// При смене фильтров список возвращается на первую страницу.
export default function usePagedList(fetchPage, filters, { size = 12, pollMs = 0 } = {}) {
  const key = JSON.stringify(filters);
  const [page, setPage] = useState(0);
```

```
  const [data, setData] = useState({ content: [], totalPages: 0, totalElements: 0 });
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const fetchRef = useRef(fetchPage);
  fetchRef.current = fetchPage;
  const seq = useRef(0); // отбрасываем устаревшие ответы

  useEffect(() => { setPage(0); }, [key]);

  const load = useCallback(async () => {
    const my = ++seq.current;
    const params = { page, size };
    Object.entries(JSON.parse(key)).forEach(([k, v]) => {
      if (v !== '' && v != null) params[k] = v;
    });
    try {
      const { data } = await fetchRef.current(params);
      if (my !== seq.current) return;
      setData(data);
      setError('');
    } catch (e) {
      if (my === seq.current) setError(extractError(e, 'Не удалось загрузить список'));
    } finally {
      if (my === seq.current) setLoading(false);
    }
  }, [key, page, size]);

  useEffect(() => {
    const t = setTimeout(load, 250); // debounce ввода в поиск
    const i = pollMs ? setInterval(load, pollMs) : null;
    return () => { clearTimeout(t); if (i) clearInterval(i); };
  }, [load, pollMs]);

  // Если после удаления текущая страница опустела — шаг назад.
  useEffect(() => {
    if (page > 0 && page >= data.totalPages) setPage(Math.max(data.totalPages - 1, 0));
  }, [page, data.totalPages]);

  return {
    items: data.content, page, totalPages: data.totalPages, totalElements: data.totalElements,
    loading, error, setPage, reload: load,
  };
}
```